Machine learning and exact preprocessing for Feynman integral reduction with Kira

Rushil Saraswat

October 17, 2026

Abstract

Integration-by-parts reduction expresses Feynman integrals in a master basis. We train models to select seeds and equations before Kira performs the exact algebra, using complete reduction time as the cost. We also test new models that jointly select seed boundaries and Kira settings. Validation retains the preceding ML application for both tested graphs because the replacements do not reach the required 2% improvement. On 24 reserved planar and nonplanar two-loop requests, the retained workflow gives 18.0% less time than tuned native Kira. A derived scale identity reuses completed reductions when all invariants change by a common positive factor. Four three-scale sequences give 64.4% less time than the preceding application without reuse, with the first reduction included. All timed outputs pass exact coefficient checks. These tests use rational kinematics, symbolic dimension, and supplied master bases; timings include decisions, algebra, checks, and retries. Earlier equation-filter tests give 4.8% less time on 68 matched batches at one through four loops. Learned row selection remains omitted because its tests show no reliable gain. We report learned and exact-method gains separately within their tested physical domains.

1 Introduction

Higher-order perturbative corrections to scattering amplitudes and particle decay amplitudes require integrals over loop momenta. As the loop order increases, a calculation produces many integrals with related numerator and denominator structures, and integration-by-parts (IBP) identities allow them to be expressed in terms of a smaller set of master integrals. This reduction is an essential step in many multiloop calculations in quantum field theory [1, 2].

Kira automates the reduction by generating algebraic identities and solving the resulting linear system [3, 4]. Although the identities are exact, the amount of work depends on which equations are generated and how the system is solved. Modern Kira versions already use improved seed and equation selection to reduce unnecessary work [5]. We call the requested integrals the targets and ask whether models trained on earlier reductions can reduce the time needed to express them in a master basis.

We investigate three places where a learned choice can affect the calculation: selecting the seeds from which IBP identities are generated, selecting the row used in an elimination step, and selecting individual equations before their symbolic coefficients are constructed. Earlier work has used machine-assisted search to discover seed-selection rules [6]. Here we assess each model through the time required to complete the reduction, including prediction, exact checks, and any repeated calculation after an unsuccessful choice. This criterion matters because an accurate prediction or a smaller equation set can still lead to a slower calculation.

The main comparisons separate two levels of decision. We first train models to choose the bounds of the seed region, using measured complete reduction times as labels. Tuned fixed bounds explain nearly all of that saving. We therefore hold those bounds fixed and train shared and graph-specific classification trees to select individual equations or whole seeds inside the

region. This second comparison measures the additional effect of learned filtering. Earlier tests of row selection, exact preprocessing, and restricted high-loop recurrences provide the separate evidence needed to identify where each saving comes from.

The next comparison trains a model to choose which targets Kira reduces together. This choice changes the seed limits of each group and can trade repeated preparation for a smaller equation system. We compare its complete time with the preceding undivided workflow and with separately tuned fixed partitions.

We then train a seed boundary directly from complete Kira times. This model changes which seeds generate identities, and a second model predicts which fitted boundary to use for a target. The comparison includes tuned native Kira, a fixed sparse boundary, and the preceding application. It therefore tests both the benefit of a learned region and the additional value of selecting a region for each request. The final study tests joint choices of seed boundaries and Kira settings using complete reduction time, including retries. Neither new predictor improves enough to replace the preceding ML application. We retain that application and add exact reuse when all invariants change by the same positive factor. Separate tests measure first-request time and sequence time with the initial reduction included.

We first define the notation used in this paper and explain the reduction procedure. We then compare learned seed bounds with tuned fixed settings and use that result to motivate the new equation filters. The main results measure their added effect on complete calculation time. The final sections distinguish learned gains from exact implementation changes and state the physical scope. The appendices retain earlier model tests, wider coverage checks, integral definitions, model specifications, and derivations.

2 Feynman integrals and their reduction with Kira

2.1 Families, indices, and sectors

An integral family specifies a set of propagators and the external scalar products. Integrals in the same family differ in the integer powers assigned to those propagators [1, 4]:

$$I_{\boldsymbol{a}}(\boldsymbol{x}, d) = \int \prod_{h=1}^{L} \frac{d^d \ell_h}{i\pi^{d/2}} \prod_{i=1}^{N} D_i^{-a_i}, \qquad D_i = q_i^2 - m_i^2 + i0. \tag{1}$$

Here L is the number of loops, ℓ_h is the momentum of loop h, and N is the number of inverse propagators, including any auxiliary entries. The momentum q_i is a linear combination of loop and external momenta, and m_i is the corresponding mass. The term $i0$ specifies the Feynman prescription. We use dimensional regularization with $d = 4 - 2\epsilon$, where ϵ is the regulator, and collect the independent momentum invariants and squared masses in $\boldsymbol{x}$.

The index a_i determines how D_i enters the integrand. A positive index places it in the denominator, a zero index removes it, and a negative index places a power of it in the numerator. Auxiliary inverse propagators represent scalar products that cannot be expressed using the physical propagators alone. Integrals with the same set of positive indices belong to the same *sector* [4]. A top sector specifies a maximal declared set of positive propagator indices, and its subsectors follow by removing propagators [4]. These definitions give Kira a common way to label the integrals and organize the equations between them.

2.2 From an IBP identity to a master-integral reduction

An IBP identity starts from an index vector $\boldsymbol{\sigma}$, called a *seed*. We multiply its integrand by a loop or external momentum v, differentiate with respect to a loop momentum ℓ_k, and integrate the

total derivative. Dimensional regularization gives [2]

$$0 = \int \prod_h \frac{d^d \ell_h}{i\pi^{d/2}} \frac{\partial}{\partial \ell_k^\mu} \left(v^\mu \prod_i D_i^{-\sigma_i} \right) = \sum_j A_{\boldsymbol{\sigma},o;j}(\boldsymbol{x}, d) I_{\boldsymbol{a}^{(j)}}. \tag{2}$$

The label o specifies the derivative through the choices of k and v, while j labels the integrals produced when the derivative is expanded. The coefficient $A_{\boldsymbol{\sigma},o;j}$ multiplies integral $I_{\boldsymbol{a}^{(j)}}$ in that identity. Since several operators can act on the same seed, one seed generally generates several equations. Appendix I derives the shifts in the integral indices and the row operation used to eliminate an integral.

Kira generates these equations within prescribed index limits, applies symmetry relations, removes redundant equations, and solves the remaining system [4, 5]. Each equation forms a row of the coefficient matrix, and each integral corresponds to a column. Elimination combines rows to remove integrals in the chosen order; back-substitution then expresses the requested targets in terms of the remaining masters. Kira can manipulate symbolic rational functions directly or use exact arithmetic modulo a prime, with reconstruction when a symbolic result is required [4, 7]. Arithmetic modulo a prime takes place in a finite field, so its numerical appearance does not introduce floating-point error.

A requested target may be a single indexed integral or a linear combination $T = \sum_u c_u I_{\boldsymbol{a}^{(u)}}$. The desired result is

$$T = \sum_{m=1}^{n_M} r_{Tm}(\boldsymbol{x}, d) M_m, \qquad r_{Tm} \in \mathbb{Q}(\boldsymbol{x}, d). \tag{3}$$

Here c_u is the coefficient of a target term, M_m is master integral m, and n_M is the number of masters. The field $\mathbb{Q}(\boldsymbol{x}, d)$ consists of rational functions of the invariants, squared masses, and dimension with rational-number coefficients. Reduction determines the coefficients r_{Tm}; evaluating the master integrals is a separate part of the amplitude calculation.

2.3 Where learned choices can reduce the work

Table 1 identifies the three decisions studied here. Seed selection changes which groups of identities are available, row selection changes the elimination steps, and equation selection can suppress an individual seed–operator pair before Kira constructs its coefficients. We use *native choices* to mean Kira's own decision rules. A learned model changes a decision, while Kira continues to carry out the exact algebra.

Every completed target must reach the prescribed master basis. In the comparisons, we also check its coefficients against an independently obtained exact reduction. If a chosen seed set or equation set is insufficient, the calculation repeats with native choices or a larger seed region. We call this repeated attempt a *fallback* and include its cost in the reported time. Thus the relevant question for each model is whether the work it avoids exceeds the cost of prediction and any fallback.

Table 1: The three decisions and the work each model can avoid.

Model	Decision	Possible saving
Seed and batch selection	Which seed regions and target groups should Kira use?	Limit equation generation and repeated work.
Row selection	Which allowed row eliminates the next integral?	Reduce intermediate expressions and operations.
Equation selection	Which seed–operator pairs are useful?	Avoid constructing unnecessary coefficients.

The following comparisons retain native row elimination. The original row network took 1.305 times the stock reduction time in 84 exact trials, while a later compiled row policy gave a ratio of

0.982 with interval $[0.959, 1.003]$. We therefore omit learned row choice and focus on seed regions and equations. Appendix B.2 gives the models and checks behind that decision.

3 Learning seed bounds from complete reduction time

The broader test found that the earlier models often kept native choices and provided little additional benefit. We therefore train a new model for a decision that directly changes equation generation: the seed limits for the requested target batch. The model chooses these limits before Kira starts. All earlier seed, row, and equation models are disabled in this comparison, and every method uses the same exact preparation. The fixed-control comparison measures the extra benefit of a learned choice after tuning the seed settings.

3.1 What the model learns and how Kira uses it

A target batch can need a larger numerator bound even when its denominators require no further increase. Other batches can use fewer seeds in lower sectors. Kira provides bounds on positive powers, numerator rank, and extra denominator powers, as well as a separate restriction for lower sectors [5]. These settings define ten candidate configurations in the present study. They include unshifted bounds, selective expansion, reduced bounds, full expansion, and four configurations with lower sector truncation. Appendix AB specifies every configuration and its recovery sequence.

The model receives 27 quantities that describe the graph and target indices. For each target $\boldsymbol{a}$, the extra denominator powers are $\delta(\boldsymbol{a}) = \sum_i \max(a_i - 1, 0)$, and the numerator rank is $\rho(\boldsymbol{a}) = \sum_i \max(-a_i, 0)$. The first sum counts denominator powers beyond one, often called dots, and the second is the total numerator power. Together with the loop count and denominator structure, these quantities help the model predict the relative complete time of each configuration. Its training labels come from actual Kira reductions, including any failed first attempt and the subsequent recovery. The prediction therefore concerns the cost of reaching a checked answer, rather than the number of selected seeds alone. Related work uses learned search to discover sparse seed patterns [6, 8]; the present experiment uses Kira's existing seed-region controls and does not implement tube seeding. The application checks the ordered momentum structure and the target-count and overall index-power ranges recorded in training and validation. Inputs outside this domain use native bounds and the same exact recovery procedure.

We compare 30 fitted models: regression trees, random forests, extremely randomized trees, and multilayer perceptrons [9, 10, 11, 12]. Each model predicts ten costs. Validation selects a random forest with a gate on predicted log cost of 0.0. The selected predictor uses 48 trees, maximum depth 8, and at least 2 training cases per leaf. The model receives no reference coefficients, case identifier, or kinematic profile number. Its inputs and fitted parameters remain fixed during the reserved test. Details of the objective, model choices, and selection rule appear in Appendix AB.

For each request, the application loads the fitted parameters, calculates the features, and selects a configuration. Kira then generates IBP equations and performs exact algebra in the supplied master basis. If the chosen seed region cannot reach that basis, the application tries the unshifted bounds and then full expansion, skipping duplicate configurations. A request that exhausts these attempts remains unsuccessful. Coefficient comparisons occur after Kira has produced its result and are included in the reported time.

3.2 Independent inputs and controls

The input grid contains ten graph structures at one through four loops, with one to three target integrals per batch. It includes vacuum and two-point integrals, planar and nonplanar boxes,

a pentagon, a box with an external virtuality, and boxes with bubble subgraphs. We define 74 training requests, 37 validation requests, and 74 reserved requests at separate rational kinematic profiles, keeping the dimension symbolic. After reference checks, complete candidate comparisons, and removal of identity-only cases, 68 training batches and 34 validation batches supply the fitted comparison. Target positions change with the profile, but graph structures and target patterns are shared. This tests transfer to new requests within known structures, not arbitrary new topologies or general symbolic kinematics.

Each reserved request first receives a reference reduction and a second exact check with a larger numerator bound. These preparation calculations provide a common basis and comparison coefficients. Agreement after one enlarged seed region does not prove that this basis has the smallest possible size. They remain outside the timed application, whose purpose is reduction in an available basis. The tests therefore include native IBP generation but do not include finding the master basis from the beginning. Some target patterns coincide when a family has no auxiliary numerator. Each declared batch remains a separate workload request, and the uncertainty calculation across graphs keeps such related requests together.

The first control uses Kira's native equation and elimination choices with the unshifted target bounds and the common expansion fallback. This is a specified seed policy, not a claim that Kira has a universal default seed region. The stronger fixed control chooses the cheapest of the same ten configurations separately for each graph, using both training and validation costs. It performs no model prediction and can use lower sector truncation. The learned method may change its choice between target batches in the same graph. All three methods use the same executable, exact optimizations, one Kira worker, basis, coefficient checks, and resource limits. Five repetitions rotate their order on the same node. The timer includes model loading, prediction, equation generation, exact elimination, retries, verification, and cleanup. Initial Python imports occur before this timer.

3.3 Measured benefit and remaining limits

Of the 74 reserved requests, 68 supply matched nontrivial reductions for all three methods and all repetitions. The record also contains 4 failed reference checks, 0 incomplete timed requests, and 2 matched identity-only requests. The full timed record contains 1050 exact completions in 1050 requests and 23295 coefficient comparisons. Failed or identity-only requests do not enter the paired speedup estimate.

Summing the median time of each matched batch gives 164.926 seconds for the control with native bounds, 119.717 seconds for the tuned fixed control, and 119.081 seconds for the learned choice. The learned method therefore takes 27.8% less time than the control with native bounds and 0.5% less time than the tuned fixed control. The corresponding speedup factors are 1.385 and 1.005. Table 2 gives the graph results, including slower groups. These are complete application-time comparisons on the same inputs; they are separate from the earlier exact-code speedups.

The learned-to-control time ratios have descriptive 95% intervals $[0.625, 0.885]$ against native bounds and $[0.980, 1.006]$ against the tuned fixed settings when whole graphs are resampled. Resampling repetitions within the fixed workload instead gives $[0.716, 0.727]$ and $[0.984, 1.006]$. The first calculation describes variation across this small graph collection; the second describes observed timing variation on these particular inputs. Neither interval proves a speedup on all integrals.

The summed generated-equation count changes from 2,029,994 under native bounds to 1,089,074 under learned choices, including equations from failed attempts. Average parameter loading and prediction take 7.067 and 0.836 milliseconds per learned request. Figure 2 shows both the time ratios for each batch and the equation counts. The domain check and its time remain part of every learned request. The largest observed process-tree memory use is 946.8 MiB within the common limit.

Table 2: Sum of batch median times in seconds for the new seed-bound comparison. Every row uses matched nontrivial reductions. The fixed control uses one configuration per graph.

Graph	Loops	Batches	Native bounds	Fixed	Learned
Massive sunrise	2	8	3.90	3.87	3.93
Planar double box	2	8	15.22	13.56	13.40
Massive vacuum	3	8	6.35	6.42	6.55
Massive three-loop ladder	3	8	37.72	21.21	21.18
Nonplanar box	2	8	31.74	27.77	28.00
Pentagon	1	8	5.13	5.14	5.17
Box with virtual leg	1	8	3.95	3.96	4.02
Box with bubbles, 3 loops	3	6	12.20	6.57	6.59
Box with bubbles, 4 loops	4	2	43.32	27.16	26.68
Massless two-point, 4 loops	4	4	5.40	4.06	3.55

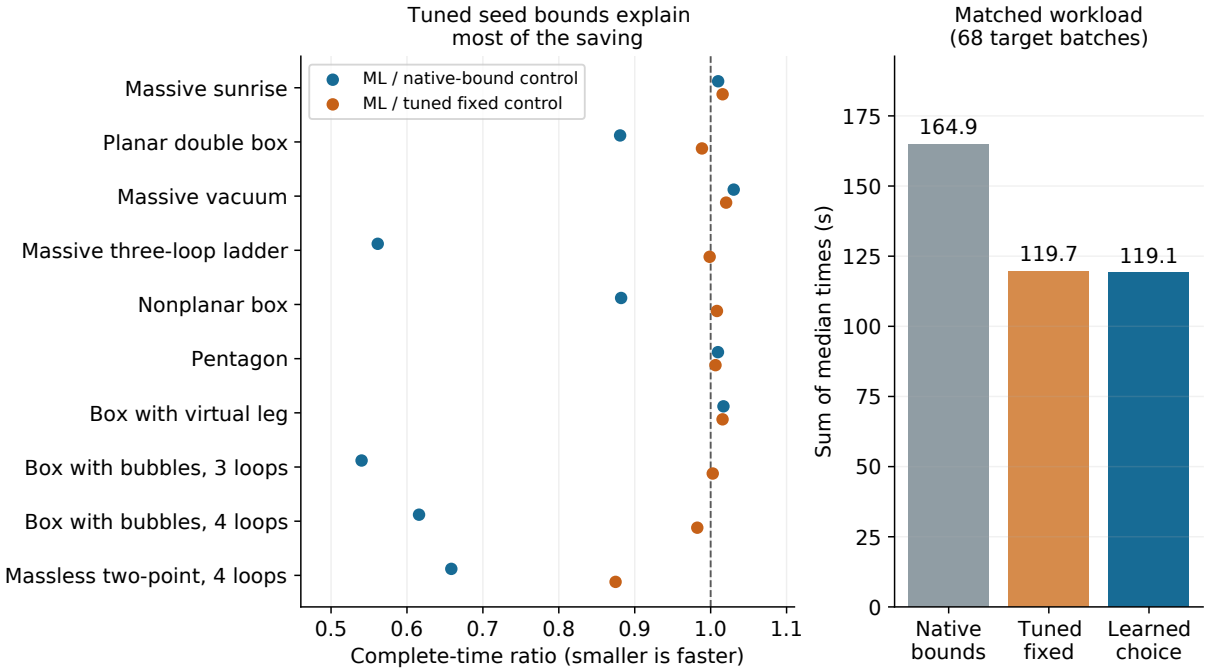


Figure 1: The left panel shows how the learned choice changes complete time for each graph; a ratio below one means a saving. The right panel shows the summed matched workload time. Both controls disable all learned choices. The graph labels refer to the explicit denominator definitions in the appendices.

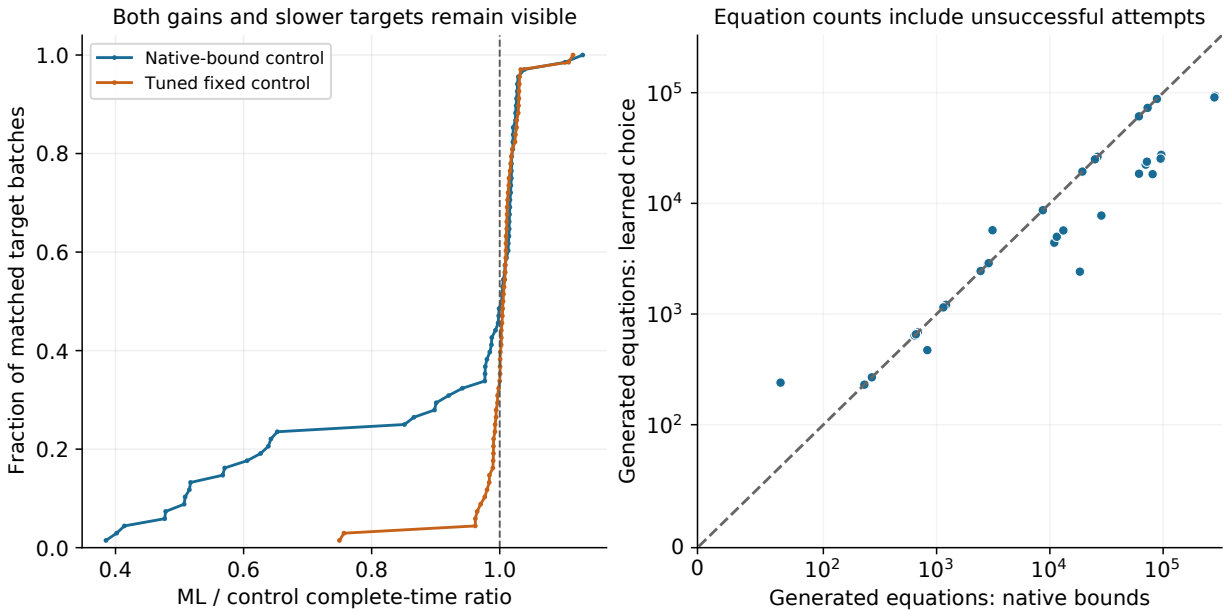


Figure 2: Left: the distribution of batch time ratios retains every matched batch, including regressions. Right: generated equations under the learned and native-bound choices, with retries included. Points below the diagonal use fewer equations. The axes use a linear scale near zero and a logarithmic scale above 100 equations. Equation count explains part of the timing change but does not replace the complete-time measurement.

Development, training-cost collection, validation, and their reference calculations take 1.37 hours of recorded calculation time, including 5.68 seconds for model fitting and selection. If future requests have the same mean saving as this matched workload, approximately 7295 requests would recover that cost relative to native bounds and 525989 relative to the fixed control. This extrapolation uses the measured workload and the derivation in Appendix AB; it is not a prediction for a different workload. The interval against the tuned fixed control includes equal time. Thus, this test establishes a saving from tuned seed settings but does not establish a reliable extra saving from learning which setting to use. The wider high-loop and symbolic coverage results retain the separate scope given in Section G.

3.4 Fresh application comparison

A separate comparison starts a new application process for every request. It uses one fixed target pattern from each graph at the first reserved profile and three repetitions. The timer includes Python imports, input parsing, model loading, calculation, result storage, process exit, and coefficient checks. On 9 matched nontrivial batches, summed median time is 29.648 seconds for native bounds, 20.913 seconds for the previous published application, and 20.797 seconds for the new model. This is 29.9% less time than the fresh native-bound application and 0.6% less time than the previous application. The previous application retains its own validated portfolio and worker choices. Its comparison therefore measures the deployed workflow as a whole; the primary comparison above isolates the new learned choice.

4 Learning which equations to construct after seed tuning

The seed-bound experiment shows that tuning a fixed configuration explains most of its time saving. We therefore keep those tuned seed limits and train a model for a finer decision: whether to construct each equation inside the selected region. This comparison asks whether learned

equation selection supplies an additional saving after the coarse seed region has already been improved.

4.1 Model and integration with exact reduction

Each candidate consists of a seed integral and one IBP or Lorentz-invariance operator. A classification tree uses the seventeen quantities in Table 37, including seed powers, target powers, distance to a target, and operator index. All these quantities are available before symbolic coefficient construction. The tree follows feature-threshold tests to a terminal node, called a leaf. The leaf score is the weighted positive fraction of its training examples, and Kira constructs the equation when that score meets the selected threshold. We also test a learned seed choice that retains every operator from a seed when at least one operator receives a score above the threshold. This uses the same trained scores and keeps more identities from each retained seed, which can prevent a failed reduction. Validation compares both uses of the model. Native symmetries, exact elimination, and back substitution remain in use. A failed attempt triggers the same tuned seed configuration with the filter disabled, followed by native and expanded bounds if needed. Thus the reported cost includes the consequences of an insufficient equation set.

The training label records whether the equation's integral support appears in Kira's final selected system. Support means the set of integrals with nonzero coefficients. This label is a proxy for usefulness and does not identify a unique necessary equation set. We collect new labels with the tuned seed settings because changing those settings changes which equations Kira can select. Collection uses a Kira build that records selected supports, and every collected reduction must agree with the independent exact reference. The timed comparisons use the same current build for both methods. The earlier filters frequently need a complete retry on these inputs, so their previous performance does not justify using them without new validation.

The four-loop box with bubbles has no completed label collection within the resource allowance, so it uses unfiltered reduction in this test. We preserve those collection failures. For the nine remaining structures, we fit eight shared classification trees and eight separate trees per graph [9, 12], with maximum depths 6, 10, 14, or 18 and minimum leaf populations 4 or 16, and a limit of 4,096 leaves. The data contain 137015 sampled equation records, including 39285 positive labels, from 66 training batches. Sampling weights preserve the native class proportions and give each batch equal total weight. The model receives no final reduction coefficients. Validation uses 33 separate batches, first to find filters that retain nearly all positive labels and then to measure complete reduction cost. A graph uses a learned seed or equation filter only when its median validation cost is at least 5% below the same tuned calculation with all ML disabled. Appendix AC specifies the fitting, threshold selection, and recovery rules.

4.2 Reserved comparison and the additional learned gain

We freeze the trees, thresholds, graph choices, and domain limits before evaluating two new rational kinematic profiles, $h = 525, 526$. The dimension remains symbolic. The ten graph structures and target patterns are the same as in Appendix AB; the new profiles supply distinct kinematic requests. Some index patterns repeat training patterns, including the planar and nonplanar boxes at these two profiles. This is a test within known structures at one through four loops. A separate comparison below changes the target-index batches as well. As in the seed-bound study, each timed request receives a common master basis, and independent reference coefficients enter only the final exact comparison.

Both methods use the same executable, tuned seed limits, native row choices, exact implementation improvements, worker count, and recovery limits. The control disables every learned selector. Five repetitions alternate the order of the two methods. Time includes policy loading, the domain check, equation prediction and generation, exact algebra, verification, retries, and

cleanup. Initial Python imports and construction of the common reference basis remain outside this repeated timer. A domain check disables filtering for an unrecognized ordered graph or target complexity outside the saved ranges.

The reserved grid has 74 requests. It supplies 68 matched nontrivial batches, with 4 failed reference checks, 0 incomplete timed batches, and 2 identity-only batches kept separately. All successful outputs pass exact coefficient comparisons: 700 completions among 700 timed requests, with 15430 comparisons. Table 3 retains each graph, including any slower result.

Table 3: Additional effect of the equation model after fixed seed tuning. Times are sums of batch medians in seconds. "Active" counts batches on which the frozen policy enables filtering. A ratio below one means less complete time.

Graph	Loops	Batches	Active	ML off	Policy	Ratio
Massive sunrise	2	8	0	4.70	4.69	0.997
Planar double box	2	8	8	18.78	15.79	0.841
Massive vacuum	3	8	8	8.07	7.55	0.937
Massive three-loop ladder	3	8	8	26.12	23.97	0.918
Nonplanar box	2	8	8	38.74	37.07	0.957
Pentagon	1	8	0	6.75	6.64	0.983
Box with virtual leg	1	8	0	4.85	4.83	0.996
Box with bubbles, 3 loops	3	6	0	9.48	9.50	1.002
Box with bubbles, 4 loops	4	2	0	30.99	31.16	1.005
Massless two-point, 4 loops	4	4	0	5.44	5.38	0.990

The full matched workload changes from 153.937 to 146.586 seconds, a time saving of 4.8%. For the 32 batches on which the frozen policy enables a filter, the corresponding times are 91.707 and 84.384 seconds, giving 8.0% less time. The 95% intervals from resampling paired repetitions are $[0.946, 0.959]$ for the full workload ratio and $[0.916, 0.927]$ for the active subset. The active subset follows the pre-test policy and is not selected from the observed test speedups. These intervals describe timing variation on the tested requests, not generalization to arbitrary graphs.

On active requests, the filters reject 36.8% of candidate construction calls, with a mean recorded filtering cost of 16.972 milliseconds per request. Repeated generation passes contribute to this count. Recovery can construct some rejected equations later, and its full cost remains in the timing comparison. The learned policy makes 5 recovery attempts in the reserved comparison. Recorded development, label collection, validation, and fitting take 39.2 minutes, excluding earlier seed-tuning costs. This expense precedes application and must be recovered over repeated use. A useful model on this collection does not establish a benefit for an unsupported topology, symbolic kinematics beyond the dimension, or all high-loop integrals. The separate wider coverage results retain their stated scope.

4.3 Complete application launches

A separate check starts a fresh application process for each request and supplies only the target definition and master basis. It uses the first supported reserved request for each graph where validation enabled filtering, with three rotated repetitions. These are a subset of the same reserved physical inputs. The timer includes Python startup, input parsing, model loading, output storage, process exit, and the exact coefficient check. Across 4 matched requests, summed median time changes from 21.835 seconds with all ML disabled to 18.946 seconds with the filter, giving a time saving of 13.2%. This comparison checks the live application overhead; it does not add new graph structures to the physical scope.

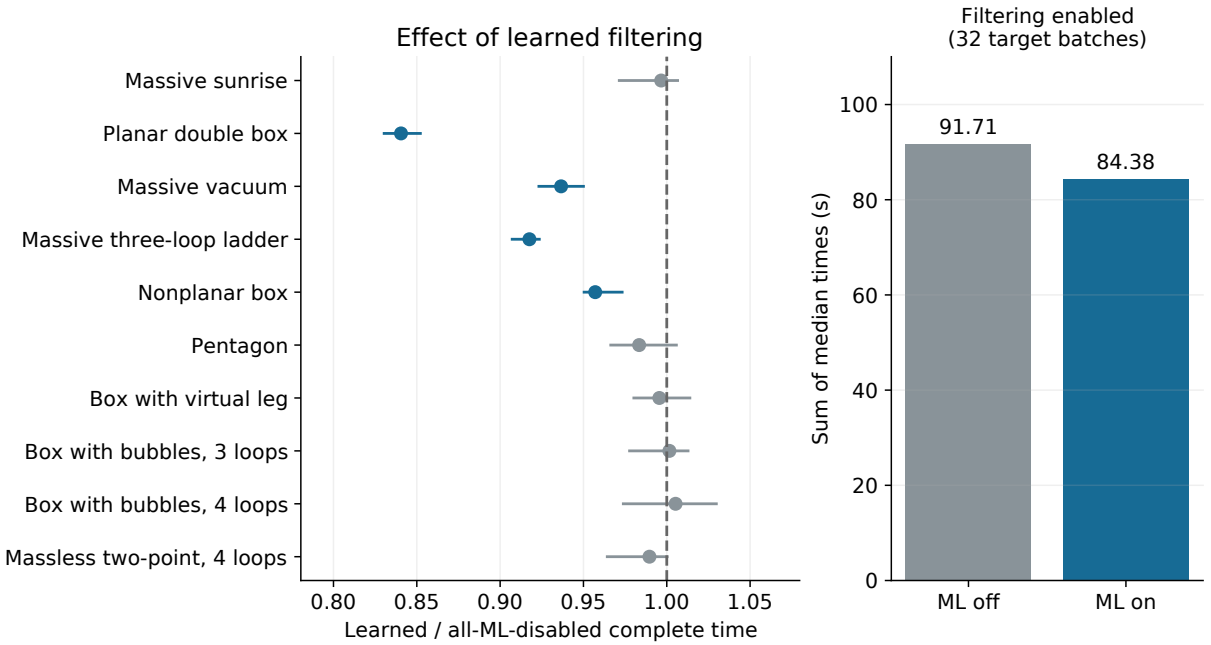


Figure 3: The left panel shows the complete-time ratio for every matched graph. Blue marks graphs with filtering enabled; grey marks graphs using the unchanged exact calculation. Horizontal intervals show the 95% range from resampling paired repetitions within each graph. The right panel isolates the batches on which the preselected policy uses the trained filter. The control shares all exact optimizations and tuned seed settings, so the difference includes only the learned policy and its consequences.

4.4 Transfer to new target-index batches

The primary comparison changes kinematics but partly repeats the index patterns seen in training. To test index transfer separately, we keep the original fitted models, thresholds, and graph choices and define profiles $h = 605, 606$. We retain a batch only if its set of index vectors is absent from both training and validation and contains at least one individual index vector absent from those sets. We also remove repeated batches within each profile. These conditions select inputs from the graph structures enabled by the original validation; no primary test timing enters this selection. The selection rules and all resulting inputs are fixed before their reductions begin. The test therefore changes the indices within known graph structures and degree ranges, while transfer to new topologies remains untested.

Each request receives the same reference reduction and enlarged-region check as the primary comparison. Three repetitions rotate the two methods. The live application reads only a target definition and a master basis. Time includes basis reading, policy loading, prediction, exact reduction, recovery, and coefficient checks; it excludes initial Python imports and reference construction. Both methods retain the same seed settings and exact implementation.

Table 4: The unchanged learned policy on new index batches. Times are sums of batch medians in seconds. The control disables every learned selector.

Graph	Batches	ML off	Policy	Ratio
Planar double box	6	14.644	14.614	0.998
Massive three-loop ladder	6	21.815	19.833	0.909
Nonplanar box	6	29.771	27.317	0.918
Massive vacuum	6	6.066	5.714	0.942

The structural rule defines 24 requests, of which 24 give matched nontrivial reductions. The record retains 0 failed reference checks, 0 incomplete comparisons, and 0 identity-only batches.

All 144 successful outputs among 144 timed requests pass exact comparison, with 5106 coefficient checks. The summed median time changes from 72.295 to 67.478 seconds, giving 6.7% less time. The 95% paired-repetition interval for the learned-to-control ratio is $[0.928, 0.940]$. Table 4 retains every matched graph, including any slower result. This check provides separate evidence about changes in target indices and is not combined with the primary timings to form a cumulative speedup.

5 Learning which targets to reduce together

The equation filters reduce work inside a seed region, but the targets grouped in one request also determine that region. A batch can contain targets with high denominator powers and other targets with high numerator rank. Their combined limits can generate equations that neither group needs separately. Dividing the batch can avoid these equations, while repeating preparation and shared algebra can make an unnecessary division slower. We therefore train a model to choose the division from measured complete reduction time. Recent tube-seeding work also uses target groups to control reduction cost [8]; the present method uses ordinary Kira seed regions and the already checked equation filters.

5.1 Model and integration with Kira

The model compares five ways to divide a batch: keep all targets together, group equal numerator ranks, group equal extra denominator powers, group equal sectors, or use consecutive groups of at most three targets. A sixth candidate keeps the batch together and disables the equation filter, allowing the new model to avoid an expensive filter or retry. It receives 32 quantities that describe the physical family, the requested powers, and their distribution across the batch. It predicts the complete relative cost of each choice before Kira generates any equations. Each selected group then uses the same fixed seed policy and native row elimination as the preceding method. The existing equation filter acts only when enabled and when the group lies inside its original checked domain. We merge the resulting exact master coefficients after checking that every target has completed.

We compare regression trees, random forests, and extremely randomized forests [9, 10, 11]. A tree compares the input quantities with learned thresholds and returns predicted costs from the final group of training examples. A forest averages the predictions of several trees. Here each prediction contains six log-cost ratios: the logarithm of a candidate’s complete time divided by the time of the undivided batch. Training uses 49 batches, and separate validation uses 49 batches. The candidate with the smallest validation cost is an extremely randomized forest with 32 trees, at most 6 threshold levels, and at least one training example in each final group. Its chosen prediction margin is 0.02 in log-cost units, as defined in Appendix AD, and its selected validation costs give 0.7% less time than fixed partitions. No predictor reaches the required three-percent validation saving, so deployment uses the fixed partitions. The appendix specifies every partition, input, training objective, recovery rule, and physical request, and derives how the independently reduced groups give the complete target reduction.

The training labels include unsuccessful attempts and recovery. Reference construction supplies a common basis and checked coefficients before timing, but the solver receives only the basis and the target definitions. Reference coefficients enter the exact check after reduction. Recorded development, training, and validation reductions take 92.6 minutes, while fitting takes 8.86 seconds; these costs precede deployment and exclude reference construction. The median loading and decision cost in the timed test is 6.73 milliseconds.

Selecting the smallest recorded cost separately for every validation batch would lower the fixed-partition cost by 4.5%. This comparison uses timing information available only after the

calculations and describes the remaining opportunity within these six candidates. It does not describe a deployable predictor. Appendix AD defines this retrospective bound.

5.2 Controls and results

The reserved comparison uses new rational profiles in the same ten structures at one through four loops, with symbolic dimension. The matched requests contain 1 to 11 targets each. Five repetitions rotate the methods through execution positions on the same worker. The previous method keeps each batch together. One fixed control chooses a partition per graph with the equation filters available; a second fixed control tunes its partitions independently with every ML model disabled. The comparison with the first fixed control measures the added batch-selection application, including its domain checks and loading cost. The second compares the combined workflow with tuned choices that perform no ML inference. All methods share the same exact implementation, basis, worker count, and checks.

Of 108 requests, 96 give matched nontrivial completions. There are 10 failed reference requests, 0 incomplete timed batches, and 2 matched identity-only batches. The comparison record contains 2450 successful results in 2450 calculations, with 90275 exact coefficient comparisons. Every successful result passes its reference check. A total of 34 requested index batches recur in this study's training or validation sets at different rational profiles. The test therefore measures new kinematics and a defined mixture of target batches within known graph structures.

The selected workflow actually calls an equation filter on 4 matched batches. On this subset it uses 7.0% less time than the separately tuned ML-disabled control. This subset records filter use; its controls can also choose different fixed partitions. Its paired-repetition ratio interval is $[0.834, 1.018]$. This interval includes equal time and does not establish a reliable saving on this subset.

Two cluster allocations reached their time limits before completing all requests. We preserve the 76 complete comparison batches and resume the 22 remaining batches with unchanged models and timing rules. For an interrupted batch, we repeat every method and repetition on one worker. The 4 saved timings from its earlier partial comparison remain available separately and do not enter the ratios. Work assignment uses validation costs only.

Summed batch median time changes from 383.760 seconds for the previous undivided workflow to 389.368 seconds for the selected workflow, giving 1.5% more time. The fixed control with filters takes 386.251 seconds, and the independently tuned all-ML-disabled control takes 389.826 seconds. Relative to these controls, the selected workflow uses 0.8% more time and 0.1% less time, respectively. The selected-to-fixed ratio has a paired-repetition interval of $[0.988, 1.035]$, and the ratio to the all-ML-disabled control has interval $[0.986, 1.025]$. Validation disables the new predictor, so this comparison establishes no additional ML gain from batch selection. Table 5 shows the separate graph results, including regressions.

The recorded generation counts sum to 2,429,859 equations for the previous workflow, 2,514,300 for the tuned ML-disabled control, and 2,494,576 for the selected workflow. These are sums of per-batch median counters over reported generation passes, including repeated passes and retries. They describe generated equations rather than the total number of arithmetic operations. Complete time remains the criterion for retaining a model.

Table 5: Sum of median complete times in seconds on matched batches. Previous keeps each batch together. Fixed uses one partition per graph with filters. ML off tunes partitions separately with every model disabled. Selected uses the frozen deployment choice.

Graph	Batches	Previous	Fixed	ML off	Selected
Three-loop bubble box	12	27.83	27.69	28.03	28.06
Four-loop bubble box	2	18.69	19.61	18.99	19.75
Planar double box	12	37.29	39.87	40.36	40.05
Massive ladder	12	98.23	103.36	102.32	102.17
Massless two-point	10	30.38	26.66	26.76	26.56
Nonplanar box	12	138.39	135.97	139.94	139.29
Virtuality box	8	5.31	5.62	5.64	5.70
Pentagon	8	6.20	6.07	6.12	5.99
Sunrise	12	13.12	12.97	12.91	13.55
Massive vacuum	8	8.31	8.41	8.76	8.23

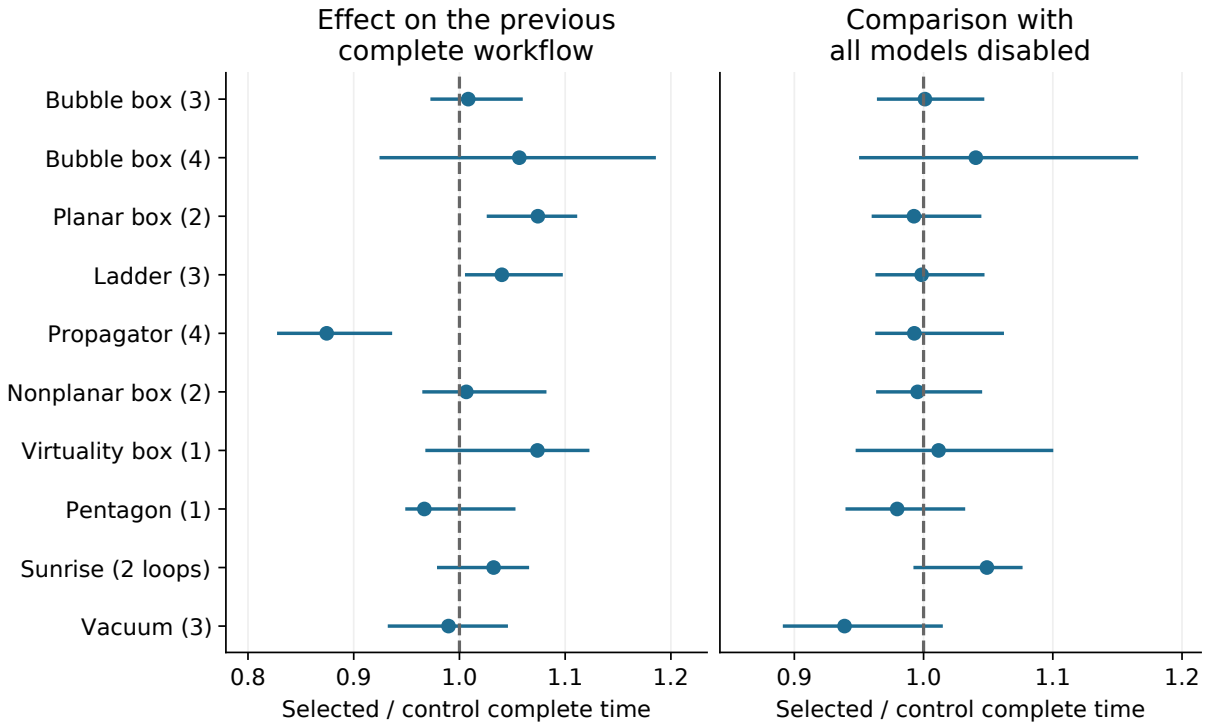


Figure 4: The left panel compares the selected complete workflow with the previous undivided workflow. The right panel compares it with independently tuned fixed partitions with every model disabled. Values below one indicate less time; the dashed line marks equal time. Segments describe paired repetition variation on this workload. Validation disables the new batch predictor; the selected workflow uses the retained fixed partitions and equation-filter settings.

A separate application check starts a new Python process for each request and includes imports, model loading, input and output, and exact checks. Deployment loads only the parts of the frozen policy that it uses: when validation disables the predictor, it retains the fixed choices and domain limits and omits the unused tree arrays. We check that this compact form gives the same choices on every training, validation, and reserved input, without reading reserved timing outcomes. The repeated comparison above retains the original frozen application; this separate check uses the compact form. It completes 36 timed requests over 4 matched batches and gives 1.2% less time than the previous workflow. These batches are a predeclared subset of the reserved inputs, so this check measures application overhead without adding evidence for new topologies.

The new study extends the batch sizes tested by the earlier equation-filter comparison. It still supplies a master basis and keeps the kinematic invariants rational. General symbolic scattering, arbitrary higher-loop structures, and globally optimal partitions remain outside this evidence. The earlier restricted vacuum recurrence covers its stated one-through-twelve-loop family separately, and its speedup cannot be multiplied by the ratios measured here.

6 Learning sparse seed regions from reduction time

The batch study leaves little additional saving among its predefined partitions. We therefore change the equations available to Kira more directly. A learned boundary selects individual seeds around the requested numerator powers before equation generation. Kira then applies its usual IBP operators to those seeds. This follows the motivation of sparse target-dependent seeding [8], while the boundary parameters in this study are fitted through complete reductions with symbolic dimension.

6.1 The seed model and its training data

The model describes a surrounding region around each target's numerator-power box. Its width can depend on the seed rank and on how many physical propagators are absent from its sector. It also controls the largest seed rank in each sector. Appendix AF defines these boundaries, derives the distance used by the model, and gives a counting example that explains why a narrow region can avoid many seeds at higher numerator rank.

We first fit the boundary parameters through evolutionary search. Each proposal runs Kira and receives a score from the measured time needed to reach the prescribed masters and pass an exact coefficient check. The score includes unsuccessful attempts and native recovery. Thus the labels describe completed calculations rather than membership in a previously selected equation set. Three training requests per graph start this search; separate validation selects the first fitted boundary. An independent planar-box comparison then tests that boundary on eight new kinematic requests.

The second stage adds twelve training requests per graph and fits models that predict the cost of different seed boundaries. These requests contain single-direction and distributed numerators of ranks two, three, and four. The candidate boundaries include those found by the first search, independent changes to the rank allowance, fixed surrounding widths, and unfiltered Kira. Trees and forests predict the log complete-time ratio of each candidate [9, 10, 11]. Ten validation requests per graph select the model, its decision margin, and a fixed sparse control. The predictor is enabled only when validation gives at least three percent less recorded cost than that fixed control. When validation disables prediction, the deployed policy uses the selected fixed boundary. Table 6 gives the selected models. The independent timing tests include the cost of loading and applying them.

Validation enables request-dependent cost prediction for the nonplanar box. Other graph choices use their fixed sparse boundary.

Table 6: Cost predictors selected before the reserved comparisons. Each graph also has a separately selected fixed sparse control.

Graph	Predictor (depth)	Training requests	Deployment
Planar box	Tree (2)	12	Fixed boundary retained
Nonplanar box	Extra trees (2)	12	Enabled

6.2 Independent reductions and controls

The first evolved planar-box boundary gives 22.3% less time, with a paired-repetition ratio interval of $[0.751, 0.813]$ relative to tuned native Kira. Relative to the fixed surrounding-width control, it gives 6.5% less time, with a paired-repetition ratio interval of $[0.899, 0.980]$. This comparison contains 168 timed reductions, and all successful outputs pass exact checks. The boundary remains fixed throughout this test.

Table 7: First independent planar-box comparison. Times are sums of per-request medians over eight requests, with seven repetitions per method. The learned boundary is fitted before this comparison.

Method	Complete time (s)
Tuned native Kira	37.893
Fixed surrounding width	31.465
Evolved seed boundary	29.424

The second comparison uses 20 matched requests from the planar and nonplanar two-loop boxes, with seven repetitions per method. All methods use the same Kira executable, exact preparation, master basis, and native row elimination. The native control disables every learned choice. The fixed sparse control uses one validation-selected boundary per graph, including boundaries discovered during training, and performs no request-dependent cost prediction. The previous application retains the earlier published batch and equation policies. These controls separate the effect of sparse seeding from the added benefit of predicting which boundary to use for a particular target.

The complete selected workflow gives 22.2% less time, with a paired-repetition ratio interval of $[0.764, 0.792]$ relative to tuned native Kira and 22.6% less time, with a paired-repetition ratio interval of $[0.762, 0.789]$ relative to the previous application. Relative to the fixed sparse control, it gives 3.6% less time, with a paired-repetition ratio interval of $[0.946, 0.981]$. Table 8 reports the measured times for both graphs. The comparison contains 560 timed reductions and 5600 exact coefficient comparisons. Failed reference requests number 0, and failed timed reductions number 0; these remain in the evidence record.

Table 8: Sum of per-request median complete times, in seconds. Native uses the tuned outer bounds with all ML disabled. Fixed uses one sparse boundary per graph. Selected uses the frozen deployment policy.

Graph	Requests	Native	Previous	Fixed	Selected
Planar box	10	20.986	21.072	15.742	15.802
Nonplanar box	10	54.857	55.100	45.462	43.169
All matched	20	75.843	76.172	61.203	58.972

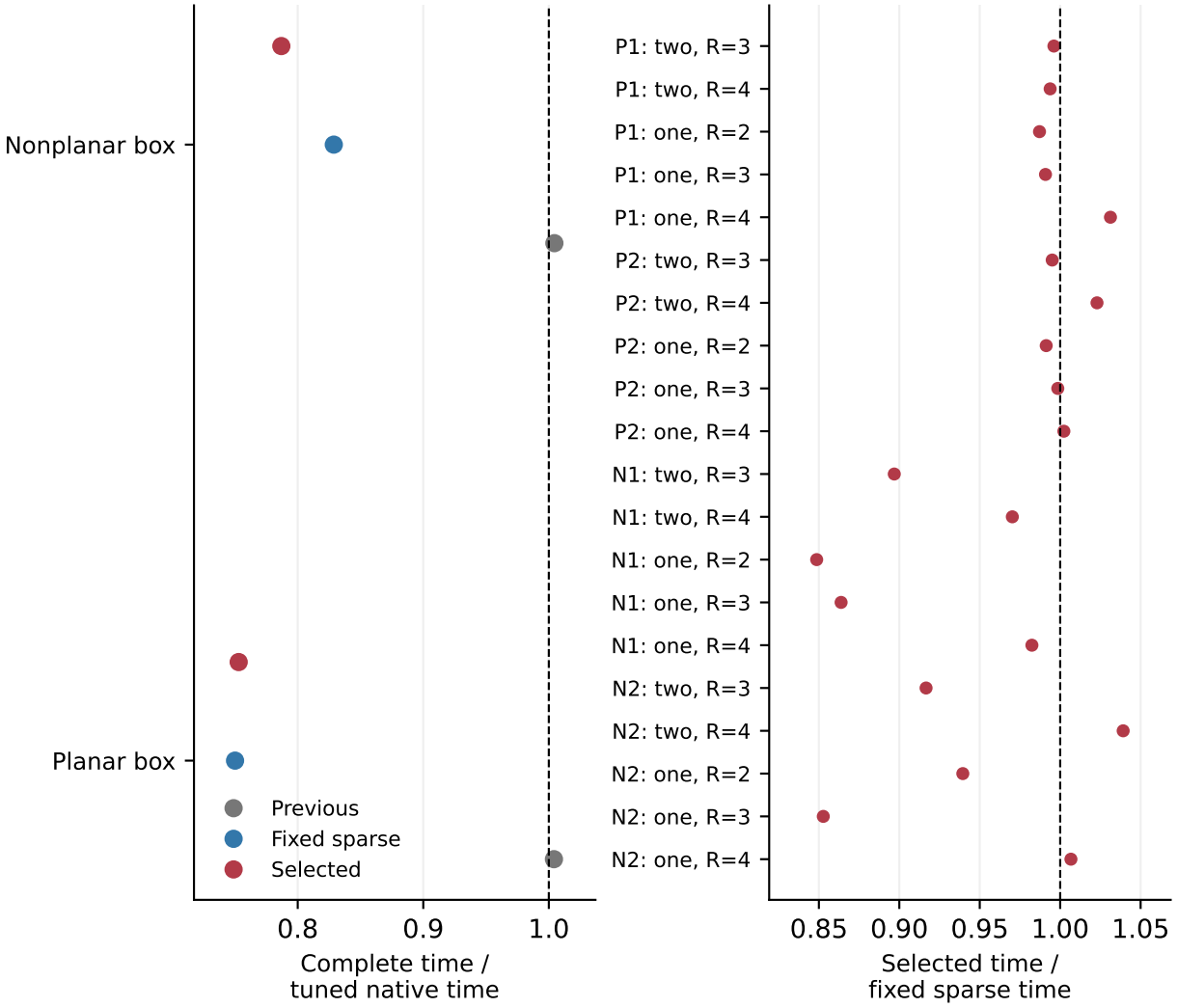


Figure 5: Left: complete-time ratios to tuned native Kira, summed within each graph. Right: the selected-to-fixed-sparse ratio for each reserved request. Labels P and N identify the planar and nonplanar graphs; 1 and 2 identify the two reserved kinematic profiles. Each label gives the number of powered numerator directions and the rank R. The dashed lines mark equal time. Values below one indicate a saving. The right panel shows the measured difference from using the fixed sparse rule.

A separate application check starts a fresh process for each request and supplies only the physical definition, target indices, and master basis. It includes imports, model loading, input and output, process exit, and a subsequent exact check. It completes 48 runs over 4 matched requests. Its selected total is 15.774 seconds, compared with 27.897 seconds for native choices and 16.187 seconds for the fixed sparse control. These requests are a predefined subset of the reserved test, so they check application overhead without adding graph coverage.

The complete integrated application also passes 24 fresh-process checks. These use two of the reserved two-loop requests, one previously tested four-loop native request, and an eight-loop massless banana request. For the two-loop requests, the summed median complete time is 8.091 seconds, compared with 17.088 seconds for the preceding complete application. The other inputs check that the existing native and analytic methods remain available and exact. They do not extend the sparse model's fitted domain.

These results concern numerator ranks two through four, unit positive physical powers, rational kinematic values, symbolic dimension, and supplied bases in two known two-loop structures. The reported production times exclude offline training and construction of the independent reference basis. Some index patterns recur between training and testing. Development probes at other loop orders do not extend the fitted model's validated domain. The application retains its earlier exact methods and native fallback outside this domain. The comparison establishes the measured effects of the new seed policy; it does not establish a universal speedup or a globally optimal reduction path.

7 Joint seed choices and exact scale reuse

The preceding study reduces unnecessary equation generation, but some sparse choices still require a repeated calculation. To reduce this cost, we train new policies that select a seed boundary and Kira configuration together, then compare them with the preceding ML application. Validation retains the preceding application for both graphs because neither replacement clears the required improvement threshold. We add an exact scaling relation that reuses a completed reduction at a different overall momentum scale. The retained ML application performs the first reduction, and the exact identity can remove later reductions with matching dimensionless kinematics.

7.1 Learning a complete reduction configuration

The candidate configurations combine sparse boundaries with Kira's integral-ordering and internal propagator-permutation options [7, 5]. The model chooses a configuration before equation generation. Kira then uses its native elimination procedure, and the earlier learned row policy remains omitted. Development reductions determine the candidate set, while separate training reductions supply the complete time of each candidate, including failed attempts and recovery. This follows the same cost-based principle as the preceding seed-boundary study; sparse seed searches also motivate the work in Ref. [8].

We compare regression trees, random forests, and extremely randomized trees [9, 10, 11]. Each predicts candidate log-time ratios from 68 features describing the integral family and target powers. Validation selects the model, its decision margin, and a fixed configuration for each graph. The model is enabled only when validation gives at least 2% less time than the fixed choice, and the new deployment must also improve on the preceding application by at least 2%. Otherwise the preceding application remains in use. The planar replacement gives 0.9% less time than the preceding application on validation. The nonplanar replacement gives 0.9% less time than the preceding application on validation. Both improvements fall below the predefined threshold, so the following tests retain the preceding ML decisions. Appendix AG specifies the candidates, inputs, training objective, and exact target definitions.

Table 9: Models selected before testing. The deployment condition uses validation times only.

Graph	Selected fitted model	Deployment
Planar	Regression tree, maximum depth 2	Previous application retained
Nonplanar	48 extremely randomized trees, maximum depth 4	Previous application retained

7.2 First-request time on separate test inputs

The test contains 24 requests from the same two massless two-loop graph structures, at new rational kinematic profiles. Targets have unit positive physical powers and numerator ranks two through four, with either one or two powered auxiliary entries. Each graph has twelve training, twelve validation, and twelve test requests. Some target index patterns recur between the splits. Five repetitions rotate the four methods on the same worker for each request, using one Kira thread. The main test performs no result reuse.

The native control disables all ML and uses the tuned outer seed bounds. The fixed control uses one validation-selected seed and ordering configuration per graph, with no prediction or model loading. The previous control uses the preceding complete application. The selected method wraps that same application with the new input checks and optional reuse interface. Since validation disabled both replacements, its first-request comparison with the previous control measures this added interface and timing variation. All methods receive the same master basis, and the timer includes input preparation, configuration selection, equation generation, exact algebra, coefficient checks, and every retry. Master discovery and offline model training precede these production timings.

Table 10 gives the sum of per-request medians. Relative to the preceding application, the selected method gives 1.2% less time, with time-ratio interval $[0.975, 1.024]$. Relative to tuned native Kira, it gives 18.0% less time, with time-ratio interval $[0.806, 0.841]$. Its comparison with the fixed control gives 5.7% less time, with time-ratio interval $[0.926, 0.966]$. These intervals use paired resampling of the five repetitions on the fixed test workload. They describe timing variation and do not establish performance for unseen graph structures. All 480 timed reductions pass 4800 exact coefficient comparisons. The retained data include individual slower requests and all failed first attempts.

Across the five repetitions, the preceding application makes 5 additional reduction attempts, the fixed control makes 45, and the selected method makes 5. These counts identify repeated algebra caused by an insufficient first choice; their costs are already included in the reported times.

The selected-to-fixed differences include configuration choices inherited from the preceding application. These comparisons therefore assess complete configurations and provide no evidence of a gain from the new predictors.

Table 10: Complete first-request times, in seconds, summed over per-request medians. There are twelve reserved requests per graph.

Graph	Tuned native	Previous	Fixed	Selected
Planar	33.082	25.449	25.326	25.406
Nonplanar	78.584	67.219	71.816	66.161
Total	111.666	92.668	97.142	91.567

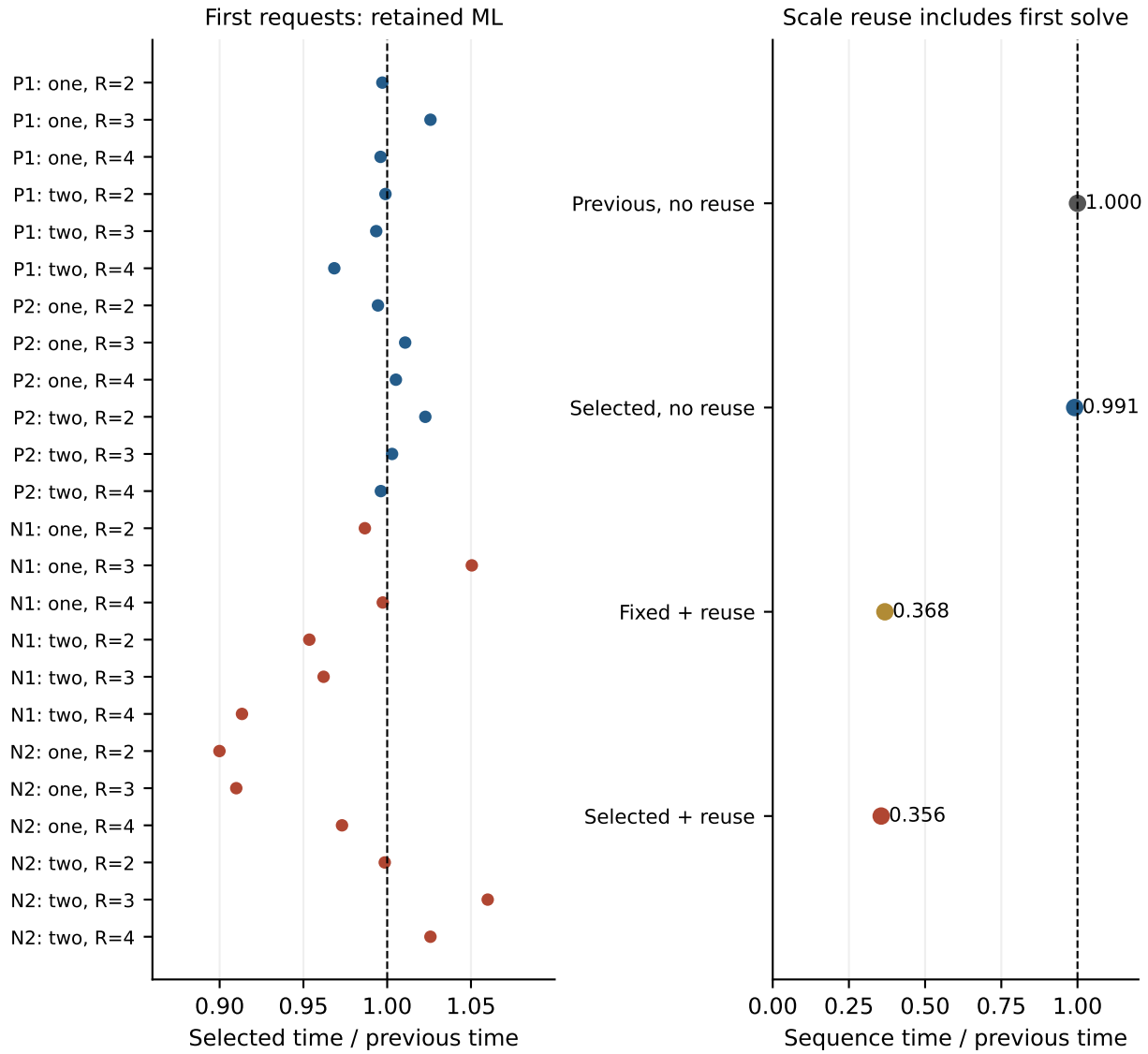


Figure 6: Left: the selected-to-previous complete-time ratio for each reserved request, including slower cases. P and N identify the planar and nonplanar graphs, 1 and 2 identify the two kinematic profiles, and the labels give the numerator pattern and rank. Right: aggregate time for the four scale sequences, normalized to the preceding application without reuse. Every sequence includes its first Kira reduction. Dashed lines mark equal time. The panels have separate workloads and their ratios cannot be multiplied.

7.3 Exact reuse when only the overall scale changes

For a massless family with homogeneous quadratic inverse propagators, a common positive change of all external scalar products gives an exact relation between reduction coefficients. If $\boldsymbol{b}_m$ is the index vector of master m, define $\nu(\boldsymbol{a}) = \sum_i a_i$. Then

$$r_{\boldsymbol{a}m}(\lambda \boldsymbol{x}, d) = \lambda^{\nu(\boldsymbol{b}_m)-\nu(\boldsymbol{a})} r_{\boldsymbol{a}m}(\boldsymbol{x}, d), \qquad \lambda > 0. \tag{4}$$

Appendix AG derives this identity by scaling the integration momenta. Its exponent is an integer, and a rational scale therefore gives an exact rational multiplier. The application checks the graph, target indices, supplied basis, and every dimensionless kinematic ratio before reusing an earlier result. Changed ratios, masses, or symbolic kinematic values continue through the reduction application. Reuse supplies no benefit before a matching exact result exists.

We test four predefined rank-four requests, two from each graph, in sequences with scale factors 1, 3/2, and 2. Each sequence starts a fresh process and includes the initial Kira reduction, imports, model loading, input and output, exact transformation, and the subsequent independent coefficient checks. Three repetitions and four methods give 48 measured sequences containing 144 requests. Independent native reductions at every scale supply the checks; their coefficients never enter the application inputs.

We sum the median sequence times over the four base requests. Under this measure, the selected policy with scale reuse uses 28.496 seconds, compared with 80.073 seconds for the preceding application without reuse and 79.334 seconds for the selected policy without reuse. The resulting change relative to the preceding application is 64.4% less time, with time-ratio interval $[0.334, 0.390]$, while reuse alone changes the selected policy's time by 64.1% less time, with time-ratio interval $[0.350, 0.383]$. A fixed configuration with the same exact reuse uses 29.466 seconds; the selected-to-fixed-reuse comparison gives 3.3% less time, with time-ratio interval $[0.770, 1.049]$. All transformed outputs pass exact checks. These sequence results measure the benefit of repeated dimensionless kinematics, and remain separate from the first-request model comparison.

The deployed application combines the retained ML application with this exact reuse and retains the existing analytic and native paths outside its fitted domain. Eight further fresh-process checks reproduce the earlier four-loop native and eight-loop analytic results. These checks preserve existing behavior and do not extend the new model's fitted range. The new timing evidence covers known two-loop boxes, rational kinematics, symbolic dimension, and supplied bases. It does not establish a fastest possible strategy for arbitrary integrals or unrestricted loop order.

8 An exact higher-loop specialization alongside the learned method

The learned choices above apply to the physical structures represented in their training and validation data. A useful reduction workflow can also use a proved integral identity when the requested family permits one. We implement this choice for massless two-point banana integrals, which have two vertices connected by $L+1$ massless lines at L loops. Repeated one-loop convolution gives their coefficients directly [13]. This saving comes from the analytic identity and is measured separately from the learned decisions.

Let p be the external momentum, with $p^2 \neq 0$, and let the physical denominators be $\ell_1^2, \ldots, \ell_L^2$ and $(p - \sum_j \ell_j)^2$. Each denominator has a positive integer power a_i. There are no numerator insertions in this specialization. Define $n = L+1$, $b_i = a_i - 1$, $K = \sum_i b_i$, and the corner integral $M_L = B_L(1, \ldots, 1)$. Appendix AE defines the measure and derives

$$B_L(\boldsymbol{a}) = \frac{(n - Ld/2)_K (n + 1 - nd/2)_K}{(p^2)^K \prod_{i=1}^n (2 - d/2)_{b_i} b_i!} M_L, \tag{5}$$

where $(z)_k = \prod_{j=0}^{k-1}(z+j)$ and $(z)_0 = 1$. Thus all scalar targets in this class have rational reduction coefficients in d and p^2.

Before applying Eq. (5), the workflow checks the propagator momenta, zero masses, nonzero external squared momentum, and allowed indices. It forms and cancels exact linear factors in d. Other families pass to the native Kira workflow with the frozen batch and equation choices. Both the ML-enabled and ML-disabled comparison paths use the same analytic identity. The identity therefore contributes no saving to the reported ML comparison.

8.1 Checks and measured time

The first native comparison asks Kira to use the corner master with seed limits set directly by the targets. It completes the one-loop requests and fails in eight higher-loop requests. These failures include rejection of the supplied basis and an allocation failure during topology preparation. We retain all of them, then compare four fixed seed choices on a separate development point and freeze the fastest successful choice for each loop count and kinematic form. A new point supplies the matched comparison in Table 11. Appendix AE gives the seed definitions and repetitions.

The follow-up completes 9 matched requests and leaves 1 request without a completed comparison. Six-loop development reports allocation failures during topology preparation under the imposed memory limit. Since no fixed seed choice passes, the six-loop reserved request does not run and receives no timing ratio. The completed requests check 81 native coefficients exactly. The complete-time speedup ranges from 21.3 to 357.8 times across the completed requests. Both paths include an exact coefficient check, and the direct path also includes its domain check. These timings exclude shared Python imports.

Table 11: Complete times in seconds for the defined scalar banana targets. Native uses the development-selected fixed seeds with all ML disabled. Direct uses the proved formula. Each native time is the median of three repetitions.

Loops	p^2	Native	Direct	Speedup
1	Rational	0.4223	0.0067	62.8×
2	Rational	0.4425	0.0194	22.8×
3	Rational	0.5373	0.0218	24.7×
4	Rational	1.4625	0.0249	58.8×
5	Rational	8.7544	0.0245	357.8×
1	Symbolic	0.4693	0.0081	58.0×
2	Symbolic	0.4966	0.0233	21.3×
3	Symbolic	0.5555	0.0246	22.6×
4	Symbolic	1.4615	0.0243	60.2×

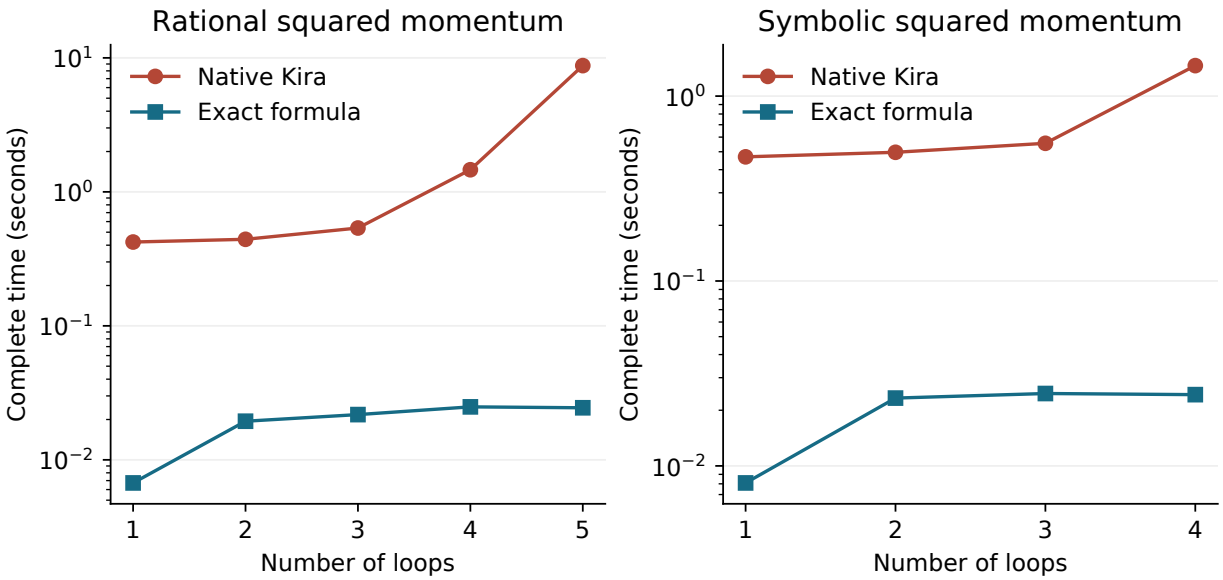


Figure 7: The direct coefficient formula avoids the native equation system for the displayed scalar family. Times include exact coefficient checks. The logarithmic time axis shows both methods, and only completed native comparisons appear. These are analytic gains, separate from ML.

An additional 240 numerical checks compare the finite rational formula with the uncancelled Gamma-function expression at one through twelve loops and at 16, 24, and 32 loops. They use 70-digit arithmetic at two noninteger dimensions. The general derivation supports this scalar family at every positive loop count; the numerical checks test its implementation at the stated counts. General higher-loop scattering remains outside this analytic specialization.

A separate test starts a new application process for each request. All 48 executions use the combined routing interface, and every successful output passes an exact check. It also passes 49 domain checks. The four analytic requests use one, four, eight, and twelve loops; their summed median application time is 0.6458 seconds. The same formula serves both comparison paths. The 4 matched native requests take 38.161 seconds with the frozen learned workflow and 38.318 seconds with the separately tuned ML-disabled workflow. These native requests repeat the predeclared application subset of the batch study, so they test integration and startup rather than new physical coverage.

9 Overall performance and physical scope

The separate analytic specialization reduces scalar massless banana targets through one corner master, with a derived coefficient valid at every positive loop count. Its 9 completed native comparisons give a speedup of 21.3–357.8 times for the stated targets. These gains come from exact mathematics and are separate from the ML gains (Section 8).

9.1 What the model comparisons establish

Section 7 retains the preceding ML application because neither new joint predictor clears the 2% validation threshold. On 24 first requests, the retained application with the new interface gives 1.2% less time, with time-ratio interval $[0.975, 1.024]$ relative to its preceding interface. This difference supplies no evidence of a gain from the new predictors. The separate scale sequences give 64.4% less time than the preceding application without reuse, including the initial Kira solve. This additional saving comes from the derived scale identity, while the first reduction still uses the preceding ML decisions.

The first evolved seed boundary gives 22.3% less time, with a paired-repetition ratio interval of $[0.751, 0.813]$ relative to tuned native Kira on eight planar two-loop requests. Its added saving relative to the fixed surrounding-width control is 6.5% less time, with a paired-repetition ratio interval of $[0.899, 0.980]$. The broader sparse-seed workflow in Section 6 gives 22.2% less time, with a paired-repetition ratio interval of $[0.764, 0.792]$ relative to tuned native Kira on 20 matched two-loop requests. Its comparison with the fixed sparse control gives 3.6% less time, with a paired-repetition ratio interval of $[0.946, 0.981]$. These results use separate workloads from the earlier equation-filter and batch tests.

The target-batch comparison in Section 5 gives 1.5% more time than the previous undivided workflow across 96 matched batches with up to 11 targets. Its additional change relative to fixed partitions with the same filters is 0.8% more time, with ratio interval $[0.988, 1.035]$. Validation disables the new predictor, so this comparison establishes no additional ML gain from batch selection. Every successful reduction passes exact coefficient checks. This is a separate workload from the smaller-batch filter study. After fixed seed tuning, newly trained equation filters give 8.0% less complete time on 32 matched batches where validation enables them, with 4.8% time saving across all 68 matched batches. These controls disable all ML and retain the same seed settings and exact implementation. Section 4 gives the failures, timing intervals, and physical limits.

The new seed-bound study in Section 3 isolates a learned choice with all earlier models disabled. On 68 matched nontrivial batches, complete application time changes from 164.926 seconds under native bounds and 119.717 seconds under tuned fixed settings to 119.081 seconds with the new model. This gives 27.8% less time than native bounds and 0.5% less time than the tuned fixed control. The comparison includes model loading, native IBP generation, exact reduction, coefficient checks, and recovery, with a supplied basis. It uses known structures at one through four loops and separate rational profiles; the additional saving over tuned fixed settings remains within timing variation. The broader coverage study in Section G defines 90 native-family requests through twelve loops. The current interface accepts 54 of them, and 39 complete three checked nontrivial full reductions with revised preparation. On 37 matched completed requests, the revised full calculation takes 88.2% less time than the saved original Kira executable. The supplied-basis selected application takes 0.1% less time than the revised native control on its separate matched set. The approximately 0.1% difference from the revised native control establishes no reliable additional learned gain on this wider set. Unsupported, failed, and identity-only requests remain outside these timing ratios.

The preparation comparison keeps every trained model fixed and reduces exact preparation around its decisions. On 26 new batches, complete time changes from 79.784 to 64.183 seconds, or 19.6% less time. Section F gives the matched controls and graph results. With the revised preparation held fixed, the selected portfolio uses 4.7% less time than the fixed portfolio by summed medians and 5.0% less time across all repetitions. The previous portfolio study in Section E remains a separate comparison on different targets.

The earlier model tests and the exact preprocessing study answer separate questions. Seed and equation selection help in their measured domains, while learned row selection remains omitted. Exact preprocessing reduces costs shared by every portfolio candidate. Restricted vacuum recurrence and repeated-request reuse supply further savings under their own assumptions. These experiments use different inputs and controls, so their speedup factors cannot be multiplied.

9.2 Exact reuse in the restricted vacuum family

The high-loop vacuum study explores a different source of acceleration. For the family in Appendix L, exact recurrences express the reduction coefficients through $z = L(d-2)$. A coefficient calculation can then serve several loop orders and several targets. The method reuses intermediate coefficients and checks each result with an independent Taylor expansion; Kira remains

Table 12: Summary of separate comparisons. Each row uses its stated inputs and baseline. The studies provide no measured product of these speedups.

Method	Tested calculation	Result for complete time
Evolved seed boundary	Eight planar two-loop targets	22.3% less time than tuned native; 6.5% less time than fixed width
Sparse-seed deployment	20 planar and nonplanar targets	22.2% less time than tuned native; 22.6% less time than previous
Retained ML, new interface	24 first requests; two-loop boxes	1.2% less time than previous; 5.7% less time than fixed
Scale reuse with ML	Four three-scale sequences	64.4% less time than previous, including first solve
Target batch choice	96 matched batches; up to 11 targets	1.5% more time than previous; 0.8% more time than fixed partitions
Trained equation filters	68 matched batches; loops 1–4	4.8% less time than tuned ML-off control; 8.0% on 32 active batches
Learned seed bounds	68 matched batches; loops 1–4	27.8% less time than native bounds; 0.5% less than tuned fixed settings
Revised exact preparation	Fresh known-graph batches, loops 1–4	19.6% less time than the previous implementation
Added outer prediction	Same revised preparation	4.7% less time by medians; 5.0% less across repetitions
Earlier cost model	Known graph structures, loops 1–4	0.5% less time than native choices in the improved executable
Earlier gated policy	Same targets; fixed settings selected per graph	1.8% less time than the graph-specific fixed control
Seed network	One target from an existing equation pool	8.6% less time
Equation tree	Native reduction in four trained graph structures	5.7% less time
Row policy	Native symbolic elimination	No reliable saving
Original equation tree	Three new scattering structures	25.2% more time
Complete-cost model	Seven known graph structures, loops 1–3	No reliable gain over native controls
Policies per graph	Higher powers and lower sectors, loops 1–3	6.3% less time
Selective seed expansion	Known graphs, new profiles, loops 1–3	14.8% less time than native
Selecting Kira workers	Known graphs, new profiles, loops 1–3	4.2% less time than previous, sum of medians
Choosing Kira settings	Known graphs, new profiles, loops 1–3	No added gain over preceding method
Joint integrated reduction	Two compatible requests in seven graph structures	48.3% less time
Exact recurrences and reuse	Restricted vacuum family, loops 1–12	543.8× faster

available if this direct check fails. The deployed recurrence uses a fixed rule. Its large saving comes from exact mathematical reuse and implementation changes. A separately trained selector for recurrence strategies takes 1.7% more time than the fixed rule and supplies no established additional saving.

The comparison uses eight fresh groups, each containing three mass profiles with two, three, or four massive denominators, and requests loop orders one through twelve. Five repetitions compare the saved original Kira 3.1 executable, a later full-pool control, and the recurrence method on identical requests. All 120 calculations pass exact checks. The final-to-original time ratio is 0.001839, corresponding to speedup 543.815 with interval $[488.134, 605.064]$. A separate check verifies 27,960 nonzero target–master coefficient polynomials. The successful recurrence calculations need no Kira calls, whereas each baseline makes 1,440 calls across its trials.

This result depends on the structure of the family. The benchmark constructs Euler and mass partial-fraction identities rather than a general high-loop IBP system. Earlier checks at loop orders 13, 16, 24, and 32 use the same family, and the recurrence proof applies to every positive integer loop order under its stated assumptions. The demonstrated high-loop result therefore concerns this restricted vacuum problem; the connected scattering comparisons in Sections D and W have separate definitions and timings.

10 Conclusion

Section 7 retains the preceding ML application because neither new joint predictor clears the 2% validation threshold. On 24 first requests, the retained application with the new interface gives 1.2% less time, with time-ratio interval $[0.975, 1.024]$ relative to its preceding interface. This difference supplies no evidence of a gain from the new predictors. The separate scale sequences give 64.4% less time than the preceding application without reuse, including the initial Kira solve. This additional saving comes from the derived scale identity, while the first reduction still uses the preceding ML decisions.

The first evolved seed boundary gives 22.3% less time, with a paired-repetition ratio interval of $[0.751, 0.813]$ relative to tuned native Kira on eight planar two-loop requests. Its added saving relative to the fixed surrounding-width control is 6.5% less time, with a paired-repetition ratio interval of $[0.899, 0.980]$. The broader sparse-seed workflow in Section 6 gives 22.2% less time, with a paired-repetition ratio interval of $[0.764, 0.792]$ relative to tuned native Kira on 20 matched two-loop requests. Its comparison with the fixed sparse control gives 3.6% less time, with a paired-repetition ratio interval of $[0.946, 0.981]$. These results use separate workloads from the earlier equation-filter and batch tests.

The target-batch comparison in Section 5 gives 1.5% more time than the previous undivided workflow across 96 matched batches with up to 11 targets. Its additional change relative to fixed partitions with the same filters is 0.8% more time, with ratio interval $[0.988, 1.035]$. Validation disables the new predictor, so this comparison establishes no additional ML gain from batch selection. Every successful reduction passes exact coefficient checks. This is a separate workload from the smaller-batch filter study.

We train models for decisions that affect exact Feynman integral reduction and evaluate their effect on complete calculation time. The new equation filters act before symbolic coefficient construction, using target and seed indices to select individual identities or whole groups from a seed. Kira performs exact elimination and retries without the filter when the first equation set is insufficient.

With tuned seed bounds and exact implementation changes held fixed, the reserved comparison changes summed median time from 153.937 to 146.586 seconds across 68 matched nontrivial batches. Filtering is enabled on 32 batches, whose corresponding times are 91.707 and 84.384 seconds. The 95% paired-repetition interval for this subset's time ratio is $[0.916, 0.927]$. The

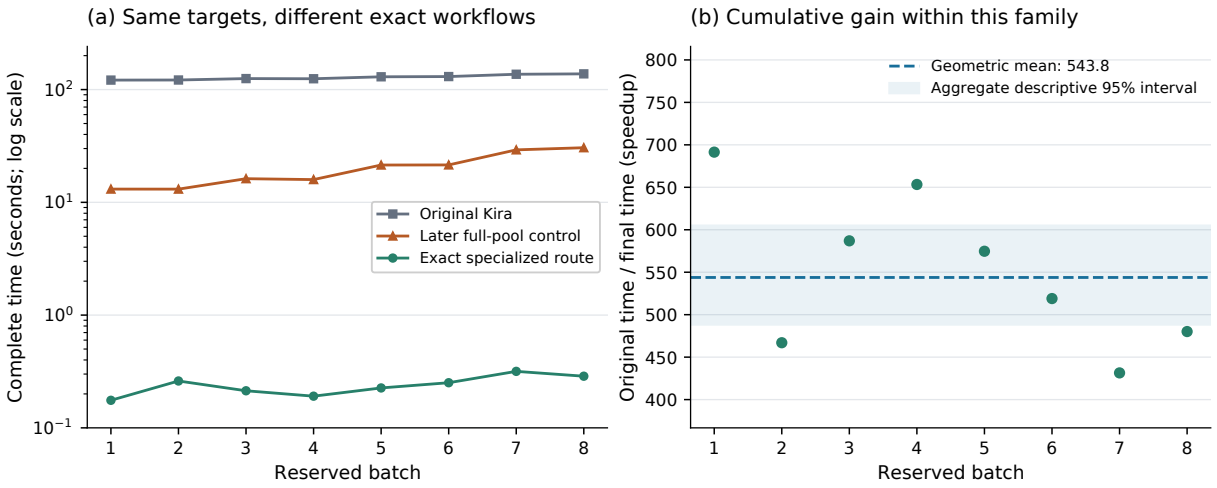


Figure 8: Exact recurrence reuse gives the large saving in the restricted vacuum family. Left: complete times on a logarithmic axis. Right: original time divided by final time. Each batch contains three mass profiles at loops 1–12. The result combines mathematical reuse and program improvements.

comparison includes prediction and recovery, and every successful output passes exact coefficient checks. The separate index-transfer check takes 6.7% less time across 24 matched batches, with ratio interval $[0.928, 0.940]$. Section 4 reports individual graphs, incomplete references, and the fresh-application check.

The seed-bound study explains why a matched fixed control is necessary. Its learned choice saves 27.8% against unshifted bounds, but the additional 0.5% over tuned fixed settings lies within timing variation. Learned row elimination also supplies no reliable gain in its recorded comparisons, so the retained application uses native row choices. These results guide deployment: use the validated filters with tuned seed settings and retain exact fallback.

The earlier full-calculation comparison gives 88.2% less time than the saved original executable on 37 matched requests, including master discovery. That saving comes from exact implementation changes and has a different workload. The restricted vacuum recurrence verifies all twelve studied loop orders under its identity assumptions. Neither result can be multiplied by the new filter gain. The new ML test supplies a master basis and covers known structures at one through four loops with rational kinematics and symbolic dimension. The target-partition study extends the tested batch sizes as described in Section 5. General high-loop scattering, unrestricted symbolic kinematics, and globally minimal reduction paths remain outside the demonstrated scope.

The separate analytic specialization reduces scalar massless banana targets through one corner master, with a derived coefficient valid at every positive loop count. Its 9 completed native comparisons give a speedup of 21.3–357.8 times for the stated targets. These gains come from exact mathematics and are separate from the ML gains (Section 8).

A Training data and experimental design

A.1 Constructing targets with known reductions

To train the original models, we need targets together with master coefficients and a recorded sequence of valid reduction steps. We obtain these data by working backwards from checked master relations. If $E_q = \sum_i A_{qi} I_i = 0$ is a source identity, we construct

$$T = \sum_m r_m M_m + \sum_q \lambda_q E_q, \qquad \lambda_q, r_m \in \mathbb{Q}(\boldsymbol{x}, d). \tag{6}$$

Expanding the right-hand side produces an expression in indexed integrals while the added zero identities leave its master coefficients unchanged. We record the identities, their seeds and operators, and the operations used in the construction. Appendix J proves the relation and explains how to recover a reduction path from this record.

This procedure usually produces a sum of integrals. It produces a single indexed integral only when the expanded expression has that form. It also supplies one sufficient seed set and one valid reduction path; it does not establish that either has minimum cost. The labels therefore teach the original models to reproduce a checked construction, and later timing experiments determine whether their choices help Kira.

A.2 Coverage and separation of training and test data

Table 13 summarizes the original collection. The 17,100 family instances contain 57,979 master entries, counted separately in each family, and 2,301,272 targets. Many instances are parameter variations of the same graph. We call a specified assignment of masses and momentum invariants a *parameter profile*. In particular, the higher-loop additions use the restricted vacuum family derived in Appendix L; they do not represent general scattering graphs through twelve loops. The test set contains 148,494 single integrals and 541,888 integral expressions. Appendices N and O specify the families and the checks used to establish their master bases.

Table 13: Original constructed data. Family instances include parameter variations of the same graph.

Part	Family instances	Targets
Training	11,963	1,610,890
Testing	5,137	690,382
Total	17,100	2,301,272

Related mass assignments remain in the same training or test group at every loop order. The seed and row networks each receive twenty additional passes through the expanded training set, starting from their earlier parameters and optimizer state. Some test profiles had already appeared in earlier evaluations, so these results concern reserved parameter groups rather than entirely new graph structures. The later native-Kira studies use separate training, validation, and test samples: training fits the models, validation selects their settings, and the reserved test measures those fixed choices.

A.3 What the timing comparisons measure

Within each comparison, every method receives the same targets and prescribed master basis and uses the same allocated resources. We use *complete reduction time* for the measured time that includes input preparation, model prediction, algebra, checks, and all fallback attempts. Offline training and the construction of reference answers are excluded. The original local-model comparisons repeat each calculation five times. The final complete-cost comparisons use three repetitions, with the method order rotated. Each comparison reports batch medians and states its repetition count.

The baseline also needs to be specified. Most native comparisons use the same modified Kira 3.1 executable for learned and native choices, so improvements shared by that executable do not count as a benefit of the model. The cumulative vacuum comparison instead uses the saved original executable as its baseline. Later sections also distinguish the sum of batch median times from a geometric mean that gives each graph equal weight. The latter averages logarithms of time ratios and then exponentiates the result. Appendix M defines these measures and their descriptive uncertainty intervals. Keeping these conditions explicit allows each speedup to be interpreted against the calculation it actually measures.

B Learning the three reduction decisions

We first test the three decisions separately. For each model, we describe the information it receives, the reference labels used in training, and its effect on exact reduction time. This separates the ability to reproduce a recorded choice from the ability to accelerate Kira.

B.1 Model 1: selecting seeds

The seed network receives the terms of the target, its family, the master indices, and a finite list of candidate seeds. It maps each target term to 32 learned numerical features and combines these with ordered pairs of terms and information about the family and masters. A seed-dependent attention operation then identifies which target terms are most relevant to that candidate. The shared representations and attention follow Refs. [14, 15]; Appendix K gives the full architecture.

The network has 28,097 fitted parameters and assigns a separate score to every candidate:

$$p_b = \frac{1}{1 + e^{-g_\theta(T,F,\mathcal{M},\boldsymbol{\sigma}_b)}}, \qquad \widehat{\mathcal{S}}(T) = \{\boldsymbol{\sigma}_b \in \mathcal{C} : p_b \geq \tfrac{1}{2}\}. \tag{7}$$

Here F denotes the family, $\mathcal{M}$ the master list, $\mathcal{C}$ the candidate list, and g_θ the output of the network with fitted parameters θ. The score estimates whether a seed belongs to the recorded sufficient set. Because a target can require several seeds, several scores can exceed the selection threshold and their sum need not be one. Training uses binary cross-entropy and AdamW [16]. The network receives neither the target's final master coefficients nor its later reduction steps.

On the 690,382 reserved targets, the model has precision 95.750%, recall 96.892%, and F_1 score 96.318%. Its complete predicted set matches the recorded set for 84.332% of targets. Kira reduces 652,517 targets with the predicted sets, corresponding to 94.515% sufficiency, and reduces all targets when the full equation pool is available. These percentages differ because matching the recorded set and supplying enough equations are different tests: another seed set can also be sufficient. Appendix M defines the metrics. We report precision and recall because the large number of negative labels can make overall classification accuracy misleading [17].

To determine whether this prediction saves time, we use a separate comparison on existing identity pools from twelve reserved one-loop box instances. The weights remain fixed, and the timing test uses threshold 0.01, chosen before that comparison; the earlier label and sufficiency tests use 1/2. Both the learned selector and a distance-based rule receive the same basis check and full-pool fallback. For single targets, the learned-to-full-pool time ratio is 0.914, with descriptive 95% interval $[0.871, 0.958]$, giving 8.6% less time. Relative to the distance rule, the ratio is 0.913 with interval $[0.855, 0.972]$, giving 8.7% less time. Every method completes all five repetitions of these twelve cases.

The gain is less clear for larger requests. For sixteen-target batches, the ratio is 0.952 with interval $[0.772, 1.148]$, which includes equal time. Across the full planned seed comparison, 345 trials finish exactly, three remain incomplete, and twelve repetitions are skipped; the timing summary uses only complete paired cases. The identity pools already exist when these timers start. A separate native test that includes IBP generation gives ratio 1.096 with interval $[0.963, 1.275]$. Thus the original network demonstrates a saving for the stated supplied-equation single-target test, while native generation requires a different assessment. Section B.4 reports new seed trees trained directly on scattering reductions.

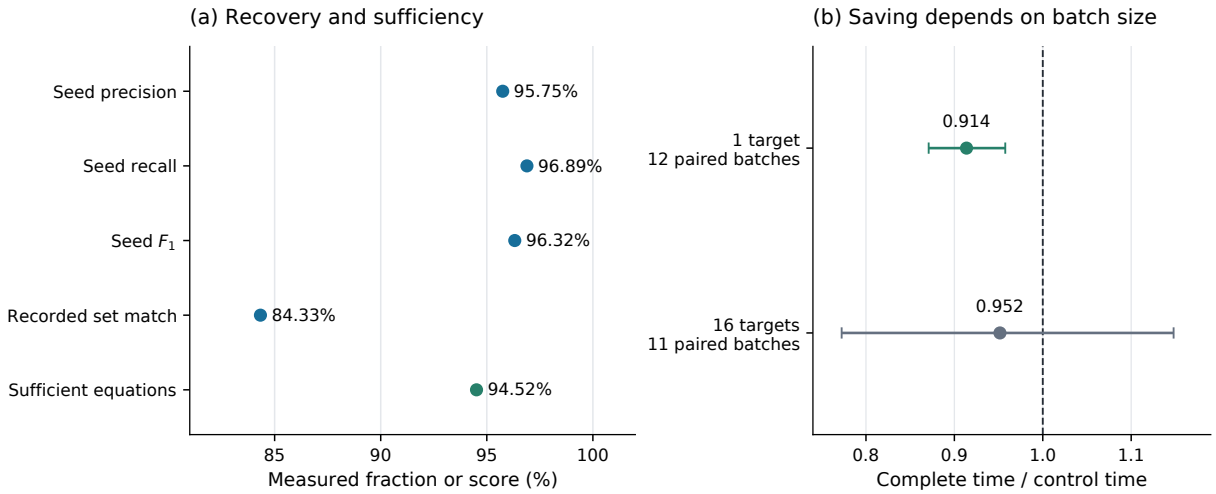


Figure 9: Seed prediction accuracy and reduction speed measure different outcomes. Left: results on the original reserved targets. Right: learned time divided by full-pool time. Values below one are faster. The single-target interval lies below one; the sixteen-target interval crosses it.

B.2 Model 2: selecting elimination rows

After the equations have been generated, several allowed source rows may eliminate the next integral. The row network scores these alternatives using 132 inputs that describe the pivot integral, nonzero entries, family, matrix size, and sampled cancellations. It maps each action to 32 learned features, combines information across the available actions, and passes the result through a hidden layer with 64 components. A softmax converts the final scores into probabilities that sum to one, and the network selects the action with the largest probability. The 10,529 fitted parameters determine the choice; exact arithmetic performs the row operation. Appendices I and K give the operation and architecture.

The original training examples follow one changing target row while the source identities stay fixed. On 2,558,472 reserved states with a choice, the network selects the recorded action in 95.845% of cases, compared with 99.473% for a local deterministic rule. Complete predicted paths agree with the saved master row at three finite-field points for 690,302 of 690,382 targets. These sampled checks support agreement at those points but do not prove symbolic equality. Including prediction, the network takes 17.657 times as long as the local rule.

The more relevant application places the fixed network inside Kira's changing symbolic matrix and preserves Kira's allowed integral order. Across 84 exactly checked trials, this implementation takes 1.305 times the time of stock Kira. We therefore also test smaller policies trained on measured algebraic cost, with compiled prediction and shared features. The latest row-only comparison gives ratio 0.982 with interval $[0.959, 1.003]$, while combining it with the seed policy gives 1.100 with interval $[0.966, 1.279]$ against native choices.

Figure 10 shows that the later intervals still include equal time. We therefore retain Kira's row choices in the final method. This result explains why the remaining studies focus on decisions that can avoid equation generation or repeated work: reproducing a plausible elimination path did not, by itself, provide a reliable reduction in complete time.

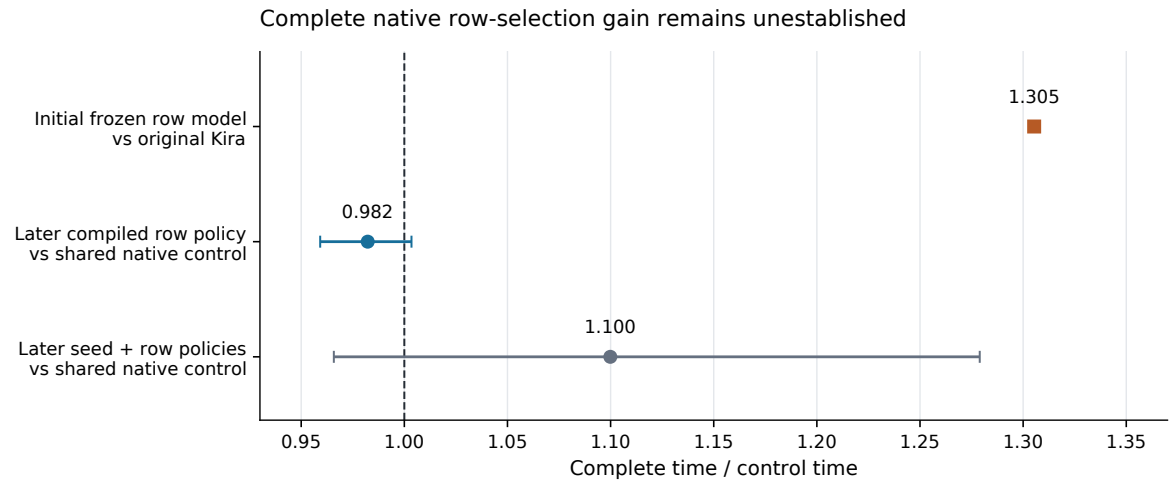


Figure 10: The row model gives no reliable complete-time saving. The first model is slower, and the later row-only interval includes equal time. The original summary provides no interval for the first result. These comparisons use different model versions.

B.3 Model 3: selecting individual equations

Equation selection offers an earlier opportunity to avoid work. The model scores each seed–operator pair before Kira constructs its symbolic coefficients. Its seventeen inputs describe the family, target list, seed indices, and operator; they include the number of loops and targets, extra propagator powers, numerator rank, and distance between seed and target indices. A classification tree [9] applies feature-threshold decisions and returns a score from its final leaf. The score estimates whether the equation belongs to the useful part of a recorded native reduction. No equation coefficients are needed for prediction, and Kira retains its native treatment of symmetries and masters.

The original fit uses 153,124 candidate equations, including 12,270 positive labels, from fourteen batches in eight parameter profiles. The four graph structures are the two-loop sunrise, planar double box, three-loop vacuum, and three-loop ladder. Validation compares trees of depth eight and twelve, containing 295 and 1,335 nodes, at threshold 0.01. It selects the deeper tree for the double box and the shallower tree for the vacuum and ladder; the 21-target sunrise case retains the preceding equation policy. A filter is enabled only when validation shows lower complete and preparation times without fallback. Adding learned seed or row choices fails this condition, so native seed and row choices remain active in this comparison.

Testing uses eight new parameter profiles and fourteen target batches. The masses and momentum invariants take rational values, while d remains symbolic, so the coefficient field is $\mathbb{Q}(d)$. Five repetitions of four methods give 280 complete calculations. Native choices, the early equation model, and the distance-based control share the same executable improvements; a fourth method uses the preceding release. This design distinguishes the effect of equation selection from improvements in the surrounding implementation.

Relative to native choices in that executable, the model reduces complete time by 5.7% and equation-preparation time by 36.2%. The corresponding ratios are 0.943 with interval $[0.915, 0.970]$ and 0.638 with interval $[0.546, 0.734]$. Preparation includes modular generation, selection, modular solving, and symbolic equation regeneration. The smaller change in complete time reflects the work that remains after preparation. All 280 calculations succeed on their first attempt, and independent exact checks cover 94,970 coefficient comparisons and eight applications supplied only with the prescribed basis.

To test how far the original tree transfers, we also apply the fixed depth-twelve model to three structures absent from its training data: a massless nonplanar two-loop box, a massless pentagon,

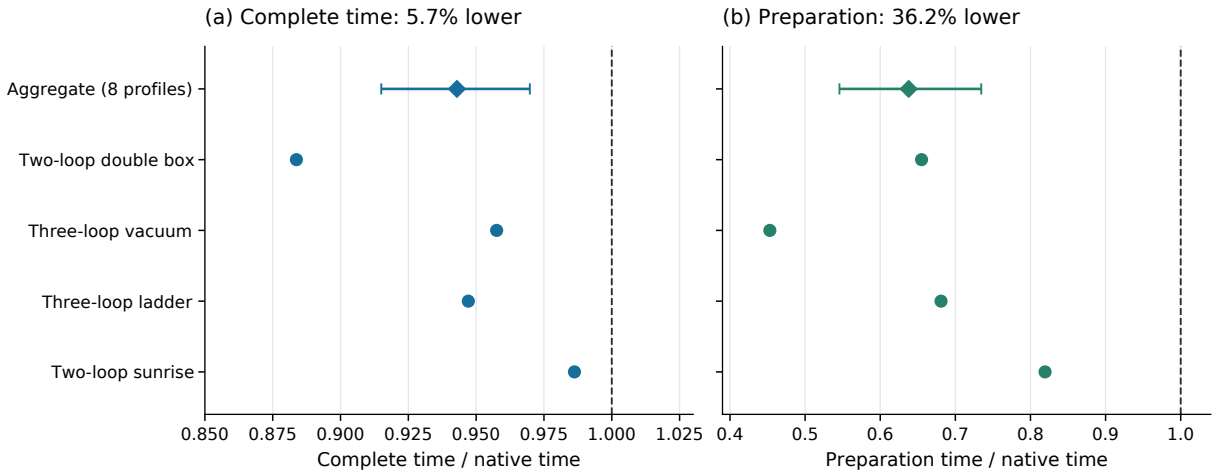


Figure 11: Equation selection saves more preparation time than complete reduction time. Each ratio compares the model with native choices in the same executable. The panels use different labelled ranges. The aggregate intervals describe variation across the eight profiles.

and a box with one external virtuality. Appendix N defines their propagators and invariants, using the massless limit of the nonplanar family in Ref. [4]. Four batches use rational kinematics and symbolic d. Three more keep the independent invariants symbolic, with coefficient fields $\mathbb{Q}(d, s, t)$, $\mathbb{Q}(d, s_{12}, s_{23}, s_{34}, s_{45}, s_{51})$, and $\mathbb{Q}(d, s, t, v)$. Here s, t are four-point invariants, $s_{i,i+1}$ are adjacent five-point invariants, and v is an external squared momentum.

The seven batches contain 88 targets and produce seventy comparison calculations. All final results pass exact checks, but every one of the 35 filtered calculations needs native fallback. The average time ratio is 1.252, or 25.2% more time. Checks cover 5,600 coefficient pairs, agreement with enlarged seed regions in all seven cases, and agreement after new rational substitutions in the three symbolic cases. The fallback therefore restores a correct answer at an additional cost. Figure 12 records this transfer test before the new training described next.

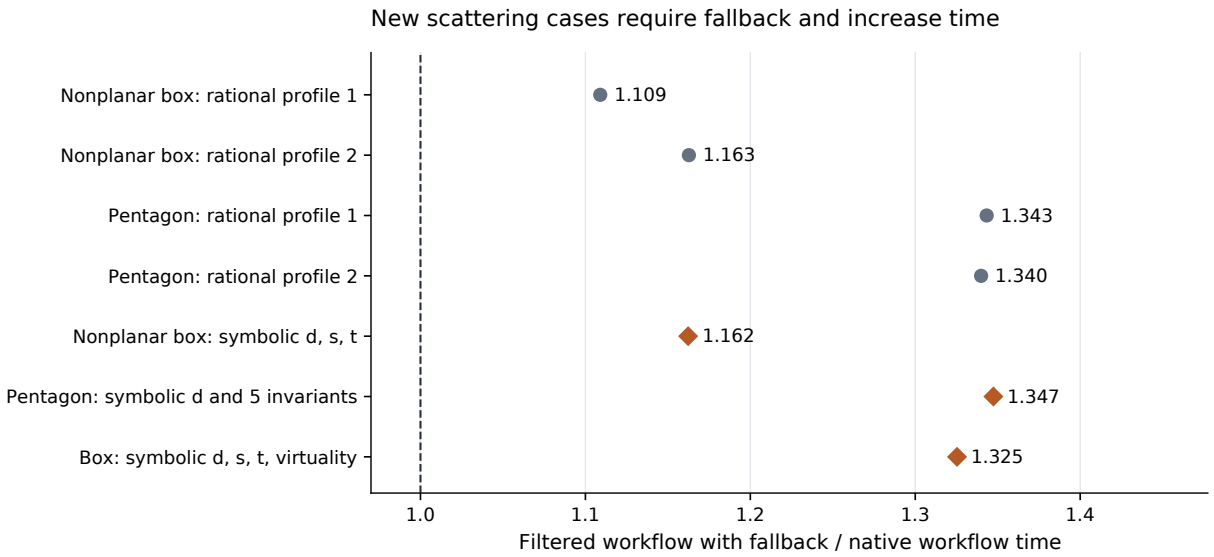


Figure 12: The original equation model is slower on the new scattering structures. Each point compares five-run median times, including fallback. All 35 filtered calculations require a retry. This historical test uses the original weights, before additional training.

B.4 Training on the additional scattering structures

The transfer failures raise a specific question: can examples from the missing structures teach a filter to retain enough equations? To test this, we train a new seed tree and equation tree for each of the nonplanar box, pentagon, and box with external virtuality. The original models remain fixed for comparison. Appendix K defines the new reference labels and tree fits.

Table 14 separates the training, validation, and test batches. Rational batches use different invariant values in each split, while symbolic batches keep the independent invariants symbolic and use disjoint target lists. The graph structures occur in all three splits. Thus the test measures learning within these structures and separates that question from transfer to an unseen graph.

Table 14: The additional scattering study. Each of the three structures contributes equally to the batch counts.

Part	Rational batches	Symbolic batches	Targets
Training	6	3	108
Validation	3	3	60
Testing	6	3	108

Each tree has maximum depth twelve. The seed tree uses sixteen features and gives all operators of a seed the same score; the equation tree also receives the operator index, giving seventeen features and allowing different decisions for different operators. In both cases Kira enumerates its usual seed region, the tree filters candidates before coefficient construction, and native elimination remains active. These fits produce new trees trained on native reductions, while the original neural networks remain unchanged.

The fits use 59,706 equation records and 4,674 grouped seed records, including candidate patterns repeated in different training batches. Table 15 gives the tree sizes. Validation compares three score thresholds and fixes a setting before the reserved test. The four test methods are native choices, the original equation tree, the new seed tree, and the new equation tree. Five repetitions on identical targets give 180 calculations, with prediction, exact algebra, coefficient checks, cleanup, and fallback included in the timer. Reference discovery, training, and later recording of results are excluded.

Table 15: Additional training records and tree sizes. Each graph uses three training batches.

Structure	Seed records	Seed nodes	Equation records	Equation nodes
Nonplanar box	4,326	67	56,238	819
Pentagon	258	15	2,838	179
Box with virtuality	90	11	630	81

All 180 calculations pass exact checks of 9,880 returned coefficient entries. Eighteen additional applications receive the target and prescribed basis without reference coefficients, and their answers also pass an independent comparison after reduction. Each new selector completes all 45 timed attempts without fallback, whereas the original equation model needs fallback in all 45 trials. The new training therefore improves sufficiency on these inputs.

The timing result is more limited. Relative to native choices, the new seed model has ratio 1.027 with descriptive 95% interval $[0.995, 1.086]$, and the new equation model has ratio 1.017 with interval $[0.988, 1.054]$. Their measured average times are 2.7% and 1.7% higher, respectively. The original filter has ratio 1.368, or 36.8% more time, on this new test. Its earlier 25.2% overhead refers to different parameter points and symbolic targets. The additional training removes much of the fallback cost but does not establish an overall saving against native Kira.

Table 16: New reserved test results. Each value uses five repetitions. Ratios divide model time by native time; values below one are faster.

Case	Native (s)	Original tree	New seeds	New equations
Nonplanar box (13)	4.163	1.293	1.023	0.980
Nonplanar box (14)	4.140	1.221	1.002	0.974
Nonplanar box (symbolic)	4.458	1.233	0.983	1.010
Pentagon (13)	1.005	1.439	0.963	1.016
Pentagon (14)	0.965	1.436	1.069	1.038
Pentagon (symbolic)	1.120	1.356	0.958	0.979
Box with virtuality (13)	0.826	1.339	0.953	0.991
Box with virtuality (14)	0.763	1.529	1.152	1.124
Box with virtuality (symbolic)	0.729	1.499	1.166	1.052
Geometric mean ratio	—	1.368	1.027	1.017

The individual results in Table 16 and Figures 13–14 explain this distinction. Some new filters reduce time, while others add overhead to cases that native Kira already solves quickly. We therefore retain native choices as the default for this scattering study. The result supports training on the missing structures to improve sufficiency, although it cannot isolate missing data as the sole cause of the original failure: the study also changes the seed-model architecture and fits separate trees for each graph. The intervals resample the three graph groups and describe this limited comparison.

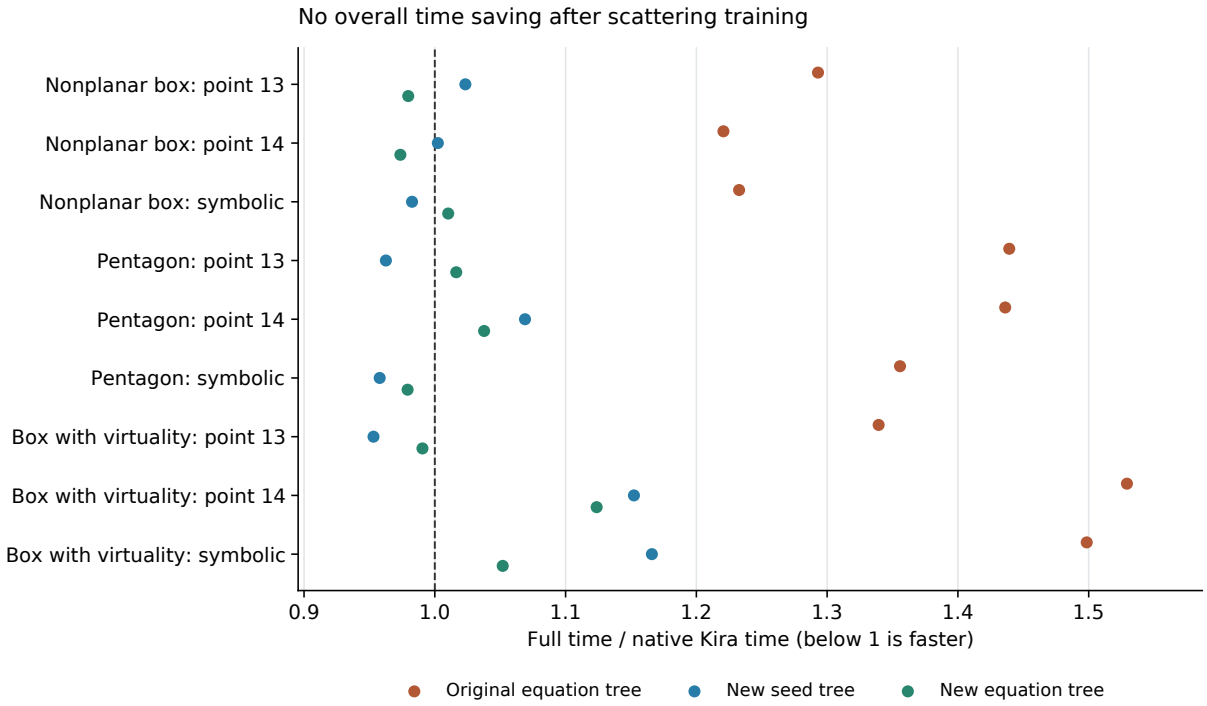


Figure 13: Training on the three structures removes much of the original filter's overhead, but gives no overall saving against native Kira. Points compare five-run median complete times, including prediction, checks and fallback. The dashed line marks equal time. No per-case uncertainty interval is inferred from five repetitions.

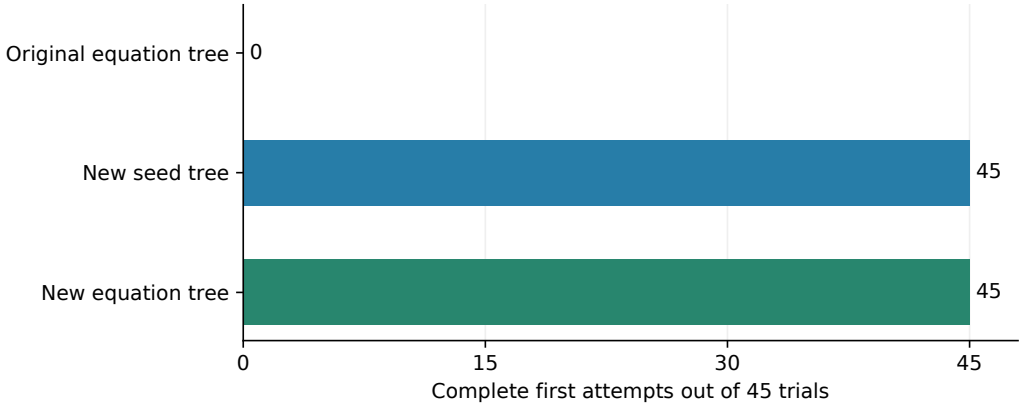


Figure 14: The new selectors provide sufficient equations on every first attempt. Each method has 45 trials across nine batches. Native fallback lets the original filter eventually complete, and its added time remains in the comparison above.

C Selecting a complete calculation before equation generation

The local-model tests show that selecting useful equations does not by itself ensure a faster complete reduction. We therefore train models on measured calculation time and move their decision to the start of a target batch. The earlier experiments examine graph-specific strategies, selective seed expansion, integral ordering, and worker count. Appendix H preserves their model definitions and results. Their gains use different target batches and cannot be multiplied into one speedup.

These experiments also raise an attribution question: how much of the saving comes from the learned choice, and how much comes from the fixed settings around it? The next section holds the other settings fixed to answer that question. Its results motivate choosing among complete reduction strategies in the following section.

D Checking the contribution of each component

The previous models form a sequence of choices, so a faster final calculation does not identify which choice supplies the saving. We compare ten methods on fourteen fresh batches from seven known graph structures and then disable individual learned decisions while keeping the other stages unchanged. Every method uses the same targets and prescribed basis. All 420 reductions in the first comparison and all 96 additional decision controls pass exact coefficient checks.

The native-choice total is 66.035 seconds, using sums of three-run batch medians. Table 17 shows the complete strategies relative to that control. The result through worker selection takes 46.777 seconds, but this saving includes fixed seed settings as well as learned decisions. The equation, seed, and row filters increase time on this particular target pattern. Earlier savings on other targets remain separate results.

Table 17: Component comparison on the same fourteen batches. Complete times include decisions, exact checks, and recovery. The first three rows retain preceding stages.

Method	Complete time (s)	Change from native
Graph-strategy model	61.779	6.4% less time
Through seed expansion	49.309	25.3% less time
Through worker selection	46.777	29.2% less time
Equation filter	72.446	9.7% more time
Seed filter	70.241	6.4% more time
Row model	105.500	59.8% more time

Table 18: The added learned decision, with other stages held fixed. Each row uses the subset on which that decision is enabled. Fixed settings were selected before this test.

Decision	Batches	Fixed (s)	Learned (s)	Change
Graph choice	6	51.448	50.188	2.4% less time
Seed expansion	6	41.030	41.093	0.2% more time
Worker count	4	24.232	25.555	5.5% more time

The isolated controls in Table 18 show why complete-time gains need a fixed comparison. On these batches, graph choice saves a small amount, while the added seed and worker choices increase time. Figure 15 presents both comparisons. These results motivate replacing the sequence of selectors with the single cost policy in the next section.

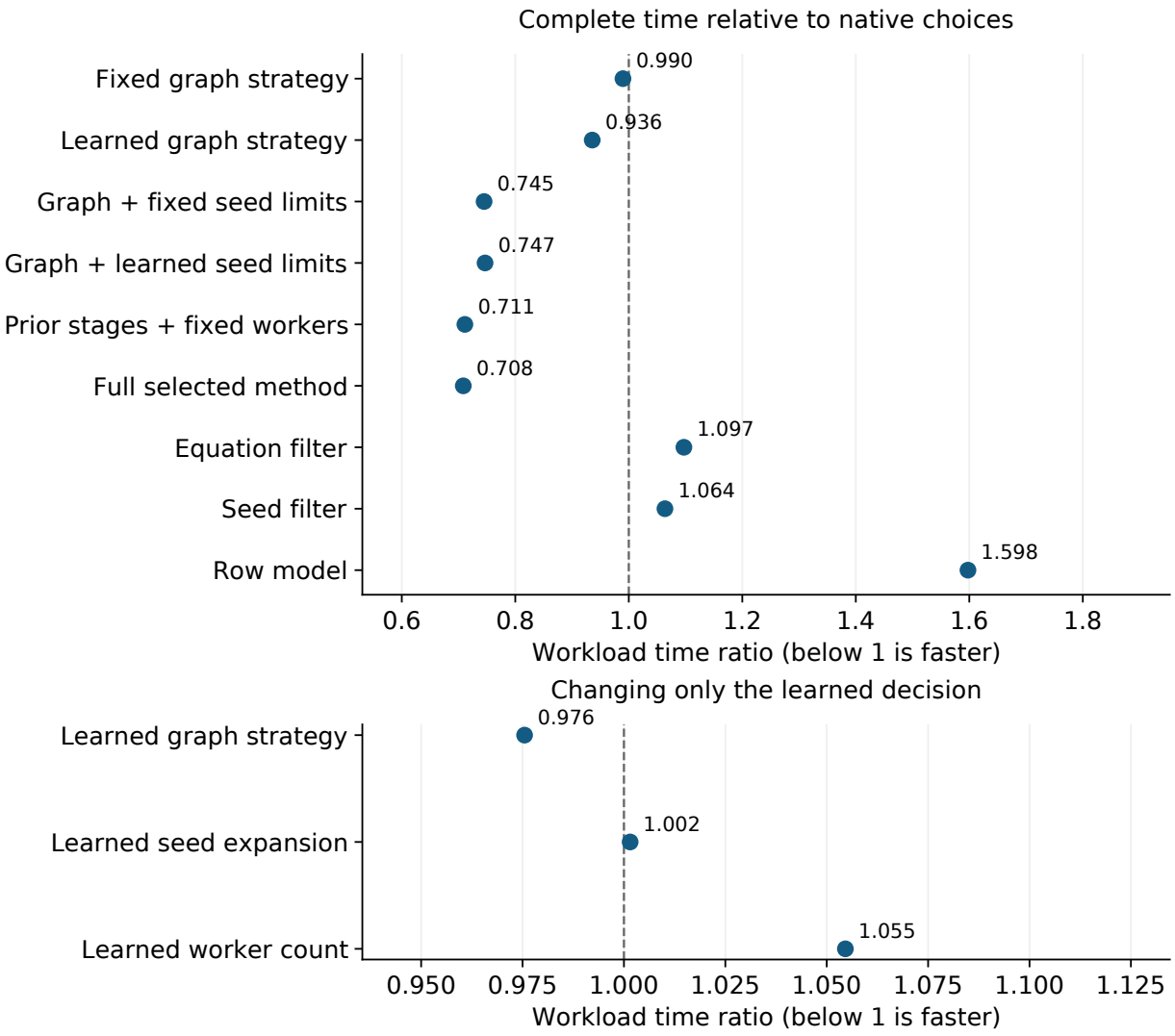


Figure 15: Complete strategies and their added learned decisions. The upper panel uses all fourteen batches. The lower panel changes one learned decision on its enabled subset. Both show ratios of summed batch medians and retain the equal-time reference.

D.1 Connected scattering and bounded higher-loop tests

We also construct connected four-point ladders and boxes with bubble subgraphs, defined in Appendix V. The two completed three-loop bubble-box profiles take 10.538 seconds with restricted initialization and native equation choices, compared with 7.468 seconds with the frozen equation filter. This is 29.1% less time under the same initialization. The complete strategy selectors otherwise retain native choices on these previously unsupported graph structures.

The bounded four- through six-loop probes identify separate costs and limits. Four-loop initialization exceeds the initial short time allowance. Five-loop attempts exceed the memory allowance, and the six-loop four-point representation requires 39 inverse propagators, above the tested executable's 31-entry limit. Later runs produce a four-loop bubble-box reference and reproduce its coefficients with enlarged seeds. Three native repetitions have median time 465.329 seconds. Two completed filtered repetitions take 520.336 and 551.161 seconds, each requiring native recovery after an insufficient equation set. Only these two filtered repetitions complete, so this diagnostic supplies no matched three-run speedup estimate. The incomplete attempt is excluded from timing results.

The new work therefore addresses both preparation and the learned choice. A complete four-loop reduction is established for the stated bubble-box family, while the failed larger-family probes supply no speedup claim. The vacuum recurrence used elsewhere in this paper does not determine the scattering coefficients, as explained in Appendix V.

E A learned portfolio of complete reduction strategies

The preceding comparisons show that the useful decision depends on both the denominator structure and the target powers. A seed region that suffices for one batch can leave another batch unreduced, while expanding every seed limit can generate equations that the calculation does not need. We therefore train a model to choose a complete calculation from several strategies before Kira generates the equations. This choice includes the previous workflow as a candidate, so the new policy can retain that workflow when its measured cost is lower.

E.1 What the model chooses

For a target batch, let r_0 be the largest sum of positive indices, s_0 the largest total numerator power, and d_0 the largest number of extra denominator powers, or dots. Kira uses limits of this kind to define the seed region [5]. Our earlier complete-cost policy chooses among seven regions or equation settings, as described in Appendix W. A development comparison reveals a useful additional choice: increase s_0 by one while leaving the positive-index and dot limits unchanged. This selective expansion can supply missing identities without the cost of increasing all three limits.

The final portfolio contains five choices: the earlier cost policy with one or two workers, selective numerator-rank expansion with one or two workers, and the previous workflow from Appendix U. A worker carries out tasks in Kira's existing parallel symbolic solver [4]. The previous workflow combines its existing graph, seed, and worker choices; its parameters remain fixed in this comparison. Each choice uses the same exact preprocessing and a supplied master basis. The three- and four-loop bubble boxes have no trained policy in the previous workflow, so its control uses the documented native fallback. We retain Kira's native row elimination because the learned row model has not reduced complete time reliably.

Each attempt must express every target in the prescribed basis. If it cannot do so, the calculation retries with native seeds and then an expanded seed region. We skip a repeated attempt when its effective seed bounds, ordering, and equation filter are identical to those already tried.

The model does not predict reduction coefficients. It predicts the relative time of these complete strategies, including checks, failed attempts, and recovery. Appendix Y defines the model inputs, objective, guard, and exact recovery procedure.

E.2 Training and independent tests

We fit 30 candidate predictors using 47 training batches and select the predictor, fixed control, and decision threshold using 26 separate validation batches. The candidates include regression trees, random forests, and extremely randomized trees [9, 10, 11]. Their twenty inputs describe the graph structure, target powers, numerator ranks, number of targets, and distribution of dots. They contain no reference coefficients or measured test times. The recorded label calculations take 1858.3 seconds, and fitting takes 2.621 seconds. These costs occur before deployment.

For each graph, validation also selects the fastest fixed portfolio choice. The model replaces this choice only if it reduces summed validation time by at least two percent and gives a saving in both validation repetitions. This rule enables target-dependent portfolio predictions for planar double box, three-loop two-point ladder, nonplanar box, sunrise. All enabled outer predictors are single regression trees. A retained portfolio choice can itself contain an earlier learned policy; therefore the fixed-portfolio control isolates the added outer decision, rather than removing all earlier learned decisions. Requests outside the recorded graph structures and feature ranges use native settings.

The reserved test uses rational kinematic profile $h = 306$ and symbolic targets with a dot position absent from the earlier symbolic training and validation sets. The earlier reserved profiles do not enter fitting or selection. The completed comparison contains 26 batches from nine known graph structures through four loops, with one to three targets per batch. Appendix Y specifies the targets and reference checks. These tests change kinematics and target positions within known structures; some rational target patterns recur across profiles. They do not test transfer to arbitrary new graphs or larger target batches.

Every method receives the same two-core allocation and an 8 GiB memory allowance. The native control and earlier cost-policy control use one Kira worker. The previous workflow, fixed portfolio, and selected portfolio can use one or two workers according to their chosen strategy. We rotate method order and repeat each method three times, with a fresh Kira process and temporary files for every attempt. Complete time includes prediction, preparation, exact algebra, coefficient checks, cleanup, and recovery. Model imports, initial model loading, reference construction, and basis discovery are separate. No coefficient cache supplies the test answers.

E.3 Complete-time results

Table 19 sums the three-run median times of the individual batches. The selected portfolio takes 85.497 seconds, compared with 108.029 seconds for native choices and 95.382 seconds for the previous workflow in the same improved executable. These results correspond to 20.9% less time than native choices and 10.4% less time than the previous workflow. The earlier cost policy takes 102.561 seconds on these same inputs. The graph-specific fixed portfolio takes 86.809 seconds, so the added portfolio decision gives 1.5% less time relative to this stronger control. Figure 16 shows where these differences occur.

Summing every timed repetition gives 19.4% less time than native choices, 9.4% less time than the previous workflow, and 0.4% more time than the fixed portfolio. The small difference from the fixed portfolio changes sign between the two summaries. These measurements establish no stable aggregate saving from the added outer prediction. The practical improvement over the previous workflow includes the revised candidate strategies and worker choices, which also benefit the fixed portfolio.

All 390 timed reductions and 26 further applications reproduce the reference coefficients exactly.

Table 19: Complete reduction times in seconds. Each row sums batch medians from matched tests with common exact preprocessing. “Fixed” selects a portfolio choice per graph using validation only.

Graph	Native	Previous	Fixed	Selected
Three-loop bubble box	2.280	2.300	2.274	2.388
Four-loop bubble box	32.420	29.726	28.981	29.129
Planar double box	9.478	8.226	8.179	7.406
Three-loop two-point ladder	23.326	17.584	14.314	14.108
Nonplanar box	28.380	25.414	20.793	20.236
Virtuality box	2.585	2.756	2.763	2.608
Pentagon	3.895	3.922	4.072	3.966
Sunrise	1.941	1.677	1.661	1.770
Three-loop vacuum	3.724	3.777	3.772	3.886
Total	108.029	95.382	86.809	85.497

The applications receive only the prescribed master basis before reduction; the comparison with reference coefficients follows execution. The selected portfolio requires 3 recovery attempts across the timed tests. Its median outer decision time is 0.575 milliseconds. The CPU-time totals are 115.370 seconds for native choices and 114.666 seconds for the selected portfolio. Reporting both wall and CPU time separates elapsed-time savings from changes in CPU use, as shown in Figure 17. Every planned batch completes both reference calculations.

The dotted three-loop two-point ladder illustrates the change in the equation system. Native choices use seed limits $(10, 0, 2)$, then $(11, 1, 3)$, while the selected portfolio uses $(10, 1, 2)$, in the order $(r, s, d_{\mathrm{dots}})$. Summing the reported generated-equation counts across all attempts gives medians of 287,558 equations for native choices and 94,380 for the selected portfolio. Their complete times are 14.637 and 7.345 seconds. This count measures equation generation; symbolic simplification can require different work per equation. The CPU-time comparison also accounts for the effect of worker count.

The application also avoids loading the previous workflow until a selected strategy requires it. On the dotted sunrise batch, three rotated comparisons of a new application instance give median request times of 3.258 seconds with immediate loading and 0.727 seconds with deferred loading. These times include construction, one reduction, an exact coefficient check, and closure, while common Python imports remain outside the timer. All six requests choose the same strategy and return the same exact coefficients. This separate interface saving applies when the request does not need the previous workflow. A request that first selects that workflow pays its loading cost within the application time. The repeated portfolio timings above already keep the models loaded, so they receive no additional saving from this change.

E.4 Separating the exact implementation changes

The portfolio tests use two exact improvements in every method. We index the sector files that exist instead of searching for absent files, while retaining their original processing order. We also simplify the criterion for zero sectors by setting Feynman parameters absent from all declared top sectors to zero before forming that criterion. The complete parametric polynomials remain available for the other operations. Appendix X proves why this projection preserves the criterion in the declared sector domain, following the parametric treatment of zero sectors [18].

Separate matched tests enable these changes in sequence for connected bubble boxes. For the 3-loop batch, the complete time changes from 6.087 to 2.939 seconds with the first study’s validated policy, an overall factor of 2.07. The 4-loop previous-preparation control does not complete after 1190.6 seconds across its bounded attempts. Further repetitions of this failed control are omitted, while the remaining settings are completed. The fully improved calculation takes 28.666 seconds,

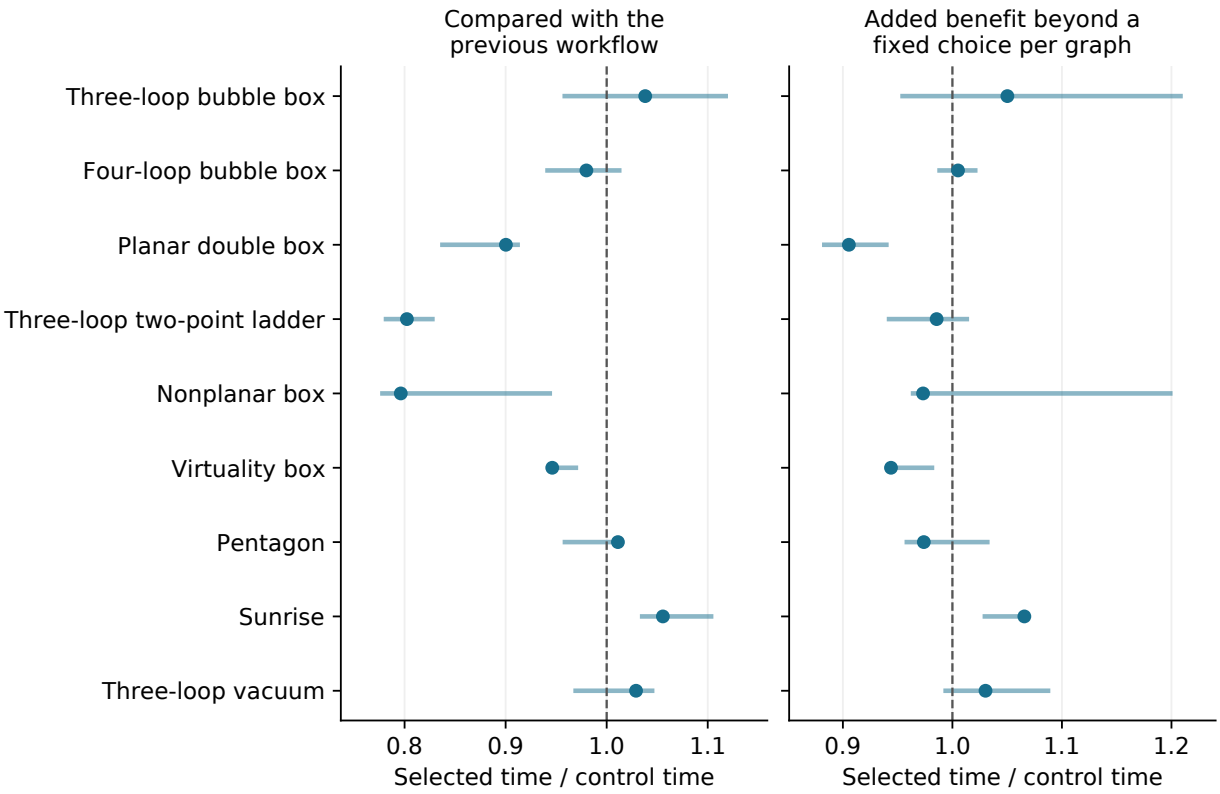


Figure 16: Selected-portfolio time relative to the previous workflow and the graph-specific fixed portfolio. Points divide sums of batch median times. Horizontal segments show the range of the three corresponding repetition totals; they are descriptive ranges, not confidence intervals. Values below one indicate a saving.

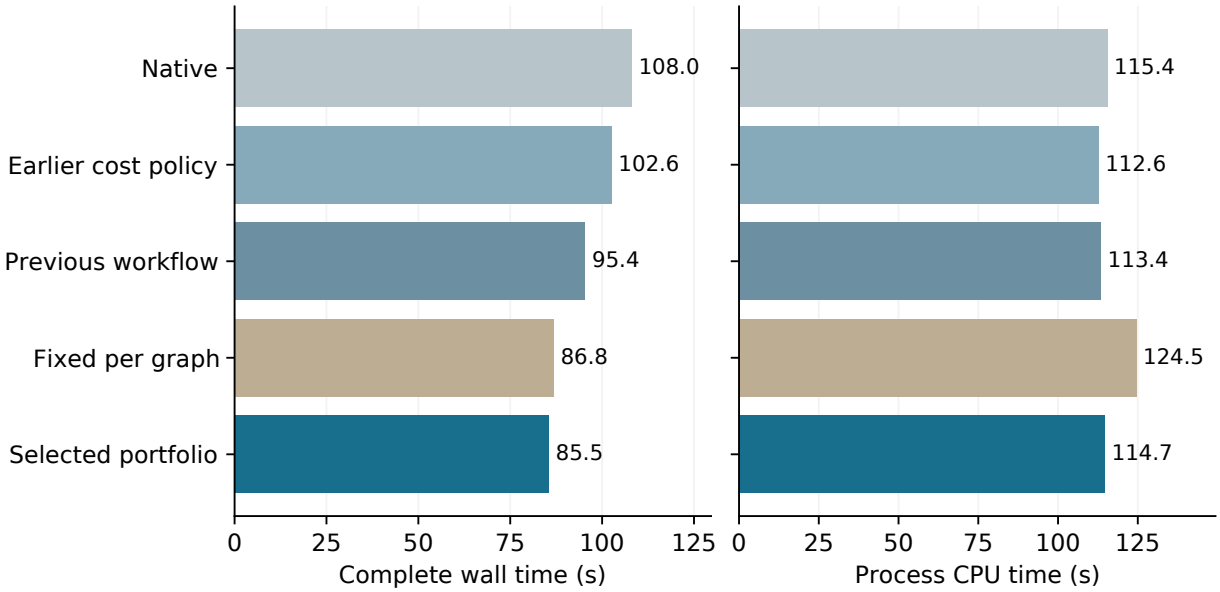


Figure 17: Complete wall time and recorded process CPU time for the same reserved batches. The five methods share exact preprocessing and a two-core allocation. CPU time counts the main driver, Kira child processes, and the timed part of the previous-workflow service. Its message conversion lies outside that service's CPU counter but remains in complete wall time. Initial loading lies outside the repeated timings.

but the incomplete control supplies no measured overall speedup factor. With file indexing already enabled, projection changes its complete time from 98.408 to 28.197 seconds, a factor of 3.49. These comparisons include exact implementation changes and therefore do not measure an isolated ML gain. Figure 29 and Table 34 preserve the intermediate controls in the earlier cost study. The common-preprocessing comparison in Table 19 measures the subsequent portfolio choices separately.

The measurements support the selected strategies in the stated domain. Timing differences can remain when two methods choose the same strategy, and the three repetitions do not establish a significant gain for every graph. General higher-loop scattering, arbitrary symbolic kinematics, automatic discovery of a minimal master basis, and globally optimal elimination paths remain outside the demonstrated scope.

F Reducing preparation around the learned choices

The previous comparison leaves a substantial cost even after the model has chosen a useful seed region. On a four-loop development batch, the actual row operations take a small fraction of the total time, while sector symmetry preparation and repeated equation generation dominate. We therefore retain the trained policies and change the exact work that surrounds their choices. This comparison tests whether better integration can make the same predictions more useful without adding another model or changing its weights.

F.1 Avoiding unused polynomial terms and repeated equations

Kira uses parametric polynomials to identify zero sectors and sector symmetries [4, 18]. Each inverse propagator has a Schwinger parameter, an auxiliary variable in this representation. An auxiliary inverse propagator can occur in a numerator while its index remains nonpositive throughout the declared sector domain. Its Schwinger parameter then vanishes in the sector polynomials used for these checks. The preceding implementation first constructs the full polynomials and removes the unused parameter terms from the zero-sector criterion. We now apply this substitution before polynomial construction and use the resulting restricted polynomials for both zero-sector and symmetry checks. Appendix Z proves that this change gives the same polynomial in every admitted sector. The full propagator list remains available when Kira forms IBP identities and transforms numerators.

Kira also constructs selected symbolic equations to check their remaining masters, then constructs those equations again for the final reduction. We keep the unmodified exact rows from the first construction. When the next selection returns the identical ordered list of source equations, the calculation uses those saved rows. If the selection changes or the storage allowance is exceeded, Kira regenerates the equations normally. The saved rows exist only within that reduction; no previous target answer supplies a later result. This change removes a repeated calculation of an established identity. It does not replace equation selection or symbolic elimination.

The application uses the same trained portfolio, graph-specific predictors, thresholds, seed settings, worker choices, and recovery rules as Section E. We fit no model in this comparison. Thus any change relative to the previous application measures the exact integration changes and timing variation. A matched fixed-portfolio control measures the additional effect of the outer prediction. The earlier separate service remains available for the few requests assigned to it, and its full cost remains in the measurement.

F.2 Fresh targets and matched controls

We reserve 26 new batches from the same nine graph structures, with one to three targets per batch and loop orders one through four. The rational cases use a fresh parameter profile. The symbolic nonplanar and pentagon cases move the raised denominator power to a new position, while the virtuality box has two singly dotted denominators. Appendix Z specifies these inputs. The frozen range guard can select native settings when a new target pattern lies outside its training range.

The first reference reduction uses expanded seeds with the revised preparation. A second calculation uses the previous executable and a larger numerator-rank limit to check the resulting coefficients in the same basis. Each timed method then receives the target and that prescribed basis. Reference coefficients enter only the comparison after the calculation. The test checks reduction to this basis, rather than proving that the basis is minimal.

We compare four methods: the previous application; native choices with revised preparation; a graph-specific fixed portfolio with revised preparation; and the selected portfolio with revised preparation. The fixed portfolio can contain an earlier learned policy, so it isolates the added outer choice. All methods use the same two-core allocation, memory limits, and exact checks. Native choices use one Kira worker, while the portfolios retain their chosen one or two workers. The revised controls share the application's input checks and portfolio proposal; the native and fixed controls then override that proposal. We report the effect of removing this common overhead below. Each attempt starts a fresh Kira process with fresh temporary files. Three repetitions rotate method order. Complete time includes decisions, input preparation, algebra, coefficient checks, cleanup, and unsuccessful attempts. Initial model loading, reference construction, and basis discovery remain outside these repeated timings.

F.3 Results and attribution

Table 20 sums the median time of each batch. The revised selected application takes 64.183 seconds, compared with 79.784 seconds for the previous application. This is 19.6% less time on the measured collection. The selected actions agree with the previous application for every batch and repetition, so this improvement comes from the exact preparation changes around the existing models. Summing all repetitions gives 19.6% less time than the previous application.

Table 20: Complete times in seconds, summed over batch medians. The previous column uses the earlier preparation. The other columns enable both exact changes for direct Kira attempts; the earlier separate service retains its preparation. Fixed denotes the validation-selected portfolio choice for each graph.

Graph	Previous	Native	Fixed	Selected
Sunrise (2 loops)	1.742	1.706	1.806	1.793
Planar double box (2)	6.155	8.501	7.123	5.953
Vacuum (3)	2.788	2.678	2.678	2.636
Two-point ladder (3)	13.393	23.400	12.917	11.581
Nonplanar box (2)	19.794	28.662	20.478	20.052
Pentagon (1)	3.094	2.989	3.003	3.022
Virtuality box (1)	2.172	2.216	2.232	2.229
Bubble box (3)	2.225	1.617	1.735	1.647
Bubble box (4)	28.421	16.078	15.409	15.271
Total	79.784	87.847	67.381	64.183

With each candidate implemented in the same way for all controls, the selected method uses 26.9% less time than native choices and 4.7% less time than the fixed portfolio. Across all repetitions, its difference from the fixed portfolio is 5.0% less time. These controls keep the effect

of learned decisions separate from exact reuse. The graph results in Figure 18 show the measured savings and regressions; a smaller total does not imply that every batch becomes faster.

The common interval for input checks and the outer proposal is at most 2.854 milliseconds per revised request. Removing this entire measured interval from the native and fixed controls gives optimistic totals of 87.828 and 67.363 seconds, respectively. The selected method then uses 4.7% less time than this optimistic fixed control. This subtraction removes useful checks as well as the unused proposal and therefore serves as a sensitivity bound, rather than a separately timed implementation.

All 312 timed reductions and 26 further basis-only applications reproduce their checked reference coefficients exactly. The reference checks, timed tests, and applications compare 8,086 target–master coefficients. The revised selected method needs 6 recovery attempts across its timed tests. The recorded process-tree memory peak among the timed reductions is 961.8 MiB. The selected method takes 92.782 seconds in summed batch median CPU time, compared with 107.054 seconds for the previous application. The bounded replay mechanism activates 117 times across the timed controls and retains at most 0.221 MiB of accounted row data in an activation.

These results support the revised application for this measured collection. They test new parameters and target patterns within known graph structures, with supplied master bases. They do not establish general high-loop scattering performance or a globally shortest reduction. The earlier row-model result remains unchanged, so the application continues to use native row elimination.

G How far the improvement extends

The preceding gain was measured on known graph structures with a supplied master basis. To test its wider use, we now vary the physical structure as well as the loop order and include calculations that find their own basis. We keep the deployed models fixed so that these measurements test the current method on new inputs. The study contains 90 native-family requests, of which 54 fit the application's current index limit. It also tests a separate restricted vacuum construction at every loop order from one through twelve. This is a finite coverage study; a loop-order label alone does not establish coverage of all graphs at that order.

G.1 Physical families and complete-time controls

The native grid includes vacuum families with one or several masses, massless and massive two-point families, four-point ladders, and boxes with bubble subgraphs. We add planar, nonplanar, and five-point examples with independent symbolic kinematic parameters. Appendix AA gives the denominator definitions, parameters, and targets. The 82 initial requests contain a sector-corner integral, one raised propagator power, and one numerator integral when an auxiliary denominator is available. Some massive two-point batches return every target unchanged as a master. We therefore preserve those results and add 8 separate batches containing every distribution of up to two dots over the physical denominators. The additional batches cover all six loop orders accepted for that two-point family, plus symbolic cases at one and two loops. Their inputs are fixed before their own comparison, and no model is refitted from either stage.

We first calculate a reference with expanded seeds and the revised implementation. A second calculation uses the previous executable and a larger numerator-rank bound, with the reference basis prescribed. Both must complete and reproduce the same coefficients before a case enters the repeated comparisons. This checks the reductions in the stated basis; it does not prove that the basis has the smallest possible size. Every native attempt has a 120-second allowance, a 6 GiB address-space limit, and a 5,200 MiB process-tree memory limit. An attempt that reaches a limit remains a failed result. Inputs with more than 31 indices exceed the current application's

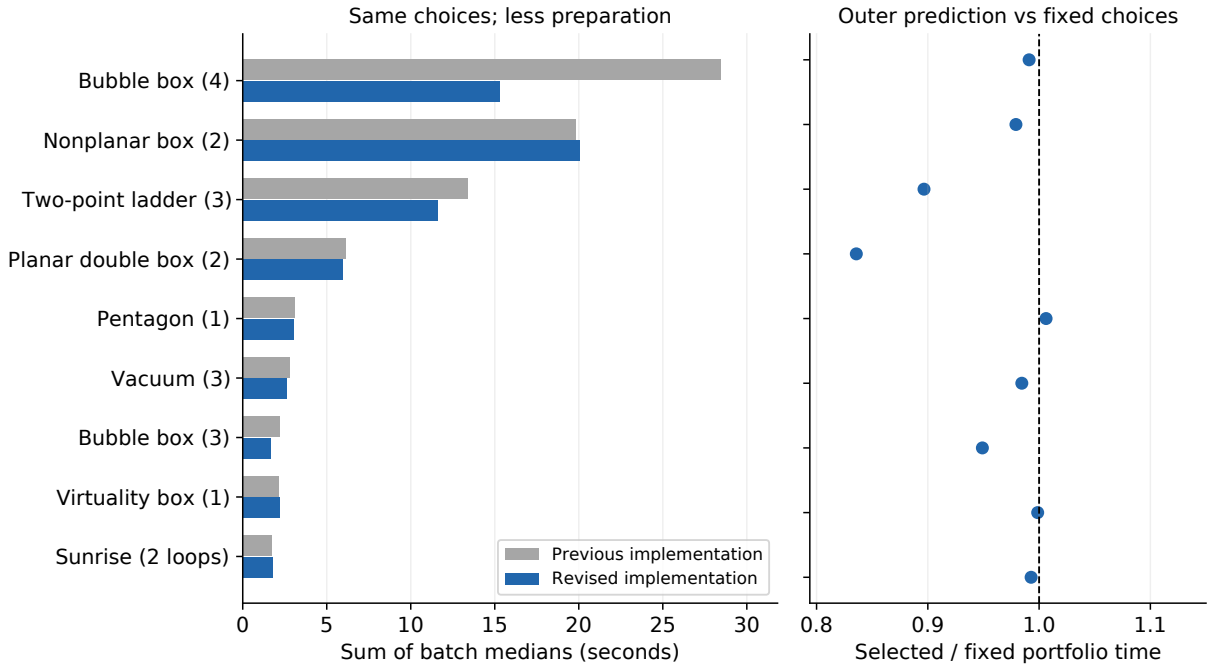


Figure 18: Left: the same learned actions run with the previous and revised preparation. Right: the selected-to-fixed time ratio with revised preparation; the dashed line marks equal time. Each point or bar uses sums of batch medians within a graph. The right panel isolates the additional outer prediction.

accepted domain and receive no timing estimate.

The first comparison runs the saved original Kira executable, the previous improved implementation, and the revised implementation with the same expanded seed region. These calculations receive no master list, so their times include basis discovery, equation generation, elimination, coefficient checks, and cleanup. This comparison measures the complete effect of the implementation. It makes no learned decision. The second comparison supplies the checked basis and runs the previous learned application, the revised learned application, and a native-choice control with revised preparation. Its times also include prediction and unsuccessful attempts. Both comparisons use three repetitions with rotated method order and fresh Kira processes. Initial caller imports and model loading remain separate from the repeated application times.

An input audit finds two box-with-bubble definitions that match saved training structures exactly but carry new family names. The initial application treats these names as unknown. We add a lookup that resolves a unique match of the ordered momentum forms, sector definition, and zero or nonzero mass pattern before prediction. Numerical mass and kinematic values remain free, as in the existing model signature, and all original feature-range checks remain active. Unmatched structures retain the native fallback. The lookup uses no measured time or reference coefficient. We repeat the paired application comparison for both names and use these corrected pairs in the application totals. The initial unknown-name measurements remain available separately. This correction allows the existing models to recognize their own domain; it does not train a new model or infer equivalence between different momentum representations.

G.2 Coverage and measured time

Figure 19 shows the rational grid, including its failures and unsupported inputs. The reference checks pass for 41 requests, and 41 requests complete all three revised full-calculation repetitions. Of these, 39 contain a nontrivial reduction and 2 contain only identity outputs. We exclude identity-only requests from speedup ratios. The supplied-basis selected application completes all repetitions for 41 requests. The remaining requests retain their stated failure or support-limit

classification. The timings therefore do not establish a nontrivial reduction for every entry in the grid.

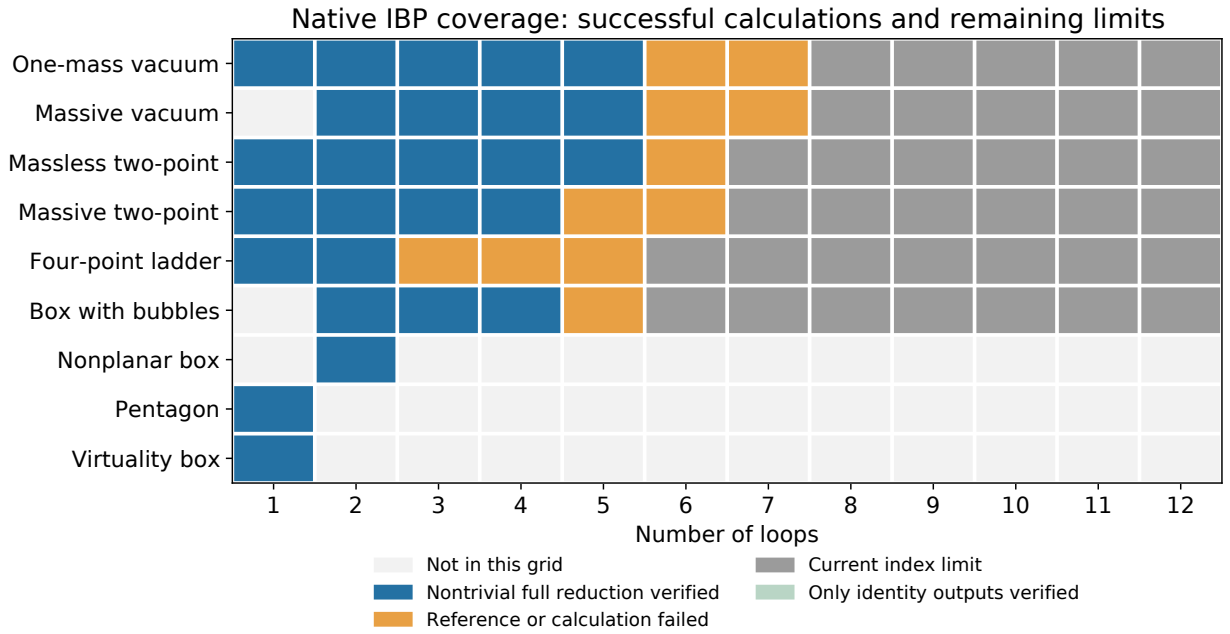


Figure 19: Native IBP coverage in the rational grid. Blue requires a checked reference, a nontrivial reduction, and three exact revised full calculations including basis discovery. Pale green records only identity outputs. Orange records a failed reference or calculation within the stated limits. Gray records the current index limit. A cell takes the strongest verified status across initial and additional batches. Blank cells are absent from this grid; the two coincident one-loop definitions appear only once. Symbolic requests enter the timing table and the text totals.

Table 21 uses a separate matched set for each comparison. A request enters a row only when both methods complete all repetitions and the reduction contains a nontrivial coefficient. We sum the median time of each request, so a smaller ratio means less total time on that completed set. The revised full calculation takes 88.2% less time than the saved original Kira executable and 54.3% less time than the preceding improved implementation on their respective matched sets. The supplied-basis selected application takes 54.1% less time than the previous application and 0.1% less time than the revised native control. These aggregates retain slower individual requests and do not turn unsuccessful cases into speedup estimates.

Table 21: Summed request median times in seconds. The revised column uses full revised calculations for the first two rows and the selected application for the last two. Each row names its own control and completed request count.

Control	Requests	Control time	Revised time	Ratio
Full: original Kira	37	369.363	43.750	0.118
Full: previous implementation	39	146.541	66.897	0.457
Basis supplied: previous application	39	146.342	67.195	0.459
Basis supplied: revised native control	39	67.269	67.195	0.999

The revised native control passes through the same input checks and portfolio proposal before overriding the proposed action. For a conservative comparison, we subtract that entire recorded interval and the family-name lookup from its time, while retaining every selected-application cost. The largest removed interval is 2.124 milliseconds. This gives an optimistic native total of 67.259 seconds, against which the selected application takes 0.1% less time. The subtraction also removes useful input checks, so it supplies a sensitivity bound rather than a separately timed

implementation.

The selected application and revised native control differ by only 0.1% in the summed medians. With only three repetitions, this small difference does not establish a reliable additional learned gain on the wider grid. The larger improvement against the preceding application includes the exact preparation changes.

The repeated native comparisons contain 752 recorded requests, with 750 exact completions and 2 unsuccessful requests. The largest recorded process-tree memory use among these timings is 961.8 MiB. The grid includes new graph definitions, so the existing domain guard often retains native choices. This is part of the deployed method and prevents us from describing the full implementation gain as a gain from newly learned decisions.

Table 22 also includes unsuccessful first timed requests. Its elapsed totals describe the cost actually incurred on that common reference-verified collection. They are not completed-reduction speedups, since a failed request supplies no accepted answer. This view complements the paired timing table and prevents completion failures from disappearing in an aggregate improvement claim.

Table 22: First timed requests on the reference-verified collection. Elapsed seconds include failed requests. Later repetitions are excluded from this table so that each method receives one request per case.

Method	Requested	Exact completions	Elapsed seconds
Full original Kira	41	39	623.836
Full previous implementation	41	41	148.606
Full revised implementation	41	41	68.568
Previous application	41	41	150.733
Selected application	41	41	68.497
Revised native control	41	41	68.721

G.3 Every tested loop order in the restricted vacuum class

The existing vacuum recurrence has a different domain: unit powers on the massless lines and several massive denominators with the same total loop momentum. We construct three new mass profiles with two, three, or four distinct masses and six target exponent vectors per profile. We compare the saved original Kira, dense and selected supplied-identity systems, the frozen learned identity selector, and the deployed exact recurrence. Each method receives the same targets and passes the independent coefficient check derived in Appendix L. The individual requests cover every loop order from one through twelve. Separate shared requests ask for all twelve orders together.

The study records 585 exact completions in 585 vacuum requests. Table 23 separates individual-loop requests from shared requests. In the individual-loop set, learned identity selection takes 8.2% more time than the fixed identity rule. The deployed recurrence takes 99.8% less time than the saved original Kira calculation. This gain uses the exact restricted-family relations and their implementation; the deployed recurrence uses its existing fixed policy. The native study and this supplied-identity study therefore establish different forms of coverage.

The broader tests identify where the current application completes an exact reduction, where its cost changes, and where its support or resource limits stop the calculation. They retain the earlier known-graph gain while restricting its interpretation to measured inputs. General high-loop scattering, arbitrary numerator rank, cuts, symbolic propagator powers, and every possible mass or kinematic assignment remain outside a universal verified claim.

Table 23: Restricted vacuum timings in seconds, summed over request medians. There are 36 individual-loop requests and three shared requests, each repeated three times. The original control solves separately for each requested loop order. The other methods can use the exact shared variable $z = L(d-2)$. All times include coefficient verification.

Method	Individual loops	Shared loops
Original Kira, separate loop orders	122.179	129.306
Improved Kira, dense identities	14.626	1.310
Fixed identity selection	13.311	1.398
Learned identity selection	14.399	1.327
Deployed exact recurrence	0.189	0.069

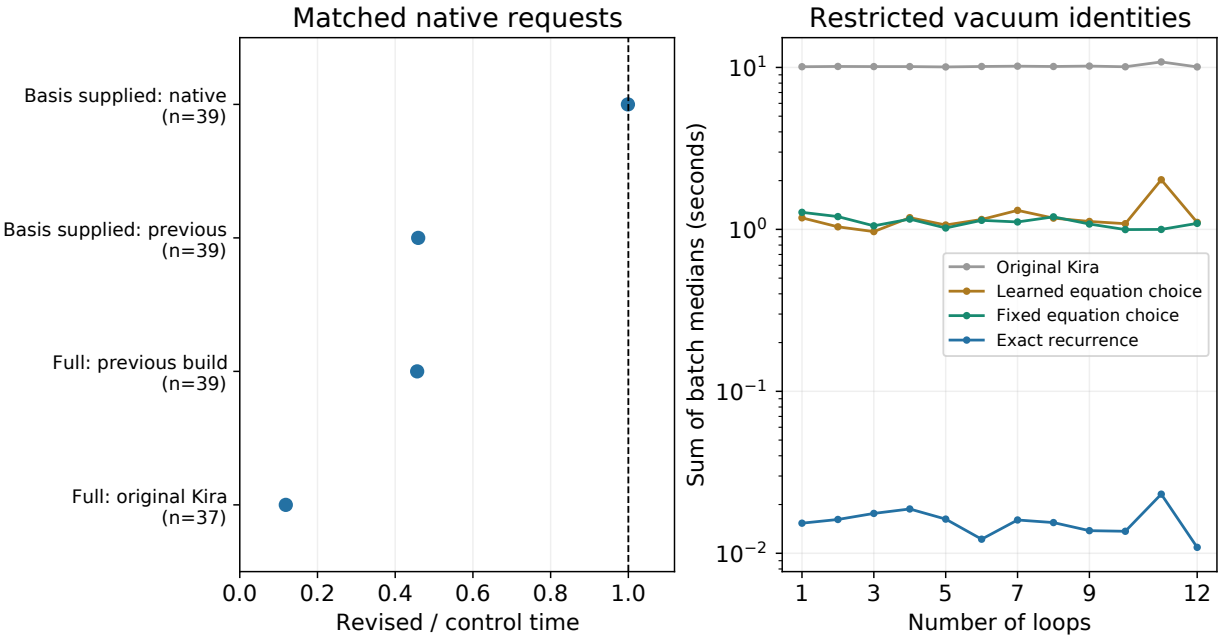


Figure 20: Left: revised-to-control time ratios on matched completed native requests; the dashed line marks equal time. The request count changes with each pair's completed coverage. Right: restricted vacuum time at each loop order, summed over the three mass profiles. The logarithmic vertical axis keeps the exact recurrence and Kira-based calculations visible. These panels use different integral definitions and cannot be combined into a universal speedup.

H Earlier models for complete calculations

The local-model tests show that useful equations, accurate labels, and faster individual stages do not always lead to a faster complete reduction. We therefore change the training objective to measured calculation time and move the decision to the start of a target batch. The following studies examine the available strategies, the benefit of separate models for each graph, the treatment of seed expansion, and the number of workers used by Kira.

H.1 Learning from complete reduction time

The scattering test shows why a useful equation label is an incomplete training objective for acceleration. A filter can retain sufficient equations and still add more overhead than it saves. We therefore train a model to predict complete reduction time and let it choose the initial seed region, an optional seed or equation filter, and an integral ordering. The model makes one choice for the whole target batch before Kira begins its exact calculation.

To evaluate that choice fairly, we also need a baseline with the same surrounding implementation. We combine the earlier equation filter with an existing improvement in temporary data handling, which reduces execution cost without changing the equations or arithmetic. The *integrated native control* uses this implementation with Kira's own decisions. Comparing the learned strategy with that control measures the benefit of the prediction beyond the shared implementation changes.

For a target batch, let $B = (r_{\max}, s_{\max}, q_{\max})$ contain the largest positive index sum, numerator rank, and dot count. A dot is one denominator power above one. Table 24 lists the strategies considered, and Appendix Q defines the quantities and derives the cost models. The seed limits and ordering options are provided by Kira [19]; ordering five changes the priority of sectors while retaining the within-sector preference of ordering one. All strategies must return the same prescribed master basis.

Table 24: Strategies available to the complete-time model. Negative seed bounds are replaced by zero. Native row choices remain active.

Strategy	Initial seed region and selection
Integrated native	Bounds B; native choices
Smaller region	Bounds $B - (1,1,1)$; native choices
Fewer dots	Bounds $B - (1,0,1)$; native choices
Equation filter	Bounds B; saved equation tree
Seed filter	Bounds B; saved seed tree where available
Different ordering	Bounds B; Kira ordering five
Larger region	Bounds $B + (1,1,1)$; native choices

A smaller region can avoid unnecessary equation generation, while a larger region can avoid a failed first attempt when the target's own index limits are insufficient. If the initial strategy fails the basis check, the calculation tries native choices at B and then at $B + (1,1,1)$ when needed. The native control also enlarges an insufficient region, and every failed attempt contributes to the training and test time. For the four earlier multiloop graphs, the seed-filter option uses native choices because this implementation has no separate seed tree for them. Its equation-filter option uses the earlier trees for those graphs and the newly trained trees for the three additional scattering structures.

The model receives eighteen features describing the graph, symbolic variables, number of targets, index powers, and sectors. For each strategy, its training label is the logarithm of the complete-time ratio to integrated native choices. We compare a constant predictor, six regression trees, three ridge-regression models, and two nearest-neighbour models. Regression trees follow

the squared-error construction of Ref. [9]; Appendix Q specifies all candidates. Validation selects ridge regression with penalty $\lambda = 10$ and decision margin $\gamma = 0$, giving validation ratio 0.993.

The best fixed strategy selected from training is integrated native reduction. Consequently, the integrated-native and best-fixed controls use the same settings in independent trials, and differences between their times indicate execution variation. The seed and equation trees retain their earlier weights throughout this study, and learned row selection remains disabled.

The data contain 31 training batches, 17 validation batches, and 31 test batches from the sunrise, planar box, three-loop vacuum, three-loop ladder, nonplanar box, pentagon, and box with external virtuality. Rational batches use different masses or invariants across the splits, while additional symbolic batches use different target lists. These are new inputs within seven known graph structures at one through three loops. The recurrence study through twelve loops remains a separate result for its restricted vacuum family.

Each training and validation cost uses three repetitions. Testing uses five repetitions to compare the preceding native implementation, integrated native choices, the best fixed strategy, and the learned strategy. The controller and models are loaded before timing, but every attempt starts a fresh Kira process. Timers include input preparation, prediction, native IBP generation, algebra, exact checks, cleanup, and retries. They exclude training, enlarged reference reductions, and later recording of results. Additional application checks measure cold startup and compare returned coefficients after a reduction supplied only with its targets and basis. For an unknown graph or target complexity outside the training range, the learned application uses native choices; the same guard applies to combined requests.

H.1.1 Results for individual target batches

All 620 timed calculations pass exact checks of 59,640 coefficient entries, and all 31 applications supplied without reference coefficients pass their subsequent comparisons. Each timed method makes fifteen fallback attempts. Thus this model changes some initial strategies but does not remove the remaining cases that need a second attempt.

Relative to the preceding native implementation, the learned strategy has time ratio 0.9903, with descriptive interval $[0.9796, 0.9996]$, or about 1.0% less time. Its ratio to integrated native choices is 0.9965 with interval $[0.9875, 1.0070]$, and its ratio to the best-fixed control is 1.0026 with interval $[0.9979, 1.0081]$. Both comparisons with the matched integrated controls include equal time. They therefore provide no reliable additional saving from the learned choices. The two identically configured controls have ratios 0.9938 and 0.9878 to the preceding native implementation, illustrating the scale of execution variation.

Table 25 reports the results by graph. The aggregate averages logarithmic ratios within each graph and then gives all seven graphs equal weight, as derived in Appendix Q. The smallest learned-to-native ratio is 0.9654 for the three-loop vacuum, while the ladder and nonplanar box take slightly more time. These results motivate testing more specific strategies for each graph rather than enabling the common model for all inputs.

Table 25: Complete-cost results. Native seconds are geometric means of batch median times. The other entries are geometric mean time ratios; a value below one means less time.

Graph	Native (s)	Integrated/native	Learned/native	Learned/integrated
Sunrise	0.502	1.0123	0.9932	0.9811
Planar box	1.621	0.9906	0.9771	0.9864
Three-loop vacuum	0.726	0.9745	0.9654	0.9907
Three-loop ladder	3.535	1.0146	1.0013	0.9869
Nonplanar box	3.119	1.0075	1.0073	0.9998
Pentagon	0.599	0.9758	0.9942	1.0189
Box with virtuality	0.484	0.9824	0.9944	1.0122

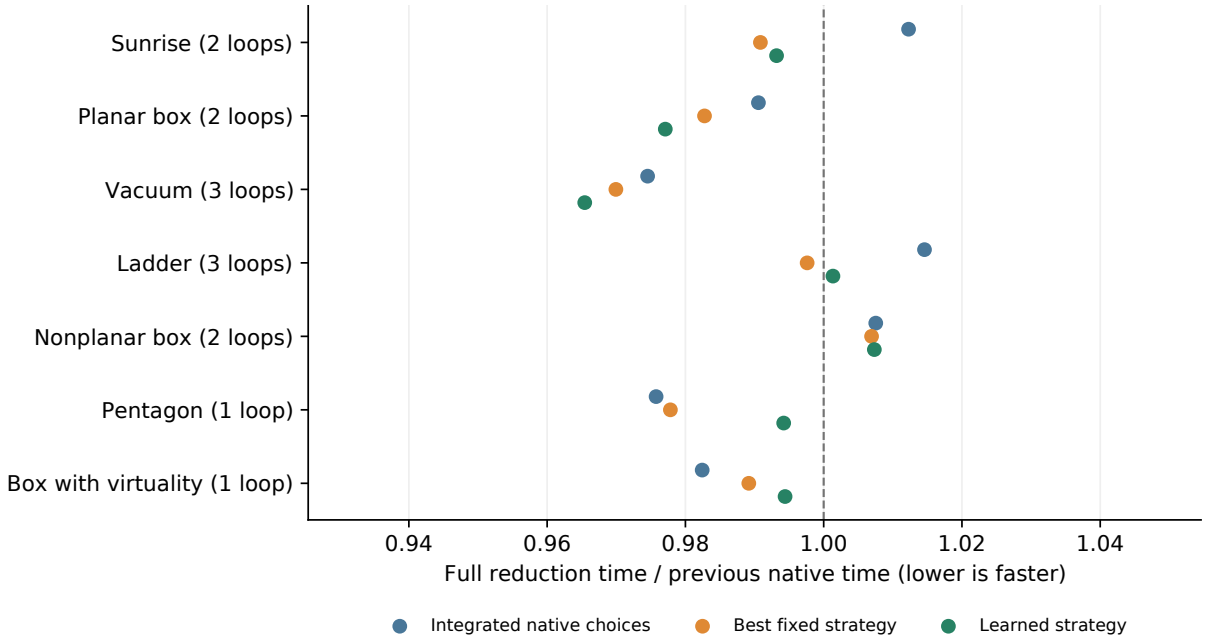


Figure 21: Learned choices and the native controls have similar complete times. The best-fixed and integrated controls use the same configuration in separate trials. The plot shows variation across graph structures; the text gives the aggregate uncertainty intervals. The axis focuses on the measured range and retains the equal-time line.

H.1.2 Sharing a reduction between compatible requests

Before changing the predictor, we test whether repeated work can be removed from the way Kira receives targets. When two available requests have the same family and kinematics, their target lists can be combined and reduced together. The resulting coefficient rows are then returned to the appropriate request. This shares family preparation and algebra between the lists. Appendix Q defines the ten request groups and derives this separation from linearity; the model and its settings remain fixed.

Five repetitions of four methods give 200 complete calculations, all of which pass exact checks. Joint native reduction has ratio 0.5252 to two separate native reductions. With the integrated implementation and native choices, the ratio is 0.5175 with interval $[0.5073, 0.5346]$, giving 48.3% less time or speedup 1.93 with interval $[1.87, 1.97]$. Every tested graph group benefits from the shared reduction. Its integrated-native ratios range from 0.5040 to 0.5679, corresponding to savings of 43.2–49.6%.

The joint learned strategy has ratio 0.5152 to separate native requests, but its ratio to joint integrated native choices is 0.9955 with interval $[0.9810, 1.0058]$. As Table 26 and Figure 22 show, the main gain comes from sharing the calculation; the predictor adds no reliable improvement over the matched joint control. The standard application therefore uses the fixed strategy selected from training, which retains native choices. Ten checks of each joint application and ten checks of the single-batch application supply only a basis and verify the returned coefficients independently, in addition to the 31 application checks above.

Table 26: Joint-request time ratios and descriptive intervals over the seven graph groups. Each group contains two compatible target requests.

Comparison	Time ratio	95% interval
Joint native / separate native	0.5252	$[0.5130, 0.5420]$
Joint integrated / separate native	0.5175	$[0.5073, 0.5346]$
Joint learned / separate native	0.5152	$[0.5022, 0.5322]$
Joint learned / joint native	0.9810	$[0.9657, 0.9946]$
Joint learned / joint integrated	0.9955	$[0.9810, 1.0058]$

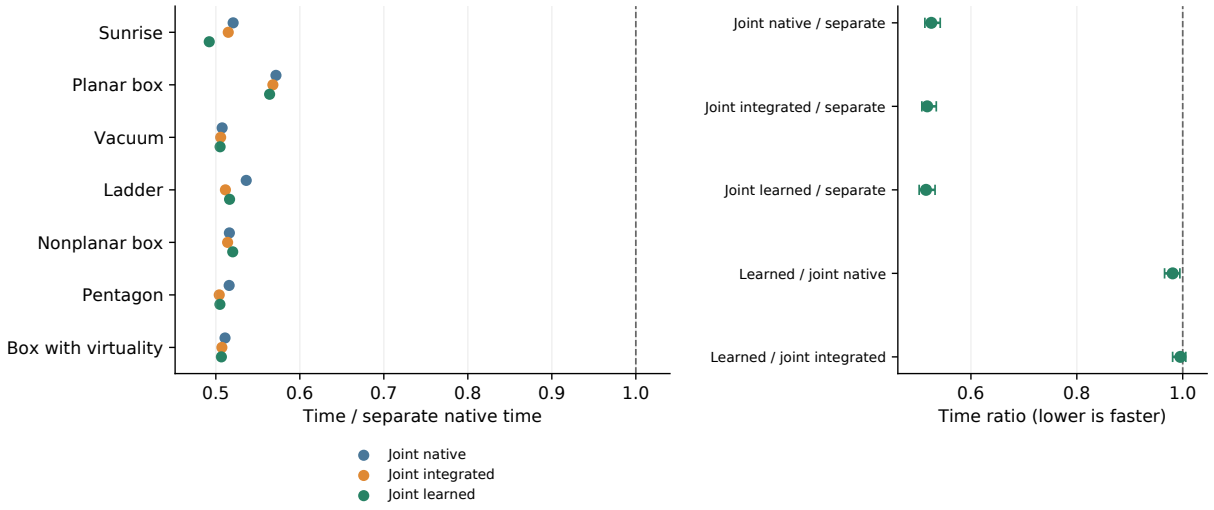


Figure 22: Combining compatible requests gives the main saving. Left: ratios by graph relative to two separate native reductions. Right: aggregate ratios and descriptive intervals. Comparing learned choices with joint integrated native choices gives no reliable extra saving. The panels retain the equal-time reference.

The joint comparison reuses the main test's reserved parameter points, so it provides an additional comparison on that population. Its saving requires requests that can share one calculation. Transfer to new graph structures, isolated arbitrary integrals, and general scattering beyond three loops remains outside this result. The next study returns to individual batches and tests whether separate models and a wider set of seed strategies give a measurable gain.

H.2 Choosing a strategy separately for each graph

A common predictor may miss differences in the work required by different graph structures. We therefore fit separate predictors for the seven graphs and enlarge the available seed strategies. The added choices can restrict seeding to maximal target sectors or reduce numerator seeding in lower sectors through Kira's truncation option [5]. Each method first enforces a valid positive-index bound for its declared sectors, and the final basis check and fallback remain in place.

For each graph, ten candidate models predict the complete-time ratios of nine strategies from nine target and family features. The candidates include regression trees, a bagged ensemble of trees, regularized linear models, and nearest-neighbour models. Their cost labels include unsuccessful attempts and retries. The data contain 36 training batches, 18 validation batches, and 36 reserved test batches at new rational parameter values, with d symbolic throughout. Targets include propagator powers up to three, lower sectors, and auxiliary numerator indices -2. Appendix R gives the exact input definitions and selection procedure.

Training and validation contain 2,798.5 seconds of measured reductions, while fitting all seventy candidates takes 0.86 seconds. These offline costs exclude reference construction, development, and replaced measurements and are not included in application time. Validation enables a new predictor only if it saves at least five percent against the fastest available control. The controls are native choices, a deterministic sector-and-truncation rule, and a fixed strategy for each graph selected from training; the fixed strategy may itself use the older equation filter. This requirement activates predictors for three graphs and retains a control for the others. Unknown graphs and features outside the training range use native choices.

Five repetitions of four methods give 720 test calculations, all of which pass exact coefficient checks. With equal weight for each graph, the selected method has ratio 0.9368 to native choices, with descriptive interval $[0.8867, 0.9878]$, corresponding to 6.3% less time. Its ratio to a fixed strategy for each graph is 0.9441 with interval $[0.8992, 0.9863]$. This second comparison isolates the benefit of selecting a strategy from the targets beyond choosing one setting for the whole graph.

Table 27 shows lower time for five of the seven groups. The enabled trees save 14.2% for the planar box, 12.4% for the ladder, and 13.8% for the nonplanar box. The vacuum and box with external virtuality take 3.7% and 0.8% more time, respectively, while retaining control strategies throughout. In particular, the vacuum uses native settings, so its difference includes the extra selection overhead and execution variation. Across 180 timed trials, the selected method needs no retries and native choices require 40; these costs are included in the ratios.

Table 27: Reduction time with a separate strategy for each graph. Times are geometric means of batch medians in seconds, and the ratio divides selected time by native time. The strategy column identifies the enabled model or retained control.

Graph	Strategy	Native	Selected	Ratio
Sunrise	Stronger truncation	0.825	0.811	0.983
Planar box	Depth-two tree	3.410	2.925	0.858
Vacuum	Native	1.033	1.072	1.037
Ladder	Depth-one tree	5.144	4.506	0.876
Nonplanar box	Depth-two tree	6.234	5.376	0.862
Pentagon	Target sectors	1.040	0.989	0.951
Virtuality box	Sectors and truncation	0.808	0.815	1.008

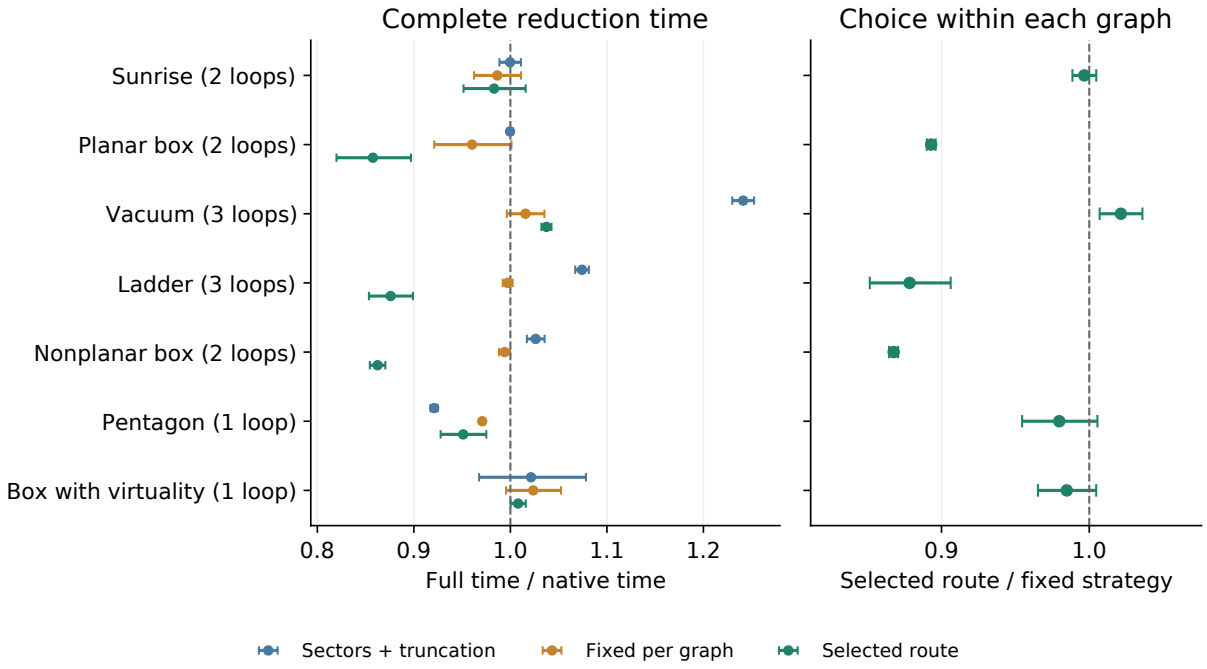


Figure 23: The effect of choosing a separate policy for each graph. Left: complete-time ratios to native choices. Right: the selected method relative to one fixed strategy per graph. Bars show the range across two reserved parameter profiles; they are not significance intervals. Both panels retain the equal-time reference.

The timed trials check 14,040 coefficient entries. A further 36 applications supplied only with the master basis and seven additional application checks verify 945 coefficients independently. These checks establish correct completion for the measured inputs. The graph structures and target patterns are known during training, and each graph has only two training parameter profiles. The reserved profiles therefore test new kinematic values within these inputs, leaving new target structures and general scattering beyond three loops for further study.

H.3 Expanding seeds selectively and using an intermediate basis

The graph-specific study identifies symbolic equation regeneration as the largest measured solver stage. Some target batches need seed limits beyond those of the targets themselves, and a general enlargement can generate many unnecessary equations. We therefore test whether selective increases in the limits, or a reduction through an intermediate master basis, can lower this remaining cost.

The candidate set contains six ways to increase selected seed limits and two ways to reduce first to a temporary basis. In the latter case, Kira reduces the remaining temporary masters to the prescribed basis, and exact multiplication combines the two coefficient matrices. The final basis requirement is the same for every method. Appendix S defines the candidates and derives this conversion, which allows the first target reduction to use a smaller system when the remaining basis conversion is inexpensive.

For development batch c, let $B_c = (r_c, s_c, q_c)$ denote valid initial limits on the positive index sum, numerator rank, and number of extra denominator powers. We retain the fastest new candidate for a development batch only if it saves at least ten percent against the graph-specific method in Section H.2. The resulting candidate set contains that preceding method, full-sector basis completion, and the selective increments $B_c + (1, 0, 0)$, $B_c + (0, 1, 0)$, and $B_c + (1, 1, 0)$. Development results select the options but do not enter the reserved comparison.

The new data contain 36 training batches, 18 validation batches, and 36 test batches from the same seven graphs and target patterns, with new rational kinematics and symbolic d. Ten predictors per graph use the same nine features to predict time relative to the preceding method. Validation compares each predictor with that method and with a fixed new strategy selected from training for each graph. A predictor is enabled only if it saves at least five percent against the faster control; otherwise that control is used. A fixed candidate may itself call the preceding learned method, so this comparison tests the additional choice among the new options.

Validation enables new predictors for three graphs. The retained training and validation records contain 1,731.7 seconds of measured reductions, and fitting the seventy candidates takes 0.88 seconds. These costs exclude reference construction and development and remain outside the reserved reduction timer.

The test compares native choices, the preceding graph-specific method, the new fixed strategy, and the selected strategy. Five repetitions give 720 calculations, all of which pass exact checks. A further 43 applications also pass when the reduction receives no target reference coefficients. With equal weight for each graph, the selected method has ratio 0.9115 to the preceding method, with descriptive interval $[0.8105, 0.9980]$, giving 8.8% less time. Its ratio to native choices in the same experiment is 0.8516 with interval $[0.7282, 0.9647]$, giving 14.8% less time. Relative to the new fixed strategy, the ratio is 0.9264 with interval $[0.8760, 0.9747]$.

The sum of batch medians answers a different question: how long would this set of requests take when each is counted once? For the 36 reserved batches, this sum is 106.24 seconds for native choices, 96.32 seconds for the preceding method, and 77.03 seconds for the selected method. The geometric ratios above instead give every graph equal weight. Both measures show a saving, although their magnitudes differ because the longer calculations contribute more to the workload total.

The stage measurements explain where part of the saving occurs. Across 180 runs per method, symbolic equation regeneration falls from 138.02 to 80.58 seconds. Neither the preceding nor the selected method makes a fallback attempt, compared with 40 for native choices. Planned second-stage reductions of temporary masters are included in complete time and are counted separately from fallback. The comparison with the fixed strategy supports an additional benefit from the selection procedure, which combines validation-based control choices with predictions for individual target batches.

Table 28: Selective seed expansion compared with the graph-specific method in Section H.2. Times are geometric means of batch medians in seconds. "Previous" denotes that graph-specific method and "New" denotes the seed-expansion method. Each graph has two reserved parameter profiles.

Graph	Policy	Previous (s)	New (s)	New/previous	New/native
Sunrise	Add s	0.596	0.593	0.995	0.979
Planar box	Tree depth 1	2.422	2.062	0.851	0.735
Vacuum	Complete basis	0.928	0.941	1.013	1.016
Ladder	Tree depth 1	4.477	3.007	0.672	0.584
Nonplanar box	Tree depth 1	4.091	3.550	0.868	0.759
Pentagon	Previous	0.723	0.754	1.043	0.998
Virtuality box	Previous	0.580	0.582	1.003	1.004

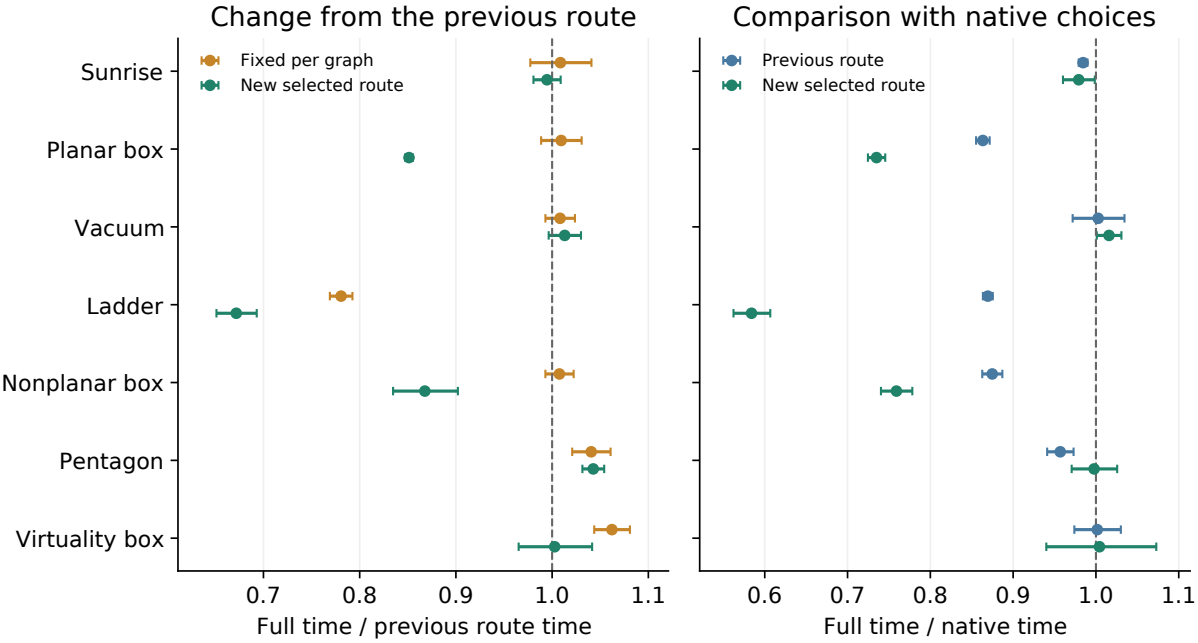


Figure 24: Selective choices change complete reduction time. Left: the new fixed strategy and selected method relative to the preceding method. Right: the preceding and new methods relative to native choices in the same test. Values below one are faster. Bars show the range over the two reserved profiles; they are not significance intervals.

The gain is not uniform across the graphs. Relative to the preceding method, the vacuum takes 1.3% more time, the pentagon 4.3% more, and the box with external virtuality 0.3% more. All reported times include prediction, native IBP generation, basis conversion, exact algebra, checks, cleanup, and fallback. These results measure the additional method directly on new parameter profiles; they do not multiply gains from earlier experiments. Their scope remains known graphs and target patterns with rational kinematics, rather than unrestricted symbolic kinematics or arbitrary high-loop scattering.

H.4 Testing integral and propagator orderings

Selective seed expansion reduces equation-generation work, but the ordering of the resulting integrals can still affect elimination. Kira provides eight integral priority rules and four automatic propagator-ordering schemes [19, 5]. We test whether a model can choose among these settings from the target indices before equation generation, while retaining the seed strategy from Section H.3. The model makes one decision for each batch, and a failed basis check triggers a repeat with the preceding settings. Appendix T defines the candidates and shows why consistent relabelling preserves the integrals.

Development first compares eleven candidates and then tests the pair of best nondefault orderings for each batch. We keep the preceding settings and any batch winner that saves at least ten percent of complete time. If ω is the integral-order number and χ the propagator-scheme number, the retained new settings are $(\omega, \chi) = (2, 1)$, $(7, 1)$, $(5, 4)$, and $(8, 3)$. The preceding method also remains available without a change in its settings.

We fit ten predictors per graph on 36 training batches, select their use on 18 validation batches, and test on 36 reserved batches. The features, complete-time labels, candidate model types, and five-percent validation requirement are the same as in the seed-expansion study. A fixed ordering still uses the preceding learned seed strategy, so the fixed-setting comparison measures the additional ordering decision. Training and validation contain 1,432.8 seconds of reductions, and fitting the seventy predictors takes 0.93 seconds. These offline costs exclude development and reference construction.

None of the predictors passes the validation requirement. The tested selected method therefore uses fixed ordering settings chosen by validation, with no newly enabled ordering predictor. Any difference in its time reflects those settings, their interaction with the retained seed strategy, and execution variation. This distinction is necessary when interpreting the results as evidence about machine learning.

All 720 timed reductions and 43 additional applications pass exact checks; the additional applications receive no target reference coefficients during reduction. With equal weight for each graph, the selected-to-preceding ratio is 1.0141 with descriptive interval $[0.9986, 1.0303]$, giving 1.4% more time. The selected-to-native ratio is 0.8499 with interval $[0.7306, 0.9642]$, giving 15.0% less time, while the ratio to the new fixed strategy is 1.0070 with interval $[0.9811, 1.0327]$. The saving against native choices therefore does not show an improvement over the already learned seed-expansion method.

The sum of median times over the 36 test batches is 110.09 seconds for native choices, 79.61 seconds for the preceding method, 79.53 seconds for the new fixed settings, and 79.68 seconds for the selected settings. On this workload, the selected method takes 0.1% more time than its predecessor. Its additional ordering decision takes a median of 271.1 microseconds per application, and both this cost and the retained seed-policy cost are included in complete time.

Table 29: Ordering settings compared with the seed-expansion method in Section H.3. Times are geometric means of batch medians in seconds. "Previous" retains the earlier ordering, and "New" uses the selected setting. The pair (ω, χ) specifies the integral order and propagator scheme.

Graph	Choice	Previous (s)	New (s)	New/previous	New/native
Sunrise	$(7, 1)$	0.610	0.633	1.037	1.034
Planar box	$(2, 1)$	2.407	2.390	0.993	0.759
Vacuum	Previous	0.975	0.982	1.007	0.983
Ladder	Previous	3.122	3.088	0.989	0.591
Nonplanar box	$(2, 1)$	3.698	3.681	0.995	0.745
Pentagon	$(2, 1)$	0.788	0.820	1.041	0.987
Virtuality box	$(5, 4)$	0.601	0.624	1.039	0.955

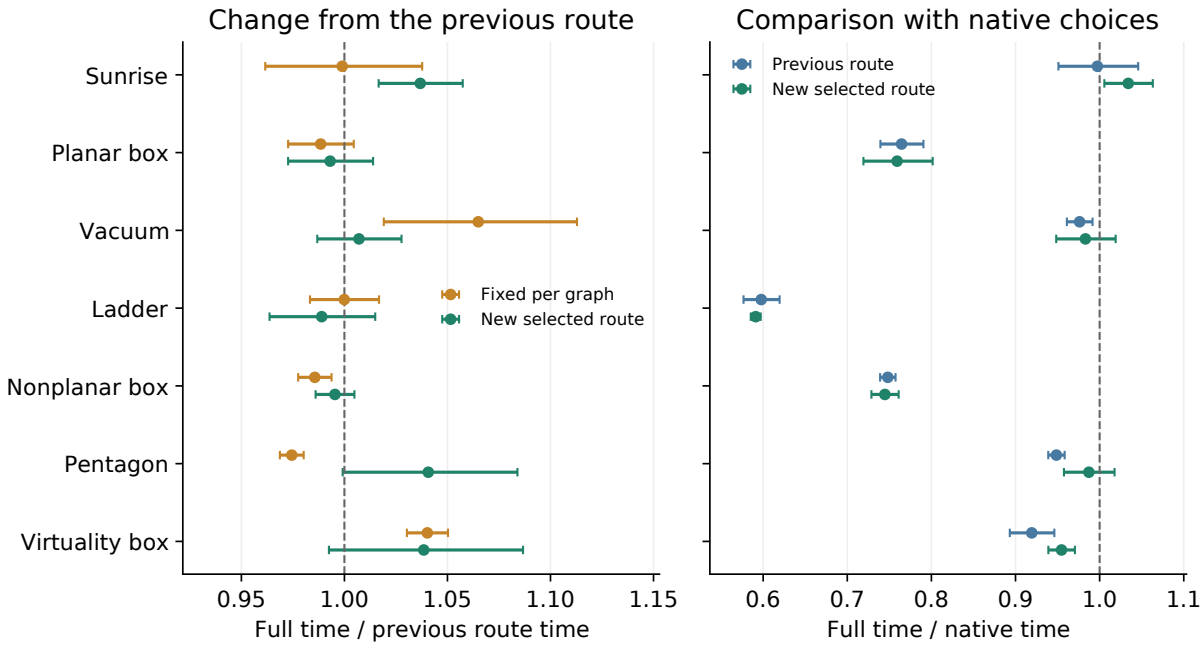


Figure 25: The effect of choosing Kira settings before reduction. Left: new fixed settings and selected settings relative to the preceding method. Right: preceding and selected methods relative to native choices. Values below one indicate less complete time. Bars give the range across two parameter profiles; they are not significance intervals.

The sunrise, vacuum, pentagon, and box with external virtuality take 3.7%, 0.7%, 4.1%, and 3.9% more time, respectively. No selected calculation requires fallback. Across 180 runs per method, symbolic equation regeneration changes from 79.88 to 80.58 seconds, while back-substitution changes from 37.99 to 37.35 seconds. These stage measurements show small changes in opposite directions rather than a consistent overall improvement.

The timers include prediction, preparation, native generation, exact checks, basis conversion, retries, and cleanup. As in the preceding comparisons, the caller and models are already loaded, and each attempt starts a fresh Kira process. The evidence therefore supports retaining the earlier ordering for this measured workload of known one- through three-loop graphs at new rational parameter values. The next comparison keeps that choice and changes the amount of parallel work instead.

H.5 Choosing how many Kira workers to use

The ordering study gives no further saving, so we retain the seed-expansion method and its earlier ordering. We next test the number of Kira workers, which determines how much of the symbolic calculation can run in parallel [4]. Development measurements show that extra workers add overhead on small batches, while some larger box batches finish sooner. This suggests choosing the worker count from the requested targets before equation generation.

The candidate counts are one, two, and four workers in the same integrated executable. The native control uses Kira's own seed and reduction decisions together with the common earlier implementation changes. For the learned comparison, the nine target features used above predict each worker count's complete time relative to one worker. We fit ten predictor types per graph on 36 training batches, select settings on 18 validation batches, and reserve 36 more batches for testing. Validation minimizes the sum of batch median times. A predictor must save at least two percent against the faster control and give lower summed time in at least two of three validation repetitions; otherwise the control remains active. Appendix U gives the labels, checks,

and comparison rules.

Validation enables a depth-two tree for the planar box and a depth-one tree for the nonplanar box. The other five graph types retain the preceding one-worker strategy. Both trees split only on the number of targets in the batch and choose at most two workers. Since the tested target patterns repeat across the parameter splits, these results establish a useful decision within those patterns; they do not establish target count as a sufficient description of arbitrary integral structures. Training and validation contain 1,098.1 seconds of reductions, and fitting all seventy candidates takes 0.82 seconds. These offline costs exclude reference construction and development.

All 900 timed reductions and 50 additional applications pass exact checks. The additional applications receive targets and a prescribed basis without reference coefficients. Seven run consecutively through a caller that keeps the models loaded, in a separate two-core allocation; each native attempt still starts a fresh Kira process. The timing experiment reserves four cores for every method so that the always-four-worker control can use its full setting. Thus every timed method has the same available resources, even when it uses fewer workers.

For the 36 reserved batches, the sum of median complete times falls from 82.21 seconds with the preceding one-worker method to 78.73 seconds with worker selection, a reduction of 4.2%. In the same experiment, native one-worker choices take 110.67 seconds, always using four workers takes 156.95 seconds, and a fixed count for each graph takes 82.45 seconds. Selection therefore gives 4.5% less time than the fixed-per-graph control. Table 30 shows where these differences arise; changes for the five graphs that retain one worker include execution variation and the additional decision cost.

Table 30: Time with and without worker selection. Entries sum the batch median times within each graph, in seconds. “Previous” uses the seed-expansion method with one worker; “Selected” uses the validation-selected count. The final column divides selected time by previous time.

Graph	Choice	Previous (s)	Selected (s)	Selected/previous
Sunrise	1 worker	5.271	5.215	0.989
Planar box	Tree depth 2	15.643	14.997	0.959
Vacuum	1 worker	4.813	4.620	0.960
Ladder	1 worker	23.197	23.602	1.017
Nonplanar box	Tree depth 1	26.768	23.819	0.890
Pentagon	1 worker	3.558	3.523	0.990
Virtuality box	1 worker	2.963	2.955	0.997

Giving each graph equal weight yields a smaller selected-to-preceding ratio of 0.9817, with descriptive interval $[0.9682, 0.9946]$, or 1.8% less time. The selected-to-native ratio is 0.8456 with interval $[0.7346, 0.9502]$, or 15.4% less time. These averages give equal importance to each graph, whereas the sum of medians measures the time for this particular collection of requests. Figure 26 also shows why simply increasing every calculation to four workers is ineffective on these small cases.

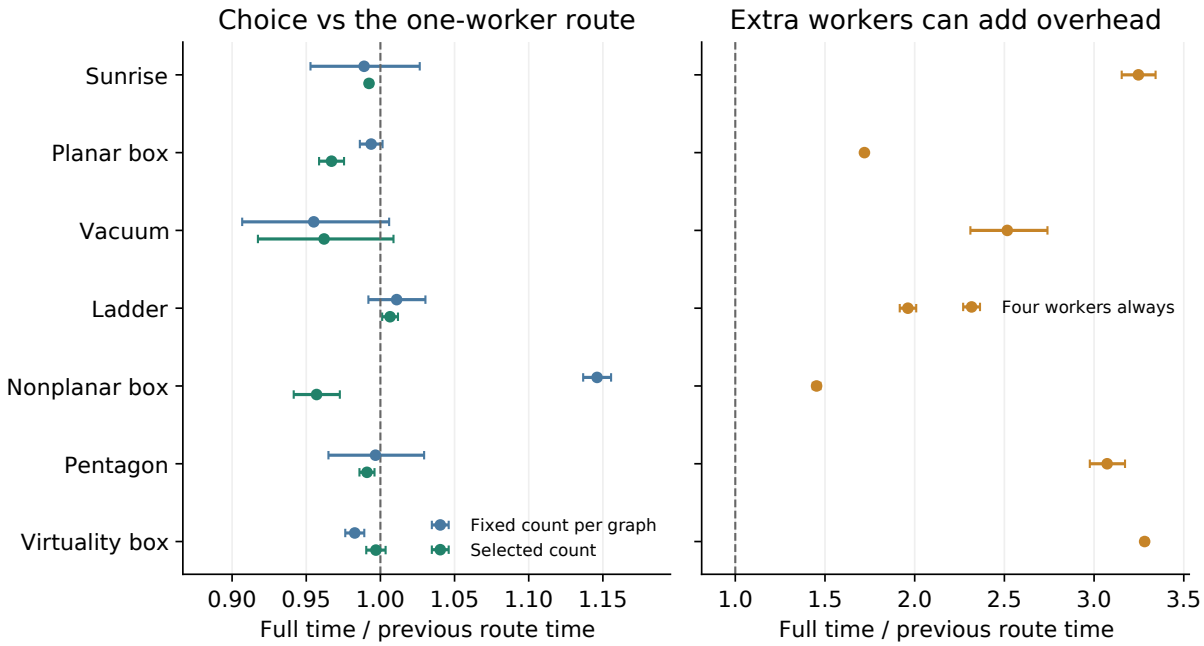


Figure 26: The effect of selecting the worker count. Left: selected and fixed worker counts compared with the preceding one-worker method. Right: always using four workers adds overhead on many small cases. Each panel has its own horizontal scale. Each point is the geometric mean of batch time ratios for one graph. Bars show the range across two parameter profiles and are not significance intervals.

Variation between repetitions matters for the size of the gain. Across all five repetitions, the selected method takes 404.84 seconds and the preceding method 414.08 seconds, giving 2.2% less elapsed time. Selection has lower summed time in four of the five repetitions, with selected-to-preceding ratios ranging from 0.9346 to 1.0555. This range describes the observed repetitions and is not a confidence interval. The ladder's sum of median times increases by 1.7%.

The saving in elapsed time also uses more CPU time. The extra prediction takes a median of 276.6 microseconds and is included in the complete-time measurements. Across 180 reductions per method, the preceding strategy uses 385.9 CPU-seconds, the always-four-worker strategy 1024.1, and the selected strategy 458.9. Selection therefore increases CPU use by 18.9%. These values measure CPU time consumed by the calculation, rather than the number of cores allocated to it.

The selected method has no failed attempts and requires no fallback. The always-four-worker method has ten unsuccessful native attempts, whose subsequent retries succeed; their costs remain in the reported time. Some native attempts report allocation failures under the imposed process limits. All comparisons include prediction, preparation, exact algebra, basis conversion, checks, retries, and cleanup, with the caller and models loaded before timing and a fresh Kira process for each attempt.

Worker selection is retained for this measured workload because it lowers elapsed time while preserving the exact results. Its demonstrated scope is seven known graph structures at one through three loops, known target patterns, and new rational kinematics with symbolic dimension. The preceding one-worker method remains available when the resource conditions or inputs differ. The result identifies a measured improvement among the tested choices, with the CPU cost stated alongside it.

I IBP relations and exact elimination

This appendix derives the identities and row operations used throughout the study. For a seed $\boldsymbol{\sigma}$, write its denominator product as $P_{\boldsymbol{\sigma}} = \prod_i D_i^{-\sigma_i}$ and define $\partial_k^\mu = \partial/\partial\ell_{k\mu}$. The product rule separates the derivative in Eq. (2) into a derivative of the momentum vector and a derivative of the propagator factors:

$$\partial_k^\mu(v_\mu P_{\boldsymbol{\sigma}}) = (\partial_k \cdot v)P_{\boldsymbol{\sigma}} + v \cdot \partial_k P_{\boldsymbol{\sigma}}, \tag{8}$$

$$v \cdot \partial_k P_{\boldsymbol{\sigma}} = -\sum_i \sigma_i (v \cdot \partial_k D_i) D_i^{-\sigma_i - 1} \prod_{j \neq i} D_j^{-\sigma_j}. \tag{9}$$

When $v = \ell_h$, the first derivative is $\partial_k \cdot v = d\delta_{kh}$, where δ_{kh} equals one for $k = h$ and zero otherwise; for an external momentum it vanishes. The integrated total derivative is zero in a region of convergence, and analytic continuation extends the relation to dimensional regularization [2, 1]. The same regularization prescription sets scaleless integrals to zero.

With a complete scalar-product basis, the scalar numerator $v \cdot \partial_k D_i$ can be written as $u_i + \sum_j v_{ij} D_j$. The coefficients u_i and v_{ij} depend on masses, invariants and the chosen momentum routing. Substituting in Eq. (9) gives

$$0 = \delta_v I_{\boldsymbol{\sigma}} - \sum_i \sigma_i u_i I_{\boldsymbol{\sigma}+\boldsymbol{e}_i} - \sum_{i,j} \sigma_i v_{ij} I_{\boldsymbol{\sigma}+\boldsymbol{e}_i-\boldsymbol{e}_j}, \tag{10}$$

Here $\delta_v = \partial_k \cdot v$, and the unit vector $\boldsymbol{e}_i$ increases index i by one. The derivative therefore relates the seed integral to integrals with shifted integer indices. Auxiliary denominators complete an otherwise incomplete scalar-product representation. When the denominator representation is dependent, additional algebraic relations are needed; the restricted vacuum family in Appendix L uses explicit mass partial fractions.

Suppose row q of such a system satisfies $\sum_j A_{qj} I_j = 0$ and $A_{qk} \neq 0$. Solving this row for the pivot integral gives

$$I_k = -\sum_{j \neq k} \frac{A_{qj}}{A_{qk}} I_j. \tag{11}$$

For a second identity row, or a target coefficient row r, subtract (A_{rk}/A_{qk}) times the zero identity. The resulting coefficients are

$$A'_{rj} = A_{rj} - \frac{A_{rk}}{A_{qk}} A_{qj}, \qquad A'_{rk} = 0. \tag{12}$$

Because the subtracted identity is zero, this operation preserves the target while removing its pivot entry. Choosing a different allowed source row can change the intermediate expressions and their cost without changing that equality. Repeated elimination removes the nonmaster columns and yields Eq. (3), provided each pivot coefficient is nonzero. Kinematic points at which a required denominator vanishes require a separate calculation.

J Reverse construction and the meaning of reference labels

The reverse construction provides targets with known answers because each checked source identity E_q is a linear combination of integrals that equals zero. Substituting these identities into Eq. (6) gives

$$T - \sum_m r_m M_m = \sum_q \lambda_q E_q = 0. \tag{13}$$

The equality fixes the master coefficients of the constructed expression. Expanding the source identities and collecting terms with equal index vectors then gives the target supplied to the

model. Recording the operations, source rows, and seed–operator labels provides a path back to the masters. The coefficients λ_q and r_m may be rational functions, including constants as a special case. Algebraic mass partial-fraction identities are recorded separately because they have no IBP seed label.

Different zero identities can give different construction histories for the same target. A saved seed set is therefore sufficient, but need not be unique or smallest. Likewise, the saved row sequence need not use the fewest operations. A seed label records membership in one checked set, and a row label records the next step in one checked path. The final master coefficients and later steps do not enter the model inputs.

K Model specification and training objectives

This appendix specifies what each model learns and how its training objective is obtained. We begin with the seed network, whose symbolic target coefficients must first be represented by numerical inputs. We split each expression into tokens and count individual tokens and consecutive pairs and triples. A fixed signed map places these counts in 64 components. Different token groups can share a component, so equal representations do not prove equal expressions. Integral indices, complexity counts and term positions give further inputs. A shared hyperbolic-tangent map converts each term into a vector $\boldsymbol{h}_u \in \mathbb{R}^{32}$. Family symbols, master indices and candidate indices give separate 32-component vectors.

K.1 Seed representation and loss

The target representation includes the following summary of ordered term pairs:

$$\boldsymbol{h}_{\mathrm{pair}} = \frac{1}{n_T(n_T-1)} \sum_{u \neq v} \psi_\theta([\boldsymbol{h}_u, \boldsymbol{h}_v, \boldsymbol{h}_u - \boldsymbol{h}_v, \boldsymbol{h}_u \odot \boldsymbol{h}_v]), \tag{14}$$

where n_T is the number of terms, square brackets concatenate vectors, $\odot$ multiplies components and ψ_θ returns 32 components. For $n_T = 1$, the summary is zero. This definition covers ordered pairs, not all larger term subsets. The candidate vector $\boldsymbol{u}_b$ weights target terms through scaled dot-product attention [15]:

$$w_{bu} = \frac{\exp(\boldsymbol{u}_b \cdot \boldsymbol{h}_u/\sqrt{32})}{\sum_v \exp(\boldsymbol{u}_b \cdot \boldsymbol{h}_v/\sqrt{32})}, \qquad \boldsymbol{v}_b = \sum_u w_{bu} \boldsymbol{h}_u. \tag{15}$$

The final input combines the mean and componentwise maximum of the term vectors with the pair, family, basis, seed, and attention-weighted term vectors. A hidden layer with 64 components produces g_θ in Eq. (7). Shared maps and aggregation follow Ref. [14], while stored positions retain the target term order. Reordering the candidate seeds changes only the order of the output scores.

If $y_b \in \{0, 1\}$ is recorded seed membership, a Bernoulli model assigns label likelihood $p_b^{y_b}(1 - p_b)^{1-y_b}$. Taking its negative logarithm gives $-y_b \log p_b - (1 - y_b) \log(1 - p_b)$. Averaging within each candidate list and then across targets gives the implemented loss

$$\mathcal{L}_s = -\frac{1}{|\mathcal{D}_s|} \sum_{T \in \mathcal{D}_s} \frac{1}{|\mathcal{C}(T)|} \sum_{b \in \mathcal{C}(T)} [y_b \log p_b + (1 - y_b) \log(1 - p_b)]. \tag{16}$$

Here $\mathcal{D}_s$ is the training-target set. This objective treats each target equally and does not claim that true seed dependencies are statistically independent. Training uses AdamW [16] with learning rate 10^{-3}, weight decay 10^{-4}, gradient norm limit one and up to 32 targets per update. The threshold 1/2 stays fixed during the expanded label test.

K.2 Row representation and loss

The row network compares the actions available at the current matrix state. Let $\mathcal{A}$ be that action list, and let $\boldsymbol{f}_a \in \mathbb{R}^{132}$ be the feature vector for action a. The encoder is $\boldsymbol{h}_a = \tanh(W\boldsymbol{f}_a + \boldsymbol{b})$ with 32 components. Concatenating $\boldsymbol{h}_a$ with the mean and componentwise maximum over $\mathcal{A}$ gives the input to a 64-component hidden layer. Its scalar output g_a defines

$$p(a \mid A, \mathcal{A}) = \frac{e^{g_a}}{\sum_{b\in\mathcal{A}} e^{g_b}}, \qquad \mathcal{L}_r = -\frac{1}{|\mathcal{D}_r|} \sum_{(A,\mathcal{A},a^*)\in\mathcal{D}_r} \log p(a^* \mid A, \mathcal{A}). \tag{17}$$

Here A is the matrix, a^* is the recorded action, and $\mathcal{D}_r$ contains the training states with a choice. The softmax denominator normalizes the probabilities to one, and taking the negative logarithm of the recorded action's probability gives the loss. A state with one action has zero loss. Training uses the seed model's AdamW settings and 256 states per update. Trial updates modulo a prime provide cancellation features. The numerical size of a modular value does not measure the size of a real coefficient.

Later row models learn the cost of an action. One fit predicts the logarithm of the measured insertion time. Another counts the remaining forward insertions after that action, with Kira choosing later steps. If insertion j uses n_j source entries, its estimated cost is $1 + 2n_j$: one division, followed by a multiplication and subtraction for each entry. Thus the cost label is $C(S, q) = \sum_j (1 + 2n_j)$ for state S and choice q. The later tree fits

$$y_q = \frac{C(S,q) - C(S,0)}{\max\{1, \max_b C(S,b) - \min_b C(S,b)\}}, \qquad \widehat{y}_q = h(\boldsymbol{x}_q - \boldsymbol{x}_0), \tag{18}$$

where 0 denotes Kira's native source and $\boldsymbol{x}_q$ contains eighteen current-row and overlap features. Weighted squared error gives each training family equal weight. Validation selects the tree and the required predicted improvement, and exact coefficients must agree before a measured continuation enters training. This count describes coefficient operations, while complete timings include the algebraic work inside each coefficient operation. Neither training objective proves a globally minimum path.

K.3 Equation features and tree fitting

The equation model uses a reference label derived from the support of an equation, meaning the set of integrals with nonzero coefficients. An equation label records whether this support appears in Kira's selected system. Table 31 groups the equation tree's seventeen inputs. They describe the seed and target before symbolic coefficient construction. For seed indices $\boldsymbol{\sigma}$ and target indices $\boldsymbol{a}$, the distance is $\sum_i |\sigma_i - a_i|$. The additional-power count is $\sum_i \max(\sigma_i - 1, 0)$, and the numerator rank is $\sum_i \max(-\sigma_i, 0)$. Neither quantity requires equation coefficients.

Table 31: Inputs to the original equation tree. The scattering seed trees omit the operator input and use sixteen features.

Group	Quantities	Number
Family and request	Loops, external momenta, index count, logarithm of target count	4
Seed	Positive indices, additional powers, numerator rank	3
Target limits	Excess seed powers and rank above target maxima	2
Sector	Whether requested; excess powers and rank above its target maxima	3
Target distance	Minimum index distance; whether the seed is a target	2
Operator	IBP operator index	1
Overall limits	Maximum target powers and numerator rank	2

Training gives each physical profile equal total weight by assigning each row weight $1/(n_b B_g)$, where n_b is its batch size and B_g is the number of training batches in its profile. At a node, let

w_+, w_- be the positive and negative total weights. Half the weighted binary Gini impurity is

$$H = \frac{w_+ w_-}{w_+ + w_-}. \tag{19}$$

Indeed, with $p = w_+/(w_+ + w_-)$, the weighted Gini expression is $(w_+ + w_-)[1 - p^2 - (1-p)^2] = 2H$. Each proposed split reduces H by the sum of its two children's values. The fit tests 25 feature quantiles between 0.02 and 0.98, rejects children below the stated population limit, and selects the largest positive reduction. This is the classification-tree construction of Ref. [9] with our bounded set of candidate splits.

A leaf with n_+ positive rows among n rows returns

$$p_{\text{leaf}} = \frac{n_+ + 1/2}{n + 1}. \tag{20}$$

Adding one half-count to each class gives a finite score even when a leaf contains only one class. The original tree keeps an equation when $p_{\text{leaf}} \geq 0.01$. These scores estimate reference-set membership; they are not calibrated probabilities that the whole reduction succeeds. The depth-eight and depth-twelve fits require at least 24 and eight rows in each child. The split procedure follows Ref. [9]. A basis check and fallback still test whether the selected system is sufficient.

K.4 Additional scattering seed and equation trees

The added scattering study fits a separate pair of trees for each of the three graph structures. The equation labels use the same selected-support rule as the original equation model. Let $y_{\sigma,o}$ be one when the native selected system contains the recorded equation support for seed σ and operator o. A seed receives label one when any of its operators has label one:

$$y_\sigma = \max_o y_{\sigma,o}. \tag{21}$$

For binary labels, this maximum is one exactly when at least one selected-support equation comes from the seed. This proves the stated membership rule. The label gives a useful reference set, not a proof that every labelled seed is necessary.

Each seed appears once per training batch. Its tree uses the sixteen features that do not depend on the operator. Thus every operator of that seed receives the same score. The early filter either permits or skips construction of all its equations. Kira still enumerates the seed region and applies its native symmetries. The equation tree uses all seventeen features and can keep different operators of the same seed.

Both fits have maximum depth twelve and at least eight records in each child. Each training batch has equal total weight. The split rule and leaf score are those derived above. Validation compares thresholds 0.001, 0.01 and 0.05 using full reduction time. Three repetitions contribute to each validation median. The chosen threshold minimizes the geometric mean ratio over the rational and symbolic validation batches of its graph. Model parameters and thresholds are fixed before testing. For the nonplanar box, both thresholds are 0.05. For the pentagon, the seed and equation thresholds are 0.05 and 0.001. For the box with virtuality, they are 0.001 and 0.01. These fits create new small tree models; they do not change the original neural checkpoints. The reserved timing test gives no overall saving, so the application keeps native choices as its default.

L Exact reuse for the restricted vacuum family

The large cumulative saving in Section 9 follows from exact recurrences in a particular Euclidean vacuum family. We derive them here to state both the source of the saving and the restrictions

on its use. The family has positive, distinct squared masses $\mu_j = \rho_j \mu$:

$$V_{\boldsymbol{b}}^{(L)} = \int \prod_{h=1}^{L} \frac{d^d \ell_h}{\pi^{d/2}} \frac{1}{\ell_1^2 \cdots \ell_L^2} \prod_{j=1}^{R} (Q^2 + \mu_j)^{-b_j}, \qquad Q = \sum_{h=1}^{L} \ell_h. \tag{22}$$

Here b_j is a nonnegative integer, $B = \sum_j b_j \geq 1$, R is the number of massive denominators, μ is a common squared-mass scale and ρ_j is a positive mass ratio. Each massless denominator has power one. All massive denominators share Q, so this family has special algebraic dependencies. Its prescribed masters are $M_j^{(L)} = V_{\boldsymbol{e}_j}^{(L)}$. The comparison uses $R = 2, 3, 4$ and $\mu = 1$ in the selected units.

L.1 Euler and mass partial-fraction identities

We first derive two relations that lower the massive propagator powers. Let f be the integrand without the measure. The simultaneous Euler derivative gives $\sum_h \ell_h \cdot \partial_{\ell_h} (\ell_i^2)^{-1} = -2(\ell_i^2)^{-1}$ for each massless factor. Also, $\sum_h \ell_h \cdot \partial_{\ell_h} Q^2 = 2Q^2$. Therefore

$$\sum_h \partial_{\ell_h} \cdot (\ell_h f) = \left[Ld - 2L - 2 \sum_j b_j \frac{Q^2}{Q^2 + \mu_j} \right] f. \tag{23}$$

Use $Q^2 = (Q^2 + \mu_j) - \mu_j$ and integrate the derivative as in Appendix I. This gives

$$0 = [L(d-2) - 2B] V_{\boldsymbol{b}}^{(L)} + 2 \sum_j b_j \mu_j V_{\boldsymbol{b}+\boldsymbol{e}_j}^{(L)}. \tag{24}$$

For two present massive denominators, the integrand identity $(Q^2 + \mu_i) - (Q^2 + \mu_j) = \mu_i - \mu_j$ separately gives

$$(\mu_i - \mu_j) V_{\boldsymbol{b}}^{(L)} = V_{\boldsymbol{b}-\boldsymbol{e}_i}^{(L)} - V_{\boldsymbol{b}-\boldsymbol{e}_j}^{(L)}. \tag{25}$$

This second relation is an algebraic partial fraction, so it introduces no additional IBP seed.

L.2 A recurrence shared across loop orders

To make the common loop dependence explicit, define $z = L(d-2)$ and $W_{\boldsymbol{b}}^{(L)} = \mu^{B-1} V_{\boldsymbol{b}}^{(L)}$. Multiplication by this power of μ gives W the same mass dimension as the masters. Substituting in Eqs. (24) and (25) removes the common scale. If $\boldsymbol{C}_{\boldsymbol{b}}(z)$ is the vector of master coefficients of $W_{\boldsymbol{b}}^{(L)}$, then

$$\boldsymbol{C}_{\boldsymbol{b}}(z) = \frac{\boldsymbol{C}_{\boldsymbol{b}-\boldsymbol{e}_i}(z) - \boldsymbol{C}_{\boldsymbol{b}-\boldsymbol{e}_j}(z)}{\rho_i - \rho_j}, \qquad b_i, b_j > 0, \tag{26}$$

$$\boldsymbol{C}_{k\boldsymbol{e}_j}(z) = \frac{2(k-1) - z}{2(k-1)\rho_j} \boldsymbol{C}_{(k-1)\boldsymbol{e}_j}(z), \qquad k \geq 2. \tag{27}$$

The initial vector is $\boldsymbol{C}_{\boldsymbol{e}_j} = \boldsymbol{e}_j$. Every recurrence lowers B, so it terminates at the prescribed masters. Its coefficients depend on L only through z. Induction on B shows that they are polynomials in z with rational coefficients at fixed rational ρ_j. One calculation can therefore supply all requested loop orders by substituting $z = L(d-2)$.

L.3 An independent coefficient check

The recurrence result is checked by obtaining the mixed-denominator coefficients through a Taylor partial-fraction expansion. For a dimensionless auxiliary variable x, write

$$\prod_i (x + \rho_i)^{-b_i} = \sum_{j : b_j > 0} \sum_{k=1}^{b_j} \frac{a_{jk}}{(x + \rho_j)^k}. \tag{28}$$

At the pole $x = -\rho_j$, set $t = x + \rho_j$. Multiplying by t^{b_j} and expanding the regular factors shows that the principal-part coefficient is

$$a_{jk} = [t^{b_j-k}] \prod_{i\neq j} (\rho_i - \rho_j + t)^{-b_i}. \tag{29}$$

Here $[t^n]$ extracts Taylor coefficient n and factors with $b_i = 0$ are one. Applying Eq. (27) repeatedly to each pole term gives

$$C_{\boldsymbol{b}j}(z) = \sum_{k=1}^{b_j} a_{jk} \prod_{h=1}^{k-1} \frac{2h - z}{2h\rho_j}, \qquad V_{\boldsymbol{b}}^{(L)} = \mu^{1-B} \sum_j C_{\boldsymbol{b}j}(L(d-2)) M_j^{(L)}. \tag{30}$$

An empty product is defined as one. This Taylor calculation checks every returned polynomial coefficient before the substitution for z and does not follow the chosen sequence of pair recurrences. The derivation requires distinct ρ_j, unit powers for the massless propagators, and the shared momentum Q. These assumptions delimit the family to which the result applies.

M Measurement definitions and comparison conditions

The comparisons use separate measures for label prediction, algebraic completion, and elapsed time. For the label measures, let TP be the number of selected reference seeds, FP the number of selected seeds absent from the reference set, and FN the number of omitted reference seeds. Precision is $P = \mathrm{TP}/(\mathrm{TP} + \mathrm{FP})$ and recall is $R = \mathrm{TP}/(\mathrm{TP} + \mathrm{FN})$. Defining F_1 as their harmonic mean gives

$$F_1 = \frac{2}{P^{-1} + R^{-1}} = \frac{2PR}{P + R} = \frac{2\mathrm{TP}}{2\mathrm{TP} + \mathrm{FP} + \mathrm{FN}}. \tag{31}$$

Complete-set agreement counts targets for which the entire predicted seed set equals the recorded set. Algebraic sufficiency instead counts targets that reach the prescribed master basis using the selected equations. Since a target can have several sufficient seed sets, these measures need not agree with each other or with the individual-label scores in Figure 9.

For timing, let T_{abr} be the measured time for method a, batch b and repetition r. We use $\widetilde{T}_{ab} = \mathrm{median}_{r=1,\ldots,5} T_{abr}$. For the later native comparisons, if physical group g contains batches B_g, define the equal-group mean logarithmic ratio

$$\overline{\ell} = \frac{1}{G} \sum_{g=1}^{G} \frac{1}{|B_g|} \sum_{b\in B_g} \log \frac{\widetilde{T}_{ab}}{\widetilde{T}_{cb}}, \qquad R_{a/c} = e^{\overline{\ell}}. \tag{32}$$

Here c identifies the control and G is the number of physical groups. Exponentiating the mean log ratio gives the geometric mean; ratios below one indicate lower time. Reversing numerator and denominator changes $\overline{\ell}$ to $-\overline{\ell}$, so speedup is $S = 1/R_{a/c}$ and the time reduction is $100(1 - R_{a/c})\%$. The cumulative study uses one batch per group. The supplied-equation seed comparison instead gives each complete paired batch equal weight; its interval resamples six related kinematic groups. Following bootstrap resampling [20], reported descriptive 95% intervals resample complete groups 10,000 times while retaining paired methods and repetitions. They describe case-to-case variation in these experiments. We do not infer new uncertainty intervals for figures whose source summaries provide only point estimates.

The supplied-equation seed test includes target preparation, inference, early basis checking, reduction, result checking and fallback. It excludes original IBP-pool generation. It rotates methods within each family and target batch on the same node, and includes only complete paired batches in its ratio summary while preserving incomplete and skipped trials in the accounting.

The native equation and transfer tests include input preparation, model loading and evaluation, Kira startup, native IBP generation, elimination, basis and coefficient checks, cleanup and every

fallback. Independent reference discovery and the later independent check lie outside the timer. In the equation test, the native and learned arms receive the same executable-level improvements. The eight reserved profiles test new masses and invariants within known structures; the seven transfer cases additionally test new structures and symbolic invariants.

The cumulative test includes fresh-worker startup, input loading, supplied Euler and mass-identity construction, algebra, verification, output saving and complete output reading. Eight groups use maximum massive powers 8, 10, 12 or 14, with two groups at each bound. They request 16, 32, 48 or 64 targets per profile, capped by availability. Each group contains three mass profiles and all loop orders 1–12. Paired methods use the same resources, and their order changes between repetition blocks. Each method recomputes the reduction. Waiting for resources, offline training and the later independent check remain outside all reported production times.

The initial live-matrix comparison uses three repetitions of four methods on seven physical cases, giving 84 workflows. Its complete ratio is the geometric mean of the seven paired median ratios and carries no reported uncertainty interval. The later native implementation comparison uses five repetitions and sixteen batches; it repeats earlier physical profiles to measure the implementation change.

The reported numbers are measurements from this study. We keep the original label tests, seed tests, row tests, equation tests, scattering tests and vacuum benchmark separate. This prevents differences in targets or timed work from being mistaken for a combined model gain. The archived notes preserve earlier experiments, including negative results.

N Integral families and kinematic conventions

The physical scope of each test is determined by its denominator momenta, masses, and external scalar products. This appendix gives those definitions for the training and test families. Together with an index vector, or a rational linear combination of index vectors in one family, they specify the targets without requiring an individual listing of the full collection. Unless stated otherwise, $D_i = q_i^2 - m_i^2 + i0$ and the measure is that of Eq. (1). The Euclidean vacuum family keeps the sign convention in Eq. (22).

N.1 Initial one-loop and two-loop families

For the one-loop tadpole, $q_1 = \ell$ and $m_1^2 = \mu > 0$. For the bubbles,

$$\boldsymbol{q} = (\ell, \ell - p), \qquad p^2 = s, \qquad (m_1^2, m_2^2) \in \{(0,0), (\mu,0), (\mu,\mu), (\mu_1,\mu_2)\}. \tag{33}$$

The symbols μ, μ_1, μ_2 denote squared masses, with $\mu_1 \neq \mu_2$ in the unequal-mass case. The triangle routing is

$$\boldsymbol{q} = (\ell, \ell - p_1, \ell - p_1 - p_2), \qquad (p_1^2, p_2^2, (p_1+p_2)^2) = (u_1, u_2, u_3). \tag{34}$$

Expanding $(p_1+p_2)^2$ gives $2p_1 \cdot p_2 = u_3 - u_1 - u_2$. Three massless triangle classes use virtualities $(0,0,s)$, $(0,u_2,u_3)$ and (u_1,u_2,u_3). Two equal-mass classes, with all internal squared masses μ, use $(0,0,s)$ and (u_1,u_2,u_3). Each displayed nonzero virtuality is independent at generic kinematics.

The initial massless boxes use four incoming external momenta, $p_4 = -p_1 - p_2 - p_3$, and

$$\boldsymbol{q} = (\ell, \ell - p_1, \ell - p_1 - p_2, \ell - p_1 - p_2 - p_3), \quad s = (p_1+p_2)^2, \quad t = (p_2+p_3)^2. \tag{35}$$

Writing $u_i = p_i^2$, the three classes have $(u_1,u_2,u_3,u_4) = (0,0,0,0)$, $(0,0,0,u_4)$ and (u_1,u_2,u_3,u_4). Expanding the definitions of s, t and p_4^2 gives

$$2p_1 \cdot p_2 = s - u_1 - u_2, \quad 2p_2 \cdot p_3 = t - u_2 - u_3, \quad 2p_1 \cdot p_3 = u_4 + u_2 - s - t. \tag{36}$$

These definitions give thirteen one-loop classes. The initial two-loop massless two-point family uses

$$\boldsymbol{q} = (\ell_1, \ell_2, \ell_1 - \ell_2, \ell_1 - p, \ell_2 - p), \qquad p^2 = s, \qquad m_i^2 = 0. \tag{37}$$

The equal-mass sunrise uses $\boldsymbol{q} = (\ell_1, \ell_2, p - \ell_1 - \ell_2, p - \ell_1, p - \ell_2)$, with masses $(\mu, \mu, \mu, 0, 0)$ and $p^2 = s$. Its last two entries are auxiliary: $a_4, a_5 \leq 0$. The equal-mass vacuum sunset has $\boldsymbol{q} = (\ell_1, \ell_2, \ell_1 - \ell_2)$ and all three squared masses μ. These three classes complete the initial sixteen. Their subsectors include the integrals obtained by removing physical denominators.

N.2 Additional one-loop mass and invariant assignments

The 984 added box families use a more general assignment than the three initial massless boxes:

$$D_i = (\ell - Q_i)^2 - \mu_{\alpha_i} + i0, \quad Q_1 = 0, \quad \mu_0 = 0, \quad \alpha_i \in \{0, 1, 2, 3\}. \tag{38}$$

The nonzero squared masses μ_1, μ_2, μ_3 are independent. Repeated labels impose equal masses. The three external shifts Q_2, Q_3, Q_4 have squared differences

$$\Delta_{ij} = (Q_i - Q_j)^2 = y_{\beta_{ij}}, \quad y_0 = 0, \quad \beta_{ij} \in \{0, 1, 2, 3\}. \tag{39}$$

The nonzero y_1, y_2, y_3 are independent invariants. Expanding a squared difference gives the Gram matrix

$$Q_i \cdot Q_j = \frac{\Delta_{1i} + \Delta_{1j} - \Delta_{ij}}{2}, \qquad i, j \in \{2, 3, 4\}. \tag{40}$$

The construction rejects singular sampled Gram matrices and identifies assignments related by propagator permutations or by relabelling nonzero masses and invariants. A fixed order selects 984 of 1,673 admissible inequivalent profiles after excluding the original massless boxes. This extends the initial collection to 1,000 families with 9,678 family-local masters and 240,472 targets before the higher-loop additions. Each profile gives a symbolic family through its zeros and equality conditions. The single-target seed timing comparison uses twelve reserved instances of these boxes. Its targets follow the reverse construction in Appendix J, rather than the native target enumeration below.

N.3 Native multiloop comparisons

The sunrise comparisons allow independent masses on the same three physical propagators. The other native comparison families use

$$\begin{aligned} \boldsymbol{q}_{\mathrm{DB}} &= (\ell_1, \ell_1 - p_1, \ell_1 - p_1 - p_2, \ell_2, \ell_2 - p_1 - p_2, \ell_2 - p_1 - p_2 - p_3, \\ &\qquad \ell_1 - \ell_2, \ell_1 - p_3, \ell_2 - p_1), \end{aligned} \tag{41}$$

$$\boldsymbol{q}_{\mathrm{V}} = (\ell_1, \ell_2, \ell_3, \ell_1 - \ell_2, \ell_2 - \ell_3, \ell_1 - \ell_3), \tag{42}$$

$$\boldsymbol{q}_{\mathrm{L}} = (\ell_1, \ell_1 - p, \ell_2, \ell_2 - p, \ell_3, \ell_3 - p, \ell_1 - \ell_2, \ell_2 - \ell_3, \ell_1 - \ell_3). \tag{43}$$

The double box has two loops and seven physical denominators; its last two entries are auxiliary. The vacuum has three loops and six physical denominators. The ladder has three loops and eight physical denominators; its last entry is auxiliary. Auxiliary indices remain nonpositive. The massless double box has $p_i^2 = 0$ and uses $2p_1 \cdot p_2 = s$, $2p_1 \cdot p_3 = t$, $2p_2 \cdot p_3 = -s - t$. This local definition of t differs from the initial box convention because the routing differs.

For the final early-equation test, the sunrise squared-mass vectors are $(2, 7, 11)$ and $(3, 7, 13)$, with $p^2 = -101, -103$. The double-box profiles are $(s, t) = (-101, -103), (-107, -109)$. The vacuum squared-mass vectors are $(1, 1, 1, 1, 1, 11)$ and $(1, 1, 1, 13, 13, 13)$. The ladder has squared masses $(1, 0, 0, 0, 0, 0, 0, 0)$ and $p^2 = -101, -103$. All values use a common squared-momentum unit. These choices keep d symbolic and give coefficients in $\mathbb{Q}(d)$.

The target order starts with the full-sector corner, then adds each single extra propagator power and each available single auxiliary numerator. The construction repeats after removing one, two and three physical denominators, retains at least two, and removes duplicates. The comparison takes 21 sunrise targets and batches of 32 and 128 targets for each other profile. This gives the fourteen batches in Section B.3.

N.4 New scattering structures and symbolic invariants

The massless nonplanar box uses the routing of Ref. [4]:

$$\begin{aligned}\boldsymbol{q}_{\mathrm{NP}} = (&\ell_1, \ell_2, p_1 - \ell_1, p_2 - \ell_2, p_1 + p_2 - p_3 - \ell_1,\\ &p_1 - \ell_1 + \ell_2, p_1 - p_3 - \ell_1 + \ell_2, \ell_1 - p_3, \ell_2 - p_1 - p_2).\end{aligned} \tag{44}$$

The first seven entries are physical propagators, and the final two are auxiliary entries. Here $p_1^2 = p_2^2 = p_3^2 = (p_1 + p_2 - p_3)^2 = 0$, $s = (p_1 + p_2)^2$ and $t = (p_1 - p_3)^2$. Expanding these squares gives $2p_1 \cdot p_2 = s$, $2p_1 \cdot p_3 = -t$ and $2p_2 \cdot p_3 = s + t$.

The massless pentagon uses $\sum_{i=1}^{5} p_i = 0$, $p_i^2 = 0$ and

$$\boldsymbol{q}_{\mathrm{P}} = (\ell, \ell - p_1, \ell - p_1 - p_2, \ell - p_1 - p_2 - p_3, \ell - p_1 - p_2 - p_3 - p_4). \tag{45}$$

All five denominators are physical. The independent invariants are $(s_{12}, s_{23}, s_{34}, s_{45}, s_{51})$, where $s_{ij} = (p_i + p_j)^2$. Squaring $p_1 + p_2 + p_3 = -(p_4 + p_5)$ gives $2p_1 \cdot p_3 = s_{45} - s_{12} - s_{23}$; the other nonadjacent products follow by cyclic relabelling. The box with an external virtuality uses $\boldsymbol{q} = (\ell, \ell - p_1, \ell - p_1 - p_2, \ell - p_3)$, $p_1^2 = p_2^2 = p_3^2 = 0$, and $v = (p_1 + p_2 - p_3)^2$. With the nonplanar convention for s, t, expanding v gives $2p_2 \cdot p_3 = s + t - v$.

The rational nonplanar profiles are $(s, t) = (-149, -147), (-167, -157)$. The pentagon profiles are $(-51, -53, -57, -59, -63)$ and $(-71, -73, -77, -79, -83)$ in the displayed invariant order. Each rational profile supplies sixteen targets. The symbolic nonplanar, pentagon and virtuality-box tests supply ten, six and eight targets. They start with the corner and single additional powers, followed by auxiliary numerators and single removed denominators where available. These definitions give the 88 targets of the transfer test.

N.5 Parameter points and targets for the additional training

The new scattering study uses the same three denominator lists and scalar-product conventions defined above. For integer profile number n, we choose

$$(s, t)_{\mathrm{NP}} = (-113 - 18n, -127 - 10n), \tag{46}$$

$$(s_{12}, s_{23}, s_{34}, s_{45}, s_{51})_{\mathrm{P}} = (-11 - 20n, -13 - 20n, -17 - 20n, -19 - 20n, -23 - 20n), \tag{47}$$

$$(s, t, v)_{\mathrm{box}} = (-113 - 18n, -127 - 10n, -149 - 12n). \tag{48}$$

These are specified input values in a common squared-momentum unit. Training uses $n = 10, 11$, validation uses $n = 12$, and testing uses $n = 13, 14$. The dimension d remains symbolic in every case. Each symbolic batch instead keeps all of its displayed invariants independent.

The target list starts with the full-sector corner. It then raises each physical denominator power once and adds each single auxiliary numerator. The same sequence follows for each possible single removed physical denominator. Rational batches take the first sixteen entries. Symbolic training takes entries 0–3, validation takes 4–7, and testing takes 8–11, with indices starting at zero. We verify that the three symbolic lists are disjoint for each structure. Rational batches reuse index templates at different parameter points, so this split does not claim entirely unseen target patterns in every representation.

O Master bases and dimensionally consistent target construction

This appendix explains the checks used to identify the stored master bases and to combine target terms with consistent mass dimensions. We first count the scalar products that an independent denominator representation must cover:

$$N_{\mathrm{sp}} = \frac{L(L+1)}{2} + LE. \tag{49}$$

There are L loop squares and $L(L-1)/2$ distinct loop cross-products, followed by LE products with the E independent external momenta. This count supplies a direct rank check for the independent denominator lists. The restricted vacuum parent in Eq. (22) instead uses the explicitly dependent massive denominators and their partial fractions.

For index vector $\boldsymbol{a}$, define the positive-index sum $r = \sum_i \max(a_i, 0)$, numerator rank $s_- = \sum_i \max(-a_i, 0)$ and additional-power count $b = \sum_i \max(a_i - 1, 0)$. If the top sector has t_{top} physical denominators, the initial Kira basis calculation uses seed limits $(r, s_-, b) \le (t_{\mathrm{top}}+2, 2, 2)$ and probe limits $(t_{\mathrm{top}}+1, 1, 1)$, component by component. Both include all subsectors. The basis check then increases every limit by one and repeats at a second finite-field point. The original sixteen definitions give the same master labels in all three calculations, for 48 completed basis checks. Such agreement supports stability in the tested domains; it does not prove completeness for all integer indices [4, 5].

The initial sixteen classes contain 49 master entries when each family is counted separately. The tadpole contributes one. The four bubble classes give one, two, two and three, in their stated order. The five triangle classes give one, two, four, three and five. The three box classes give three, four and eleven. The massless two-loop two-point, sunrise and vacuum sunset classes give two, three and two. For example, the equal-mass sunrise basis is

$$M_1 = I_{(1,1,0,0,0)}, \qquad M_2 = I_{(1,1,1,0,0)}, \qquad M_3 = I_{(1,1,1,-2,0)}. \tag{50}$$

The first integral separates into two tadpoles, and the last contains the numerator $[(p-\ell_1)^2]^2$. These integral labels define the basis even when its numerical values have not been evaluated. Reduction training uses the relations to that basis, so it does not require those values. Each later family instance likewise retains its explicit master indices and parameter assignment.

The mass dimension of Eq. (1) is $Ld - 2\sum_i a_i$: each loop measure contributes d powers of mass and each inverse propagator contributes two. In the restricted vacuum data, $[V^{(L)}_{\boldsymbol{b}}] = Ld - 2L - 2B$. Thus targets of different total powers can be combined consistently as

$$T = \sum_{\nu} \kappa_\nu d^{\eta_\nu} \mu^{B_\nu - B_*} V^{(L)}_{\boldsymbol{b}_\nu}, \qquad B_* = \max_\nu B_\nu. \tag{51}$$

Every term has dimension $Ld - 2L - 2B_*$. The construction uses $\eta_\nu \in \{0, 1\}$ and $\kappa_\nu = n_\nu/q_\nu$, with $n_\nu \in \{-3, -2, -1, 1, 2, 3\}$ and $q_\nu \in \{1, 2, 3\}$. Each expression contains one, two or three distinct roots. Individual roots have $2 \le B \le 5$; Euler seeds have $1 \le B \le 5$, and mass identities extend through $B = 6$. Each added family contains 128 distinct targets, including individual roots. The identity pools contain 35, 160 or 545 rows for two, three or four massive denominators.

For integer profile number n, the generated mass ratios are $R = 2 + ((n-1) \bmod 3)$, $\rho_1 = 1$ and $\rho_{j+2} = 2 + 3n + j(n+2)$ for $0 \le j \le R-2$. Profiles $n = 1, \ldots, 100$ enter at loop orders 2–12; profiles 101–1350 enter at orders 1–12. Distinct masses make the partial-fraction denominators nonzero. A profile remains in the same training or testing group at every loop order. These rules state the extent of the high-loop collection and its restricted algebraic structure.

O.1 Analytic check of the restricted masters

The recurrences in Appendix L reduce the specified family to $M^{(L)}_j$. We can also derive their values to check this choice. Write $\delta = d/2$, $\alpha = L(\delta - 1)$ and $c = L - (L-1)\delta$. The symbol

Γ denotes the Euler gamma function, and a, b below are denominator powers. The standard Euclidean two-denominator convolution follows from Feynman parameters [1]:

$$\begin{aligned} J(a,b;Q) &= \int \frac{d^dk}{\pi^\delta} \frac{1}{(k^2)^a((Q-k)^2)^b} \\ &= (Q^2)^{\delta-a-b} \frac{\Gamma(a+b-\delta)\Gamma(\delta-a)\Gamma(\delta-b)}{\Gamma(a)\Gamma(b)\Gamma(2\delta-a-b)}. \end{aligned} \tag{52}$$

To obtain this expression, combine the denominators with

$$\frac{1}{A^aB^b} = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \int_0^1 \frac{x^{a-1}(1-x)^{b-1}\,dx}{[xA+(1-x)B]^{a+b}}. \tag{53}$$

Shifting k completes the square and gives the radial denominator $k^2 + x(1-x)Q^2$. The radial integral equals $\Gamma(a+b-\delta)[x(1-x)Q^2]^{\delta-a-b}/\Gamma(a+b)$. The remaining beta integral is $\Gamma(\delta - b)\Gamma(\delta-a)/\Gamma(2\delta-a-b)$, which yields Eq. (52).

At fixed Q, successive convolution of the L unit-power massless factors gives

$$C_L(Q^2)^{-c}, \qquad C_L = \frac{\Gamma(c)\Gamma(\delta-1)^L}{\Gamma(\alpha)}. \tag{54}$$

For $L = 1$ the coefficient is one. Applying Eq. (52) with $a = c$, $b = 1$ advances c to $c + 1 - \delta$ and proves the formula by induction. The final integral over Q then gives

$$\begin{aligned} M_j^{(L)} &= \frac{C_L}{\Gamma(\delta)} \int_0^\infty \frac{x^{\alpha-1}\,dx}{x+\mu_j} \\ &= \mu_j^{\alpha-1} \frac{\Gamma(c)\Gamma(\delta-1)^L\Gamma(1-\alpha)}{\Gamma(\delta)}. \end{aligned} \tag{55}$$

The change $x = \mu_j y$ reduces the integral to $\Gamma(\alpha)\Gamma(1-\alpha)$ for $0 < \operatorname{Re}\alpha < 1$. Analytic continuation gives the dimensionally regulated result [1]. Distinct positive fixed mass ratios give distinct exponentials $\rho_j^{L(d/2-1)-1}$ as functions of d. After common factors cancel, a rational-function relation would have a unique largest exponential as real d increases. Rational coefficients grow at most as powers and cannot cancel that term. Removing its zero coefficient and repeating establishes independence over $\mathbb{Q}(d)$ at these fixed mass ratios. Special dimensions and coincident masses require their own analysis.

P Finite minimum-operation checks

The saved construction supplies a valid reduction path, but a minimum-cost claim requires a defined set of alternatives. We therefore specify the allowed equations, row operations, and cost before checking all paths for two small systems. These bounded examples make precise the distinction between a known path and a certified minimum. Define the massless bubble $B_{a,b} = I_{(a,b)}$ with denominators $\ell^2 + i0$ and $(\ell - p)^2 + i0$, where $p^2 = s \neq 0$. Contracting the derivative with ℓ uses $2\ell \cdot (\ell - p) = \ell^2 + (\ell - p)^2 - s$. Equation (9) then gives

$$0 = (d - 2a - b)B_{a,b} - bB_{a-1,b+1} + bsB_{a,b+1}. \tag{56}$$

Changing variables $\ell \mapsto p - \ell$ exchanges a, b and gives the second operator identity. Integrals with only a massless denominator vanish in dimensional regularization [1]. The two operators at $(1,1)$, the first at $(2,1)$ and the second at $(1,2)$ therefore give

$$\begin{pmatrix} 0 & s & 0 & d-3 \\ s & 0 & 0 & d-3 \\ d-5 & -1 & s & 0 \\ -1 & d-5 & s & 0 \end{pmatrix} \begin{pmatrix} B_{2,1} \\ B_{1,2} \\ B_{2,2} \\ B_{1,1} \end{pmatrix} = 0. \tag{57}$$

The first two rows yield $B_{2,1} = B_{1,2} = (3-d)B_{1,1}/s$. Substituting in the third yields $B_{2,2} = (d-6)(d-3)B_{1,1}/s^2$.

For the massive tadpole $T_a = \int d^d\ell\,(\ell^2 - m^2 + i0)^{-a}/(i\pi^{d/2})$, the same derivative gives

$$0 = dT_a - 2a\int \frac{d^d\ell}{i\pi^{d/2}} \frac{\ell^2}{(\ell^2 - m^2 + i0)^{a+1}} = (d-2a)T_a - 2am^2 T_{a+1}. \tag{58}$$

Seeds $a = 1, 2, 3$ reduce T_2, T_3, T_4 to T_1. We keep the master column protected in both systems. An allowed step selects a nonzero pivot in an unprocessed nonmaster column, normalizes its row and eliminates that column from the other rows. The cost counts one division per nonzero nonpivot source entry when normalization is needed, plus one multiplication and one subtraction for each nonpivot source-entry update. It excludes assignments and the internal polynomial operations of a rational-function operation.

Let S be the exact matrix state, $\mathcal{A}(S)$ its allowed actions, $c(S,a)$ the stated step cost and S_a the resulting state. Splitting every complete path into its first action and its remaining path gives the finite recurrence [21]

$$V(S) = \min_{a\in\mathcal{A}(S)} \{c(S,a) + V(S_a)\}, \qquad V(S_{\text{done}}) = 0. \tag{59}$$

Each action removes an unprocessed column. Induction on the number of remaining columns proves the minimum after all actions have been evaluated. Repeating this calculation for every source-row subset of nonmaster rank three also minimizes over initial equation selection.

Our bubble check covers 113 states, 234 transitions and 152 complete paths across five sufficient subsets. Its minimum is seven coefficient operations with subset selection and thirteen with the full pool. The tadpole check covers 24 states, 32 transitions and fifteen paths; its minimum is seven operations with its single sufficient subset. Each pool requires at least three equations. The smallest number of distinct seed labels is two for the bubble and three for the tadpole. Exact replay and six native Kira master-coefficient comparisons agree. These are minima for the displayed identities, protected basis, allowed operations and cost. They do not establish minimum elapsed time or an optimum over all IBP identities.

Q Learning the cost of a complete reduction

This appendix defines the complete-time prediction problem, derives the fitted models, and specifies the physical inputs used in Section H.1. The cost model receives a feature vector $\boldsymbol{f}_c$ for target batch c. Its entries are the number of loops, external momenta, inverse propagators, physical propagators and symbolic invariants; $\log(1+n_c)$, where n_c is the number of targets; the maximum dot count, maximum numerator rank, mean dot count and mean numerator rank; the number of target sectors; and seven indicators for the graph structures. An indicator is one for its graph and zero for the other graphs. Together these quantities give eighteen inputs, all available from the requested targets and family before reduction.

For an index vector $\boldsymbol{a}$, define

$$r(\boldsymbol{a}) = \sum_i \max(a_i, 0), \qquad s(\boldsymbol{a}) = \sum_i \max(-a_i, 0), \qquad q(\boldsymbol{a}) = \sum_i \max(a_i - 1, 0). \tag{60}$$

Here r is the positive index sum, s is the numerator rank, and q is the number of extra denominator powers, called dots. These definitions follow directly by counting the denominator and numerator factors. The initial bounds $B_c = (r_c, s_c, q_c)$ are the componentwise maxima over the targets. The trial seed regions use B_c, $B_c - (1,1,1)$, $B_c - (1,0,1)$, or $B_c + (1,1,1)$. Negative bounds are set to zero. The choices change the region in which equations are generated. Since a smaller region can omit required relations, every attempt must still pass the prescribed-basis check.

Let t_{cj} be the median complete time of strategy j on training batch c, and let t_{c0} be the time of the integrated native strategy. The training response is

$$y_{cj} = \log \frac{t_{cj}}{t_{c0}}. \tag{61}$$

Each time includes unsuccessful attempts and the eventual successful fallback, so a strategy that omits necessary equations is penalized by the extra calculation. The logarithm preserves the ordering of positive time ratios; minimizing the response therefore also minimizes the complete time relative to the common baseline.

We fit constant, tree, linear and nearest-neighbour predictors of this response. The constant predictor averages each strategy's training responses. A tree splits the feature space and uses a constant vector within each leaf, following regression-tree construction [9]. For a leaf containing training set A, minimizing $\sum_{c\in A}(y_{cj} - v_j)^2$ gives

$$\frac{\partial}{\partial v_j}\sum_{c\in A}(y_{cj} - v_j)^2 = 2|A|v_j - 2\sum_{c\in A} y_{cj} = 0, \qquad v_j = \frac{1}{|A|}\sum_{c\in A} y_{cj}. \tag{62}$$

Each split minimizes the sum of squared errors over the seven strategies. The tested maximum depths are two, four and six. Each depth is tested with a minimum of either one or three training batches per child.

For the linear models, we first subtract the training mean of each feature and divide by its training population standard deviation, computed with divisor equal to the number of training batches. A constant feature uses divisor one. Let Z contain these standardized feature rows with a leading intercept column, and let Y contain the responses. Ridge regression minimizes

$$J(W) = \|ZW - Y\|_F^2 + \lambda \operatorname{tr}(W^T P W), \qquad P = \operatorname{diag}(0, 1, \ldots, 1). \tag{63}$$

Here W is the matrix of fitted coefficients, $\|\cdot\|_F^2$ is the sum of squared matrix entries, and P leaves the intercept unpenalized. Differentiating each entry of W gives

$$\nabla_W J = 2Z^T(ZW - Y) + 2\lambda PW = 0, \qquad (Z^T Z + \lambda P)W = Z^T Y. \tag{64}$$

We solve this system for $\lambda = 0.1, 1, 10$.

The nearest-neighbour models estimate a cost from similar training batches. Eligible neighbours must have the same graph, loop count, external-momentum count, propagator counts, and number of symbolic invariants. They choose up to two or four training batches with the smallest squared distance in standardized features. The predicted response is the mean of their response vectors. The leaf-mean derivative above also derives this local constant estimate. If no eligible neighbour exists, the predictor returns equal costs and the selector uses native choices.

Let j_* be the strategy with the smallest predicted logarithmic cost $\widehat{y}_j(\boldsymbol{f})$. Validation selects a margin $\gamma \in \{0, 0.05, 0.15\}$, and the model chooses the strategy as follows:

$$\widehat{j}(\boldsymbol{f}) = \begin{cases} j_*, & \widehat{y}_{j_*}(\boldsymbol{f}) < \widehat{y}_0(\boldsymbol{f}) - \gamma, \\ 0, & \text{otherwise.} \end{cases} \tag{65}$$

Subtracting the two predicted logarithmic costs shows that the condition requires a predicted relative cost below $e^{-\gamma}$. Validation selects the model and margin with the smallest geometric mean time ratio. The selected settings remain fixed during reserved tests. This finite comparison establishes the best measured candidate within the study. It does not prove a globally optimal reduction strategy.

Q.1 Parameter points and target lists

The comparison uses the sunrise, planar box, three-loop vacuum, three-loop ladder, nonplanar box, pentagon, and box with external virtuality, with the same denominator routings and kinematic conventions as the preceding tests. The definitions below specify the parameter values and target lists for each split. Let h identify a rational parameter profile. Training uses $h = 41, 42$, validation uses $h = 43$, and testing uses $h = 44, 45$. In a common squared-momentum unit, the sunrise has squared masses $(3, 7, 13)$ and $p^2 = -1009 - 12h$. The planar box has $(s, t) = (-1009 - 12h, -1021 - 14h)$. The three-loop vacuum has squared masses $(1, 1, 1, 13, 13, 53 + 2h)$. The ladder keeps squared masses $(1, 0, 0, 0, 0, 0, 0, 0)$ and has $p^2 = -1009 - 12h$. The scattering assignments are

$$(s, t)_{\mathrm{NP}} = (-113 - 18h, -127 - 10h), \tag{66}$$

$$(s_{12}, s_{23}, s_{34}, s_{45}, s_{51})_{\mathrm{P}} = (-11 - 20h, -13 - 20h, -17 - 20h, -19 - 20h, -23 - 20h), \tag{67}$$

$$(s, t, v)_{\mathrm{box}} = (-113 - 18h, -127 - 10h, -149 - 12h). \tag{68}$$

These assignments fix the rational parameter values in each profile while leaving the dimension symbolic. The additional symbolic batches instead retain all independent invariants of their scattering graph as symbols.

For the four earlier multiloop graphs, we retain the ordered target list that starts with the corner, raises each physical denominator power once, and then introduces each single auxiliary numerator. We repeat these steps after removing one, two and three physical denominators, retain at least two, and remove duplicates. Each rational profile supplies a four-target batch and a batch of the first 32 targets, or all 21 targets for the sunrise. The three scattering graphs use the same construction with at most one removed denominator. Their larger batches have 32, 31 and 21 targets for the nonplanar box, pentagon and virtuality box.

For the symbolic nonplanar box and pentagon, training takes target entries 12–15, validation takes 16–19, and testing takes 20–23, with numbering starting at zero. These lists do not overlap. For the symbolic virtuality box, the respective single targets are $(3, 1, 1, 1)$, $(1, 3, 1, 1)$ and $(1, 1, 3, 1)$. These targets have two extra powers on one physical denominator. The rational batches reuse target patterns at different parameter points.

The joint-request test uses the larger rational batch at $h = 44$ for each graph and the three symbolic test batches. The symbolic virtuality box also includes $(1, 1, 1, 3)$, so it can supply two disjoint requests. A group of n targets is divided after entry $\lfloor n/2 \rfloor$, with the first request containing that many entries. The two requests share the same family definition and are available together, and their union contains each target once. By linearity, selecting the corresponding target rows from the joint reduction returns the coefficients for each original request. This separation leaves every coefficient unchanged and requires no new integral identity. The test reuses reserved parameter points and keeps the learned model fixed.

Q.2 Combining the measured times

Let m_{cj} be the median of the five test times for batch c and method j, and let method zero be its stated native control. For graph g with batch set C_g, we define

$$R_{gj} = \exp\left[\frac{1}{|C_g|}\sum_{c\in C_g}\log\frac{m_{cj}}{m_{c0}}\right], \qquad R_j = \exp\left[\frac{1}{7}\sum_{g=1}^{7}\log R_{gj}\right]. \tag{69}$$

The logarithm of a product equals the sum of the logarithms, so each expression is the corresponding geometric mean of the time ratios. The second expression assigns equal weight to each graph even when the numbers of batches differ. A ratio $R_j < 1$ means less time. The percentage saving is $100(1 - R_j)$, and the reciprocal $1/R_j$ is the speedup.

For the descriptive intervals, we sample seven graph labels with replacement, calculate their mean logarithmic ratio, and exponentiate. We repeat this 10,000 times and report the 2.5th and 97.5th percentiles. This procedure describes variation among the seven tested graphs. It does not provide a population guarantee for other integral families. The joint-request comparison uses the same calculation, with separate native requests as its first control. Its parameter points overlap the main reserved test, so it is an additional comparison on the same test population.

R Separate cost models for each graph

The graph-specific study changes the target batches and available strategies while retaining the seven denominator definitions and parameter formulas in Appendix Q. This appendix specifies those changes and the rule for deciding whether a new predictor is used. Development uses profile $h = 61$, training uses $h = 62, 63$, validation uses $h = 64$, and the reserved test uses $h = 65, 66$. The dimension remains symbolic. Related target batches share the same split, so their rational kinematic values cannot pass from the training set into the reserved set. The graph structures and target patterns occur in both sets.

Let P be the number of physical propagators and let $\boldsymbol{b} = (1, \ldots, 1, 0, \ldots, 0)$ have P entries equal to one. Let $\boldsymbol{e}_j$ be the unit vector for index j. The first batch contains $\boldsymbol{b} + 2\boldsymbol{e}_j$ for $j = 1, \ldots, \min(P, 3)$. Thus one denominator has power three in each target. For the second batch, define the lower-sector corner

$$\boldsymbol{b}^{\mathrm{low}} = \begin{cases} \boldsymbol{b} - \boldsymbol{e}_1, & P < 5, \\ \boldsymbol{b} - \boldsymbol{e}_1 - \boldsymbol{e}_3, & P \geq 5. \end{cases} \tag{70}$$

Include this corner and up to three single-dot targets, $\boldsymbol{b}^{\mathrm{low}} + \boldsymbol{e}_j$ with $b_j^{\mathrm{low}} = 1$, taking the first available indices in increasing order. The third batch uses the original corner $\boldsymbol{b}$ and contains $\boldsymbol{b} - 2\boldsymbol{e}_j$ for the first three auxiliary entries, or all auxiliary entries when fewer than three exist. When a family has no auxiliary entry, this construction repeats the first batch and we remove the duplicate. These formulas specify the requested integral indices. Their master coefficients are obtained by exact reduction and are not used to construct the model inputs.

R.1 The allowed changes to Kira

For target batch $\mathcal{T}$ and a target $\boldsymbol{a}$, let $S(\boldsymbol{a}) = \{i : a_i > 0\}$ be its positive-index set. One strategy supplies only maximal target sectors:

$$\mathcal{S}_{\max} = \{S(\boldsymbol{a}) : \boldsymbol{a} \in \mathcal{T},\ S(\boldsymbol{a}) \not\subseteq S(\boldsymbol{a}') \text{ for every } \boldsymbol{a}' \in \mathcal{T}\}. \tag{71}$$

Kira then includes their subsectors. To see why this is a useful candidate, consider the shifted integrals produced by the IBP derivative. A raised index $a_i + 1$ carries a factor a_i, and the other shifts lower an index. If $a_i = 0$, the raising term vanishes; if $a_i < 0$, raising it once leaves it nonpositive. Thus an IBP equation seeded in one sector stays in that sector or its subsectors. The sector property makes this restriction a useful candidate, but it does not establish that the restricted pool contains every relation needed for the prescribed basis. We therefore retain the basis check and the option to repeat the calculation with the full sector set.

Another strategy uses Kira's numerator truncation [5]. In a sector with t positive indices, Kira restricts the inherited numerator bound by

$$s \leq \max(1, t - \tau + 1, s_{\mathrm{explicit}}), \tag{72}$$

where τ is the chosen integer cutoff and s_{explicit} is a bound explicitly supplied for that sector. We test $\tau = P - s_c$ and $\tau = P - s_c + 1$, where s_c is the largest target numerator rank. The top-level

sector retains its explicit target bound. The first cutoff combined with maximal target sectors forms the deterministic control. The development probes also test $\tau = P - s_c - 1$.

The remaining choices use the original bounds B_c, the smaller bounds $B_c - (1, 0, 1)$ with negative entries replaced by zero, the larger bounds $B_c + (1, 1, 1)$, Kira integral ordering five [19], or the preceding trained equation filter. Together these choices give nine strategies. Before each attempt, every strategy enforces a nonempty seed region in its declared top-level sectors. If $\mathcal{S}_0$ is that sector list and r_{req} is the requested positive-index bound, the actual bound is

$$r_{\mathrm{run}} = \max\left(r_{\mathrm{req}}, \max_{S \in \mathcal{S}_0} |S|\right). \tag{73}$$

Each positive index is at least one, so a seed in sector S has $\sum_{i \in S} a_i \geq |S|$. This proves the lower bound used in Eq. (73). Enforcing this bound prevents an invalid configuration from causing a failed startup before any meaningful reduction can occur. Identical input configurations share one training or validation measurement. If a proposed reduction fails, it retries with native choices at B_c and then at $B_c + (1, 1, 1)$. The recorded cost includes every attempt.

R.2 Fitting and selecting the models

Each graph has its own predictors of $y_{cj} = \log(t_{cj}/t_{c0})$, where t_{cj} is the median complete cost of strategy j on batch c and zero denotes native choices. The nine input features are the target count; maximum dot count and numerator rank; minimum and maximum positive-index counts; mean dot count and numerator rank; number of maximal target sectors; and number of physical propagators. The features are calculated from the requested indices and the family definition, so no reduced coefficients enter the prediction.

We fit ten candidates per graph: one constant predictor, trees of maximum depth one, two and three, a bagged ensemble of sixteen depth-three trees, ridge regressions with penalties $0.1, 1, 10$, and nearest-neighbour predictors with one and three neighbours. Trees use squared-error splits with at least one row in each child. Each ensemble tree receives a bootstrap sample of the training batches and uses all features at each split. This follows bootstrap aggregation [22]. For fitted trees $f_1, \ldots, f_{16}$, the ensemble prediction is

$$\widehat{y}_j(\boldsymbol{f}) = \frac{1}{16} \sum_{b=1}^{16} f_{bj}(\boldsymbol{f}). \tag{74}$$

This average also follows by minimizing $\sum_b (f_{bj}(\boldsymbol{f}) - v_j)^2$ with respect to v_j, using the leaf-mean derivative in Appendix Q. That appendix derives the ridge fit and the local-mean estimates used here. Neighbour distances use the training standard deviations; a constant feature uses divisor one. We use two timing repetitions for training and three for validation. Within each batch comparison, every strategy uses the same computer. Different training batches can use different computers, and the training response normalizes each cost to its own batch baseline.

Validation considers decision margins $0, 0.05, 0.15$ in Eq. (65). It chooses the candidate and margin with the lowest complete cost separately for each graph. The controls are native choices, the deterministic sector-and-truncation rule, and the single strategy with the lowest geometric mean training cost for that graph. A fixed strategy can include the older equation filter. We enable the new model only when its validation cost ratio to the fastest control is at most 0.95. Otherwise, the method uses that control. The threshold determines when the model is enabled; statistical interpretation comes from the separately reported test measurements. Unknown graph structures and features outside the fitted range use native choices.

The reserved comparison uses five repetitions of native choices, the deterministic rule, the fixed strategy per graph, and the selected method. The order rotates across batches and repetitions, giving every method the same number of appearances in each run position over the full test. Each

run uses the same targets, prescribed basis, executable, and one processor. All methods retain the preceding exact implementation improvements, so the native control uses that same implementation with native selection rules. Full time includes input preparation, prediction, native IBP generation, exact reduction, coefficient checks, cleanup, and retries. Offline fitting, reference discovery, and saving measurement records stay outside the timer. Separate application checks receive the basis without target coefficients and compare their answers only after completion. The calling program and fitted parameters are loaded before the timing trials, and each trial starts a fresh Kira process. The additional application checks also record time with a newly started calling program; they do not supply a matched speed comparison for that condition.

We combine five-run median ratios within each graph and then give the seven graphs equal weight, as derived in Appendix Q. The aggregate intervals resample graph groups. Per-graph plot bars show the range across the two reserved kinematic profiles. These ranges show the variation observed in two profiles. The sample is too small to support a general significance claim for the integral type.

S Selective seed expansion and exact basis completion

The seed-expansion study asks whether a smaller first calculation and an exact conversion of its remaining integrals can be cheaper than one large reduction. It uses the seven families in Appendix Q and the target definitions in Appendix R. Development uses profile $h = 71$, training uses $h = 72, 73$, validation uses $h = 74$, and the reserved test uses $h = 75, 76$. These are separate rational parameter profiles. The graph structures and target index patterns occur in all splits, and d remains symbolic.

S.1 Changing individual seed bounds

The bounds $B_c = (r_c, s_c, q_c)$ retain the definitions in Appendix Q: r_c bounds the sum of positive indices, s_c bounds the numerator rank, and q_c bounds the number of extra denominator powers. Each component starts from the largest value among the requested targets. We first enforce the sector lower bound in Eq. (73). The six new direct reductions use

$$B_c + \boldsymbol{\delta}, \qquad \boldsymbol{\delta} \in \{(1,0,0), (0,1,0), (0,0,1), (1,0,1), (1,1,0), (0,1,1)\}. \tag{75}$$

For a seed with t positive indices, summing $a_i = 1 + (a_i - 1)$ over those indices gives $r = t + q$. The positive-index and extra-power limits can therefore overlap, and different requested triples can give the same seed region after all limits are applied. The labels therefore identify requested limits rather than necessarily distinct equation sets. Their purpose is to allow selective expansion, such as requesting more numerator identities without also increasing both denominator limits. Sufficiency remains a property to check for the resulting system. Each direct reduction keeps the prescribed-basis check described in Ref. [5]. If it fails, the calculation calls the preceding fixed graph-specific method and includes both attempts in its time.

S.2 Completing a temporary basis

We also test two ways to complete a smaller reduction. The first stage uses B_c and supplies the preferred masters, while allowing Kira to return additional integrals. Kira supports this behaviour when its early master-basis check is disabled [5]. Let M_m denote prescribed masters and let U_u denote additional integrals that remain in the first-stage output. For target I_t, this stage returns exact relations

$$I_t = \sum_m A_{tm} M_m + \sum_u C_{tu} U_u. \tag{76}$$

Here A_{tm} is the coefficient of prescribed master M_m, and C_{tu} is the coefficient of unresolved integral U_u. These coefficients are rational functions of the kinematics and d. The integrals U_u are unresolved in the smaller first-stage system and need not be independent masters of the full family. Their index vectors define the targets of the second reduction, allowing that calculation to focus on the integrals that actually remain. That request uses its own componentwise target bounds plus $(1, 1, 1)$ and again supplies the prescribed basis. One candidate uses the maximal sectors of the U_u and their subsectors; the other keeps the original top-level sectors. Both retain the positive-index lower bound.

If the second reduction succeeds, it gives

$$U_u = \sum_m H_{um} M_m. \tag{77}$$

The coefficient H_{um} multiplies master M_m in the reduction of U_u. Substitution of Eq. (77) into Eq. (76) yields

$$I_t = \sum_m A_{tm} M_m + \sum_u C_{tu} \sum_m H_{um} M_m \tag{78}$$

$$= \sum_m \left(A_{tm} + \sum_u C_{tu} H_{um} \right) M_m. \tag{79}$$

The final coefficient matrix is therefore $A + CH$. Computing its entries exactly in the rational-function field, cancelling common factors, and checking the final integral labels gives a reduction in the prescribed basis. When either reduction fails, the preceding method supplies a full fallback. If no additional integral occurs, the second reduction is unnecessary. All basis conversion and fallback costs enter the timer. Independent reference coefficients check the final result only; the policy and the two reduction stages do not receive them.

S.3 Training, model selection, and comparison

The development set compares the preceding method, six direct expansions, and two basis-completion choices. We retain the preceding method and each case's fastest candidate when it lowers that case's complete time by at least ten percent. This rule selects candidates before training and does not inspect reserved data.

We then fit the ten model types and nine target features defined in Appendix R, separately for each graph. The new response is $\log(t_{cj}/t_{\mathrm{cprev}})$, so it directly measures improvement over the preceding method. Here t_{cj} is the median complete time of candidate j on batch c, and t_{cprev} is the corresponding time of the preceding method. Two repetitions supply each training label, and three repetitions supply each validation cost. A fixed strategy minimizes geometric mean training time for each graph. Validation first selects the faster control between that fixed strategy and the preceding method. It then tests the fitted predictors at margins $0, 0.05, 0.15$ and enables one only when it lowers the control's validation time by at least five percent. The fitting formulas and their derivations remain those in Appendices Q and R.

The fixed control may call the learned method retained from the preceding study. Its purpose is to isolate the benefit of the new choice among candidates while preserving the earlier improvements in every method.

The chosen graph policy and candidate set are fixed before the reserved tests. Inputs with another graph structure or target features outside the training range use the preceding method. That method retains its own native fallback. Each reserved batch compares native choices, the preceding method, the new fixed strategy, and the selected new method on the same computer. Five repetitions use a rotating order. All methods use the same executable and exact final coefficient checks. The time includes preparation, prediction, Kira, any basis conversion, checking,

cleanup, and every retry. Offline training, independent reference generation, and writing benchmark records remain outside these times. The selection program and fitted parameters are loaded before these trials; each attempt starts a fresh Kira process. Separate application tests start a new caller and supply only the targets, family, and prescribed basis. They check the coefficients after the reduction.

We first take the median time for each batch and method, then combine ratios within each graph and give all seven graphs equal weight. The geometric-mean derivation in Appendix Q also applies here. Descriptive aggregate intervals resample the seven graph groups. The per-graph bars show the range over the two reserved parameter profiles. These ranges describe the tested profiles and do not establish a gain for all integrals with the same graph.

T Learned choices of integral and propagator order

This appendix specifies the ordering choices and explains why consistent relabelling leaves the integrals unchanged. The comparison uses the family definitions and target patterns in Appendix R. We use parameter profile 81 for development, profiles 82 and 83 for training, profile 84 for validation, and profiles 85 and 86 for the reserved comparison. Each profile uses the rational parameters defined there, while the dimension d remains symbolic. These are new parameter values of known graphs and target patterns.

T.1 What the choice changes

Kira provides eight integral orderings [19]. Order one treats numerator powers as simpler than extra denominator powers, called dots; order two reverses that preference. Orders three and four first compare their sum and then favour numerator powers or dots, respectively. Orders one through four compare sectors by sector number. Orders five through eight use the corresponding index rules but first compare the number of positive indices and favour subsectors of the declared top sectors. We also test the four automatic propagator permutation schemes of Kira [5]. The final coefficients use the original index convention in every comparison.

The development candidates retain the preceding method, test each integral order from two through eight with propagator scheme one, and test propagator schemes two through four with integral order one. These are eleven initial choices. For each development batch, we also combine its fastest nondefault integral order with its fastest nondefault propagator scheme and time that pair. This adds one development calculation per batch. The preceding seed policy remains active in each new choice. It can use truncation, selected seed bounds, or temporary-basis completion as defined in the preceding appendices. A changed ordering is applied to each Kira attempt in that method. If it fails to reduce to the prescribed basis, we repeat the preceding method with its original settings. Both attempts enter the recorded time. This finite comparison does not search every joint ordering or every possible elimination sequence.

To see why changing labels preserves an integral, let π be a permutation of the N propagator positions. Set $D'_i = D_{\pi(i)}$ and $a'_i = a_{\pi(i)}$. Reindexing the product gives

$$\prod_{i=1}^{N} (D'_i)^{-a'_i} = \prod_{i=1}^{N} D_{\pi(i)}^{-a_{\pi(i)}} = \prod_{j=1}^{N} D_j^{-a_j}, \qquad I_{D'}(\mathbf{a}') = I_D(\mathbf{a}). \tag{80}$$

Here I_D denotes the integral defined by propagators D_i. Since the relabelling also leaves the measure unchanged, it preserves the integral. Changing the priority of integrals instead changes the sequence in which Kira solves the IBP equations. If $A\mathbf{I} = 0$ denotes those equations and P is the permutation matrix that reorders the integral columns, then

$$\mathbf{J} = P\mathbf{I}, \qquad AP^{-1}\mathbf{J} = A\mathbf{I} = 0. \tag{81}$$

The permutation therefore represents the same equations with a different column order. In a finite calculation, however, the ordering can also affect which equations are selected, so the equivalence of representations does not replace the checks of completeness and exact output coefficients. We require every target to reduce to the prescribed master basis and compare every output coefficient with an independent expanded-seed reduction. The predictor receives neither those coefficients nor the reference reduction steps.

T.2 Training, validation, and timing

For each development batch, we retain the fastest candidate if its full time is at least ten percent below the preceding method. We also retain the preceding method. The resulting candidate set is fixed before training. We fit the same ten predictor types per graph as in Appendix S, using the same nine features and complete-time log-ratio labels. Training has two measurements per candidate and batch; validation has three. Validation chooses a fixed control and enables a predictor only when it saves at least five percent against that control. We freeze the models, controls, and selection requirements before using the reserved profiles.

Each application selects its ordering before Kira creates integral weights and generates equations. The prediction is made once per requested batch. The selected method retains the earlier seed policy, and Kira performs all algebraic operations. The decision selects a setting for the complete reduction before it starts. Kira continues to choose individual row operations, as required by the earlier row-model comparison.

The reserved comparison measures native choices, the preceding method, the training-selected fixed setting for each graph, and the new selected method. It uses five repetitions with rotating execution order. The same worker handles all methods for each batch. The timer includes prediction, input preparation, native equation generation, exact algebra, basis conversion where needed, coefficient checks, fallback, and cleanup. The caller and models are already loaded, and each attempt starts a new Kira process. Offline development, reference construction, training, and record writing are excluded. Additional applications receive the target indices and master basis without reference coefficients. Their results are checked independently after the reduction.

U Selecting the number of Kira workers

The worker-count study retains the seed strategy from Section H.3 and varies only the number of Kira workers used for a requested batch. The families and target patterns are those in Appendix R. Parameter profile 91 supplies the development test, profiles 92 and 93 supply training, profile 94 supplies validation, and profiles 95 and 96 supply the reserved comparison. The dimension remains symbolic. Target patterns overlap across these sets; the rational kinematic values differ.

U.1 Choices and exactness

The preceding learned method chooses seed bounds and can reduce through a temporary basis. We retain that method and let a new policy choose one, two, or four Kira workers for each requested batch. The choice applies to each Kira attempt, including a temporary-basis conversion. Kira supports parallel symbolic reduction [4]. Changing the worker count changes how operations are scheduled, while each operation continues to use exact rational arithmetic. Every output must reach the prescribed master basis and agree with an independent expanded-seed reduction. If a new choice fails, its time and the repeat with the preceding one-worker method both enter the complete time.

Let $A\mathbf{I} = 0$ denote the IBP system. An exact elimination step left-multiplies it by an invertible

elementary matrix E, so

$$A\mathbf{I} = 0 \quad \Longleftrightarrow \quad EA\mathbf{I} = 0. \tag{82}$$

Multiplication by E proves the forward implication, and multiplication by E^{-1} proves the reverse. Repeating valid steps therefore preserves the system, provided parallel execution respects their dependencies. Kira manages those dependencies through its existing workers, while the predictor sets only their number. This argument preserves relations but does not prove that a finite seed set is sufficient. The basis and coefficient checks therefore remain part of every measured reduction.

U.2 Cost labels and selection

Write x for the nine target features defined in Appendix S, and $q \in \{1, 2, 4\}$ for the worker count. For training batch b, let $t_{b,q}$ be the median complete time of two measurements. The training label is

$$y_{b,q} = \log(t_{b,q}/t_{b,1}). \tag{83}$$

Because the logarithm is strictly increasing and $t_{b,1} > 0$ is common to all choices for that batch, minimizing this label also minimizes $t_{b,q}$. We fit the same ten predictor types per graph as in Appendix S. They include constants, trees, bagged trees, regularized linear fits, and nearest-neighbour fits.

The training-selected fixed control minimizes $\sum_b t_{b,q}$. Validation uses three measurements per choice and batch. It first selects the faster of that fixed control and the preceding method, using the sum of batch medians. It then compares each predictor with decision margins 0, 0.05, and 0.15 in log relative cost. A predictor is enabled only if its summed median cost is at least two percent lower than the control and its summed cost is lower in at least two of the three validation repetitions. Otherwise the control stays active. This objective measures the time for the workload by summing batch costs. It therefore gives a different weighting from the geometric mean that treats graphs equally. The selected controls and models remain fixed throughout the reserved tests. Unknown graph structures or features outside the training range use the preceding method.

U.3 Complete-time comparisons

The reserved comparison has 36 batches and five repetitions. It measures native choices with one worker, the preceding learned method with one worker, that same method with four workers throughout, a training-selected fixed worker count per graph, and the final selected method. All methods use the same integrated executable and its common earlier optimizations. The native control uses Kira's native seed and reduction choices in that executable. All methods run sequentially in the same four-core allocation. Execution order rotates between repetitions. Thus the always-four-worker control measures the cost of using the extra cores without target-dependent selection.

Full time includes model decisions, input preparation, Kira startup, equation generation, exact algebra, any basis conversion, coefficient checks, fallback, and cleanup. The caller and models are already loaded, while every attempt starts a new Kira process. Offline reference construction, training, and record writing are excluded. CPU time sums user and system time of the caller and its completed child processes. This measures the CPU time actually consumed. Every method has four reserved cores, including the methods that use only one worker. The comparison reports both the sum of batch medians and the graph-weighted geometric ratios defined in the earlier appendices. Additional applications receive the targets and basis without reference coefficients and are checked after reduction.

V Connected scattering families and the initialization test

The restricted vacuum recurrence uses a particular denominator structure. To examine a different physical problem, we construct connected massless four-point families with three independent external momenta. Let $p_1 + p_2 + p_3 + p_4 = 0$, $p_i^2 = 0$, $P = p_1 + p_2$, and $Q = p_1 + p_2 + p_3 = -p_4$. We define

$$\begin{aligned} s = P^2, \qquad t = (p_2 + p_3)^2, \\ 2p_1 \cdot p_2 = s, \qquad 2p_2 \cdot p_3 = t, \\ 2p_1 \cdot p_3 = -s - t. \end{aligned} \tag{84}$$

The first two scalar-product relations follow by expanding the squared momenta and using $p_i^2 = 0$. Expanding $Q^2 = p_4^2 = 0$ then gives the third relation. Dimension d remains symbolic. Rational profiles use $s = -1009 - 12h$ and $t = -1021 - 14h$ in a common squared-momentum unit. The initial probes use $h = 211$, the short comparison uses $h = 223, 224$, and the longer comparison uses $h = 225$. An attempted symbolic probe produces no retained measurement and is excluded from the numerical results.

For an L-loop four-point ladder, the physical inverse propagators are

$$\begin{aligned} D_i = \ell_i^2, \qquad D_{L+i} = (\ell_i - P)^2 \quad (1 \le i \le L), \\ D_{2L+i} = (\ell_i - \ell_{i+1})^2 \quad (1 \le i < L), \\ D_{3L} = (\ell_1 - p_1)^2, \qquad D_{3L+1} = (\ell_L - Q)^2. \end{aligned} \tag{85}$$

The Feynman prescription of Eq. (1) is implicit. There are $2L + (L-1) + 2 = 3L + 1$ physical propagators. The differences between adjacent loop momenta connect the loops, while the two end propagators carry the external momentum dependence. This four-point ladder differs from the three-loop two-point ladder in the earlier comparisons.

The second family is a box with massless bubble subgraphs. Its physical inverse propagators are

$$\begin{aligned} D_1 = \ell_1^2, \qquad D_2 = (\ell_1 - p_1)^2, \\ D_3 = (\ell_1 - P)^2, \qquad D_4 = (\ell_1 - Q)^2, \\ D_{2j+1} = \ell_j^2, \qquad D_{2j+2} = (\ell_j - \ell_1)^2 \quad (2 \le j \le L). \end{aligned} \tag{86}$$

There are $4 + 2(L-1) = 2L + 2$ physical propagators. Each additional loop couples to the box momentum ℓ_1. The family admits subloop integration, so it is a restricted scattering class; it does not represent an arbitrary L-loop graph.

For three independent external momenta, a complete scalar-product representation has

$$N = L + \frac{L(L-1)}{2} + 3L = \frac{L(L+7)}{2} \tag{87}$$

entries. The three terms count loop squares, distinct products of two loop momenta, and loop–external products. This is the usual counting used to complete Kira families [5]. Thus $N = 15, 22, 30, 39$ for $L = 3, 4, 5, 6$. We append independent auxiliary inverse propagators until this rank is reached. The candidates are ℓ_i^2 and $(\ell_i - v_j)^2$, where the ordered list of v_j contains the remaining loop momenta followed by p_1, p_2, p_3. Exact row reduction of their scalar-product coefficients retains a candidate only when it increases the rank.

Each batch in the earlier scattering comparison contains the full-sector integral, the integral with D_1 raised to power two, and the integral with one power of the first auxiliary inverse propagator in the numerator. All other physical indices are one and all remaining auxiliary indices are zero. A reference reduction uses positive-index limit $n + 2$, numerator limit one, and dot limit two, where n is the number of physical propagators. A second reduction increases the numerator limit to two and must reproduce all coefficients in the reference basis. At least one target must have a nonzero reduction that differs from the identity. A failed reference or failed enlargement prevents a completed-speedup claim for that case.

V.1 Why the declared sector domain closes under IBP

The initialization experiment restricts zero-sector classification to the declared top sectors and their subsectors. This restriction requires a mathematical justification because an omitted equation can otherwise prevent a complete reduction. Consider an index $a_j \leq 0$ for a propagator outside the declared sector. Differentiating its factor gives

$$\frac{\partial}{\partial \ell_k^\mu} D_j^{-a_j} = -a_j D_j^{-a_j-1} \frac{\partial D_j}{\partial \ell_k^\mu}. \tag{88}$$

The only increase of a_j is therefore multiplied by a_j. At $a_j = 0$ this term vanishes, and at $a_j < 0$ the increased index remains nonpositive. Rewriting scalar products in the numerator as linear combinations of inverse propagators can lower indices, but cannot introduce a new positive index. Consequently, IBP equations seeded in a sector remain within that sector and its subsectors. The union of several such domains also closes under these equations.

The test applies the existing zero-sector criterion within this domain. It does not declare omitted integrals to be zero. The experiment uses a single family, no cut propagators, and no symbolic propagator powers. It retains Kira's exact elimination and admits only the prescribed final basis. Comparisons with the unchanged initialization check the coefficients on the seven earlier graph types before the new scattering tests are interpreted. This establishes the stated restricted implementation on the tested inputs; it does not prove that every optional symmetry or extra-relation procedure can be restricted in the same way.

For a single top sector containing n positive indices, its subsectors number 2^n, since each propagator is either present or absent. Classifying all N family entries instead considers 2^N sectors. At four loops this changes the candidate count from 2^{22} to 2^{13} for the ladder and to 2^{10} for the bubble box. These are counts of initialization candidates, not predictions of total runtime.

V.2 Separating the contributions to complete time

The component comparison uses the seven earlier graphs at profiles $h = 221, 222$ and the three full-sector targets with one physical propagator raised from power one to power three. The construction uses the first three physical propagators. It compares ten methods on the same targets and prescribed basis: native choices, fixed and learned graph strategies, fixed and learned seed expansion, fixed and learned worker counts, the equation filter, the seed filter, and the row model. Fixed settings come from the earlier training data, and all model parameters remain frozen.

Every completed method has three timing repetitions. The method order rotates within each case and repetition, and every method receives the same two-core allocation. The complete timer includes decisions, preparation, exact algebra, coefficient checks, retries, and cleanup. It excludes offline reference construction, training, and the initial loading of the caller and models. A larger reference reduction must reproduce the coefficients before a case enters the timing comparison. Ratios compare batch medians on matching completed inputs, using the complete-time definitions of this study. Cases that cannot complete remain in the failure table and receive no completed-time ratio.

The first scattering comparisons allow 120 seconds per native attempt, including a ten-second reserve for export checking and cleanup. A follow-up increases this allowance to 600 seconds. Both initially limit solver address space to 2,400 MiB and summed resident memory to 2,100 MiB. When the larger reference checks reach the memory limit, a further comparison reserves more memory through the scheduler and raises these limits to 6,144 and 5,200 MiB, respectively. Its native and learned methods use identical limits. Temporary files remain bounded by 32 MiB, and all earlier failed attempts remain in the results. A completed earlier reference can be reused as an offline coefficient check; every reported follow-up time comes from a new reduction.

The fixed control at each stage can retain choices from earlier stages. The first controls use the best fixed training settings. A further 96 paired reductions disable only the current learned decision and use its fixed validation control, while retaining earlier stages. This second comparison isolates the added choice more directly, while comparison with native choices measures the combined strategy. These effects need not add or multiply across stages, since a changed seed region alters the equations and later algebra. The new scattering comparison additionally tests the frozen equation and row models, the complete strategy selector with its domain checks, and two separate requests against their combined reduction. The separated requests use the same frozen seed limits as the combined request, which isolates repeated work under those settings. Failed filtered attempts retry with native choices and retain that cost.

V.3 Why the vacuum formula does not establish a scattering reduction

For the vacuum family, the derivation above gives closed recurrences in $z = L(d-2)$ under its specific denominator and mass assumptions. The families in Eqs. (85) and (86) also depend on two external invariants. Dimensional scaling alone therefore permits dependence on their ratio t/s; it cannot determine that dependence from z. Applying the vacuum formula would require a new identity relating these scattering coefficients, which the vacuum derivation does not supply. We consequently exclude that formula from the scattering comparison. Combining compatible target requests remains a separate, generally applicable use of the linear reduction map, as derived in Appendix Q.

W Learning one policy for complete reduction time

The component controls show that adding another learned decision can increase time when a fixed setting already suits the targets. We therefore replace the sequence of strategy selectors with one policy that compares complete calculations before equation generation. Its purpose is to choose a small sufficient calculation when possible and to avoid an expensive failed attempt when more seeds are needed. Kira continues to generate and solve the exact identities.

W.1 Model, training data, and comparison

The policy uses the target indices and the family's loop, momentum, mass, and denominator structure. It compares seven choices: the target-based native seed limits, three smaller seed regions, integral ordering 5, the existing equation filter, and direct expansion of the seed limits. Each training label is the measured time of a complete reduction, including retries and exact coefficient checks. The new model therefore learns the cost of an insufficient equation set as well as the cost of a successful first attempt. Appendix X gives the inputs, cost function, prediction rule, and recovery steps.

We fit regression trees, random forests, and ensembles of extremely randomized trees [9, 10, 11]. The comparison uses nine candidate models, with validation selecting both the model and a minimum predicted saving required to depart from its fixed control. The selected model is a regression tree with maximum depth 8 and 1 tree. Training supplies 47 complete cases, validation supplies 26 cases, and the reserved test contains 26 completed batches. Constructing and validating the cost data takes 4703.1 seconds of recorded reduction time, while fitting the nine models takes 0.409 seconds. These are offline costs; a saved policy can serve later requests without repeating them. The tests cover the earlier seven graph structures and connected bubble-box scattering families at three and four loops. Rational kinematics change between splits, while dimension remains symbolic. The symbolic scattering tests use disjoint target indices within each graph. These are tests within known structures, with complete definitions in the appendices.

Before fitting the models, we also remove two deterministic costs from Kira's preparation. The first reads the existing sector files directly in their original order. The second simplifies the test for zero sectors, whose integrals vanish in dimensional regularization. That test uses parametric polynomials formed from the propagator denominators [18, 5]. We set the parameters of propagators absent from every declared top sector to zero before constructing the test. Equation (99) proves that this change preserves the test within the declared domain. The complete polynomials remain available for other operations. Nine coefficient comparisons against earlier independent references pass before the new training begins. Both exact changes are present in every method in Table 32, so that table measures the policy after the common implementation improvement.

W.2 Results on the reserved targets

Table 32 gives sums of three-run batch median times. Native choices take 111.269 seconds, the validation-selected fixed choice takes 111.305 seconds, and the learned policy takes 110.693 seconds. The selected fixed choice is the target-based native seed region. The learned result uses 0.5% less time than native choices and 0.5% less time than the fixed control. Summing all repetitions gives learned/native and learned/fixed ratios of 0.9945 and 0.9940, respectively. These additional ratios show the effect of keeping every measured time. The same targets and prescribed master basis are used in each comparison, and every attempt uses one Kira worker within the same one-core, 8 GiB allocation. Figure 27 shows how the added effect varies across graph structures.

Table 32: Complete times on reserved targets, in seconds. Each graph row sums its batch medians. All three methods use the same improved executable; the fixed choice is selected on validation data.

Graph	Loops	Batches	Native	Fixed	Learned
Three-loop bubble box	3	1	2.628	2.599	2.654
Four-loop bubble box	4	1	29.580	29.746	30.454
Planar double box	2	3	9.450	9.664	9.195
Three-loop two-point ladder	3	3	24.190	24.626	23.381
Nonplanar box	2	4	31.250	30.932	31.167
Virtuality box	1	4	3.175	3.197	3.385
Pentagon	1	4	4.934	4.778	4.833
Sunrise	2	3	2.068	2.041	1.982
Three-loop vacuum	3	3	3.993	3.721	3.642
Total		26	111.269	111.305	110.693

All 234 timed reductions and 26 further applications agree exactly with their reference coefficients. The additional applications receive the master basis but no target coefficients before execution. A separate check of the public application interface completes 9 batches. Native choices require 15 recovery attempts across the timed tests, compared with 9 for the learned policy. The median decision time across the test batches is 0.519 milliseconds, including feature construction and tree evaluation. The prediction cost, all failed attempts, and recovery are included in complete time. Model loading, offline reference construction, and basis discovery are outside this repeated-reduction comparison. 1 planned reserved batch(es) lack a completed reference or enlarged check and receive no speedup estimate.

The four-loop confirmation follows a condition registered before fitting the model. When the original reserved reference check does not complete, a fresh profile uses the two nested seed regions already verified during development. The model remains fixed, and the original incomplete case remains in the failure count.

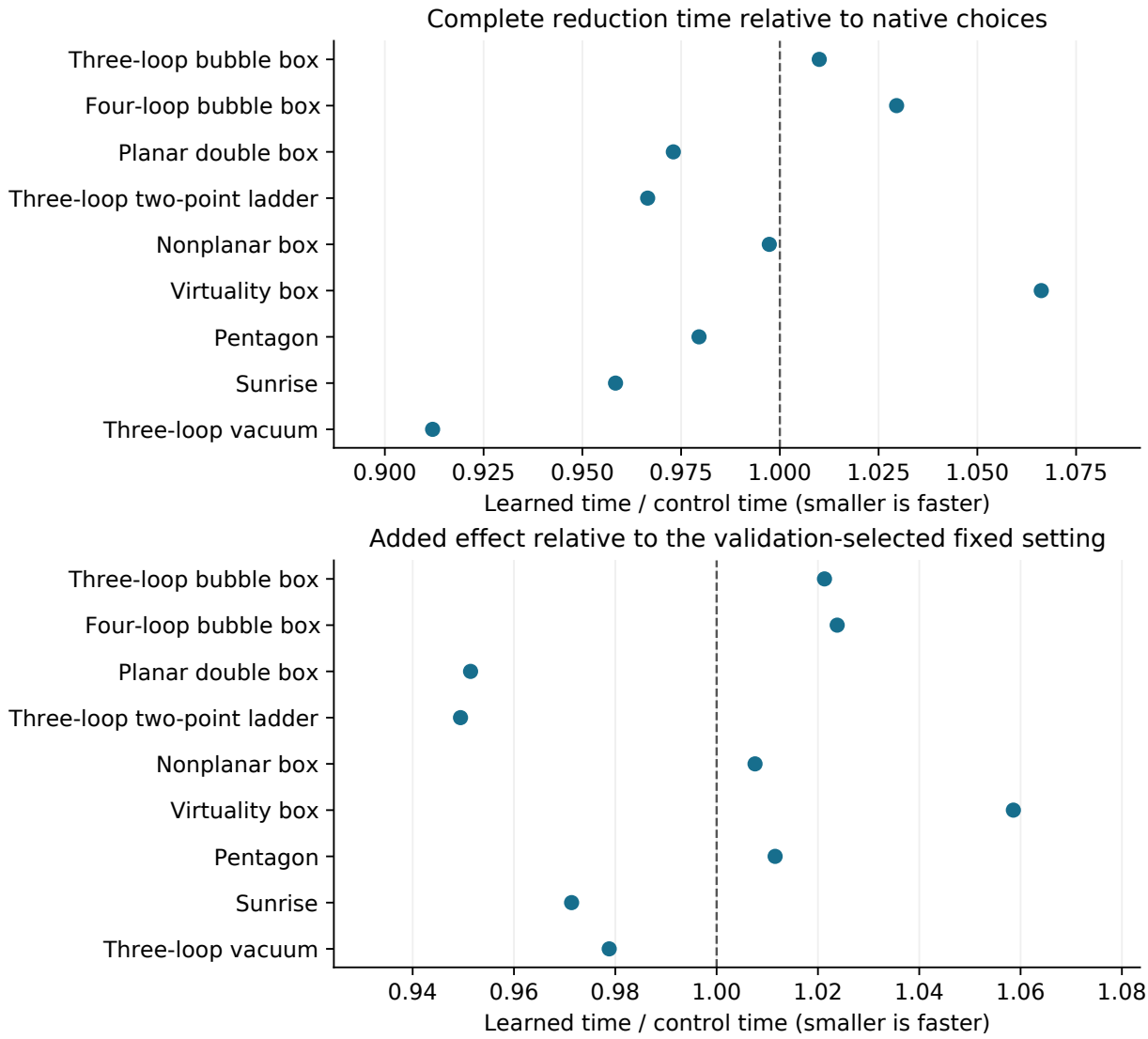


Figure 27: The upper panel compares the learned policy with native choices after the common exact improvements. The lower panel compares it with the best fixed validation choice. Each point divides sums of batch medians within one graph structure. Values below one indicate less time; the equal-time line remains visible. Three repetitions support descriptive comparisons, rather than a general speed guarantee.

W.3 A stronger fixed control and the deployed policy

One fixed setting for all graphs can be a weak control because different denominator structures can favour different seed regions. We therefore select a separate fixed setting for each graph using validation data only. We also fit 21 small regression trees separately for the graphs with sufficient training cases. They use the same measured cost labels and can specialize in the effect of target powers and numerator rank within one denominator structure. Validation compares these trees with the global model and selects a predictor and gate for each graph. Fitting the additional trees takes 0.723 seconds, with no additional label-generation cost.

We compare the graph-specific fixed settings, the original global model, and a validated policy that uses its selected predictor only where validation supports an additional saving. This procedure was specified before any reserved timings were collected. The final rule requires at least two percent less summed validation time and a saving in both validation repetitions; otherwise the graph's fixed setting is used. All choices remain fixed during the matched comparison. Appendix X defines the local models and selection rule.

Validation enables target-dependent predictions for planar double box, three-loop two-point ladder, nonplanar box. Table 33 gives a separate matched comparison on the same reserved targets, with three repetitions per method. The graph-specific fixed choices take 105.909 seconds, the cost model takes 103.896 seconds, and the validated policy takes 103.983 seconds, using sums of batch medians. The validated policy uses 1.8% less time than the graph-specific control. Across all repetitions, its time ratio is 0.9830. All 234 reductions pass exact coefficient checks. The fixed choices and validated policy require 21 and 9 recovery attempts, respectively. Figure 28 shows where predictions add value beyond a fixed choice for each graph.

Table 33: Matched complete times in seconds against the stronger fixed control. The validated policy keeps fixed choices where validation does not support the model.

Graph	Fixed per graph	Cost model	Validated policy
Three-loop bubble box	2.197	2.190	2.201
Four-loop bubble box	28.777	29.170	28.961
Planar double box	9.932	9.275	9.333
Three-loop two-point ladder	23.542	22.620	22.518
Nonplanar box	30.572	29.734	29.839
Virtuality box	2.207	2.160	2.213
Pentagon	3.124	3.295	3.371
Sunrise	1.855	1.873	1.791
Three-loop vacuum	3.702	3.576	3.756
Total	105.909	103.896	103.983

The public application uses the validated policy and completes the separate exact checks reported above. It keeps native choices for requests outside the recorded family structures and target ranges. An additional application check confirms this fallback for an unsupported graph label. This guard limits the domain in which the learned prediction is used; it does not establish performance on unseen graph structures.

W.4 Separating preprocessing and prediction

We repeat the connected three- and four-loop bubble-box calculations with each exact change enabled in turn, followed by the validated policy. Table 34 reports matched complete times from these separate trials. Every method receives the same 600-second allowance per attempt. The previous-preparation control retains declared-sector initialization and uses native equation choices in the same executable. It disables the two new exact changes. Thus the overall factor includes deterministic preprocessing as well as any subsequent benefit from its selected settings.

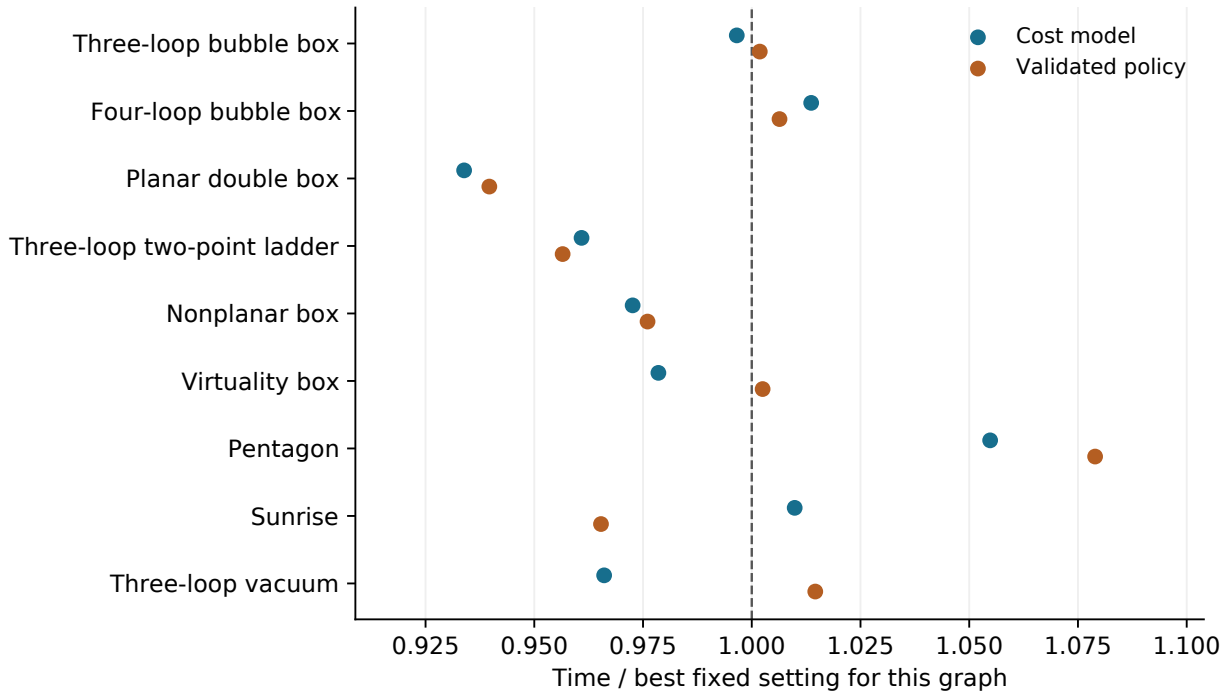


Figure 28: Added effect of target-dependent choices relative to the best fixed validation setting for each graph. Each point divides summed batch medians in the matched comparison. The validated policy uses the model only in the graph structures selected by its validation rule. Timing differences also remain when two methods choose the same setting.

The final change from projected preparation includes the validated policy and its graph-specific fixed settings; Figure 29 keeps these contributions separate.

The 4-loop previous control does not complete within its bounded attempts, after 1190.6 seconds of recorded work. Its remaining 2 repetitions are omitted after this failure, while the other pre-specified settings are completed. It receives no median completion time or speedup factor.

Table 34: Matched preparation comparison for the connected bubble boxes. Times are medians of three complete reductions, in seconds. The final column divides previous-preparation time by validated-policy time.

Loops	Previous	File index	Projection	Validated policy	Overall factor
3	6.087	5.525	2.906	2.939	2.07
4	Incomplete	98.408	28.197	28.666	–

The new policy makes one inexpensive decision before the costly work begins and retains native row elimination. Its tests cover stated target patterns and kinematics in known families. They do not establish a gain for every integral type, a minimal master basis, or a globally fastest reduction sequence. Reference failures and bounded higher-loop attempts remain part of the evidence. These limits determine where the current measurements support use of the method.

X A single policy for complete reduction cost

The new policy selects a calculation before Kira generates its equations. This appendix specifies its inputs, training objective, recovery rule, and exact preprocessing changes. The measured results appear in Section W.

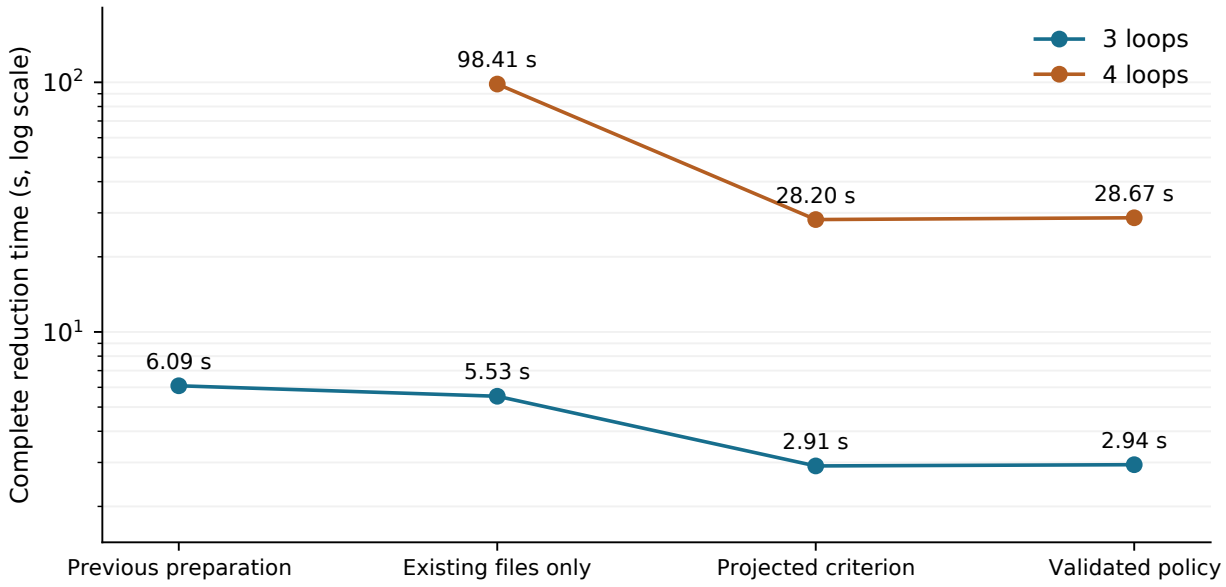


Figure 29: Complete time for the matched preparation study. The logarithmic vertical axis displays the large deterministic change and the final change from the validated policy. The separate graph-specific comparison measures the added effect of prediction. All plotted points represent completed reductions into the same prescribed basis with exact coefficient checks. Incomplete controls are omitted from the curve and reported in the table.

X.1 Inputs and allowed calculations

For a target set $\mathcal{T}$, define

$$r_{\mathcal{T}} = \max_{\boldsymbol{a}\in\mathcal{T}} \sum_i \max(a_i, 0), \qquad s_{\mathcal{T}} = \max_{\boldsymbol{a}\in\mathcal{T}} \sum_i \max(-a_i, 0), \qquad d_{\mathcal{T}} = \max_{\boldsymbol{a}\in\mathcal{T}} \sum_i \max(a_i - 1, 0). \quad (89)$$

These definitions give the largest sum of positive indices, numerator rank, and number of extra denominator powers among the targets. The last quantity counts the powers above one, often called dots. The policy chooses from seven calculations. Four use integral ordering 1 and seed limits $(r_{\mathcal{T}}, s_{\mathcal{T}}, d_{\mathcal{T}})$, $(r_{\mathcal{T}} - 1, s_{\mathcal{T}} - 1, d_{\mathcal{T}} - 1)$, $(r_{\mathcal{T}} - 1, s_{\mathcal{T}}, d_{\mathcal{T}} - 1)$, or $(r_{\mathcal{T}}, s_{\mathcal{T}} - 1, d_{\mathcal{T}})$. Negative limits are replaced by zero, and the positive-index limit is at least the number of positive indices in the largest declared top sector. A fifth calculation uses the first set of limits with Kira ordering 5. A sixth applies the previously fitted equation filter with the first set of limits. The seventh calculation starts directly with each native limit increased by one. The equation-filter parameters remain fixed during this study.

The input vector contains 18 quantities obtained from the family and targets. They describe the number of loops, external momenta, inverse propagators, physical propagators, and symbolic invariants; the natural logarithm of one plus the target count; maximum and mean dots and numerator ranks; minimum and maximum positive-index counts; target-sector count; massive-denominator count; and three measures of loop incidence in physical denominators. The final input is the mean number of external momenta present in a denominator momentum. A loop occurs in a denominator when its momentum appears in that denominator's momentum expression. The incidence measures count denominators involving several loops, the largest number of loops in one denominator, and the largest number of denominators containing one loop. Target coefficients and reference reductions are absent from these inputs.

X.2 Physical inputs and target construction

The seven earlier graph structures retain the ordered denominator momenta and scalar-product conventions of Appendix N. In a common squared-momentum unit, the sunrise has squared

masses $(3, 7, 13)$ and the vacuum has squared masses $(1, 1, 1, 13, 13, 53 + 2h)$. The two-point ladder has squared masses $(1, 0, 0, 0, 0, 0, 0, 0)$ on its physical denominators. The planar double box, nonplanar box, pentagon, and virtuality box have massless internal denominators. The parameter assignments are

$$\begin{aligned} p^2_{\text{sunrise}} = p^2_{\text{ladder}} &= -1009 - 12h, \\ (s,t)_{\text{DB}} &= (-1009 - 12h, -1021 - 14h), \\ (s,t)_{\text{NP}} &= (-113 - 18h, -127 - 10h), \\ (s_{12}, s_{23}, s_{34}, s_{45}, s_{51})_{\text{P}} &= (-11 - 20h, -13 - 20h, -17 - 20h, -19 - 20h, -23 - 20h), \\ (s,t,v)_{\text{box}} &= (-113 - 18h, -127 - 10h, -149 - 12h). \end{aligned} \tag{90}$$

Here h is the integer profile number, v is the external virtuality of the box, and the remaining invariants have their family-specific definitions in Appendix N. These assignments leave the dimension symbolic. The connected bubble boxes instead use Eq. (84) and its rational profile formula.

The seven earlier graph structures use profiles $h = 301, 302$ for training, $h = 303$ for validation, and $h = 304$ for testing. Each profile supplies three target patterns: two extra powers on a denominator; a dot with an auxiliary numerator, or two dots when no auxiliary entry exists; and a removed denominator with a dot on an adjacent denominator. Each batch has up to three targets, with positions shifted by the profile. The three- and four-loop bubble boxes of Eq. (86) use profiles $301, 303, 304$ and the full-sector, one-dot, and one-numerator targets. The symbolic nonplanar box, pentagon, and external-virtuality box use one target per split, with power three on the first, second, or third physical denominator. Their target lists are disjoint. Thus the tests measure new kinematics and targets within known graph structures.

The following rule specifies the rational target lists. Let n be the number of physical denominators, let N be the total number of family entries, and let $\boldsymbol{a}^{(0)}$ contain n ones followed by $N - n$ zeros. For $j = 0, 1, 2$, set $i_j = 1 + [(h + j) \bmod n]$ and $i_j^+ = 1 + (i_j \bmod n)$. With $\boldsymbol{e}_i$ the unit index vector, the dotted and pinched targets have indices

$$\boldsymbol{a}_j^{\text{dots}} = \boldsymbol{a}^{(0)} + 2\boldsymbol{e}_{i_j}, \qquad \boldsymbol{a}_j^{\text{pinched}} = \boldsymbol{a}^{(0)} - \boldsymbol{e}_{i_j} + \boldsymbol{e}_{i_j^+}. \tag{91}$$

For the mixed targets with $N > n$, set $k_j = n + 1 + [(h + j) \bmod (N - n)]$ and use $\boldsymbol{a}^{(0)} + \boldsymbol{e}_{i_j} - \boldsymbol{e}_{k_j}$. When $N = n$, use $\boldsymbol{a}^{(0)} + \boldsymbol{e}_{i_j} + \boldsymbol{e}_{i_j^+}$. Repeated vectors are kept once. The target patterns are shared across the rational splits, and some index sets recur; their kinematic profiles remain separate. The comparison therefore does not test arbitrary unseen index patterns.

The common basis for each batch comes from a native reduction with seed limits $(r_{\mathcal{T}} + 1, s_{\mathcal{T}} + 1, d_{\mathcal{T}} + 1)$. For a reserved batch, a second native reduction uses $(r_{\mathcal{T}} + 1, s_{\mathcal{T}} + 2, d_{\mathcal{T}} + 1)$ and must reproduce the reference coefficients in the same basis. Both calculations keep the positive-index floor specified above. A failed reference or enlarged check excludes that batch from speedup estimates. The conditional four-loop confirmation, when needed, uses the separately stated pair of nested regions.

X.3 Training from measured cost

Let T_{ca} be the measured complete time for training case c and initial choice a, including every recovery attempt. Let $a = 0$ denote native equation choices with the unshifted target limits. We fit the log time ratio

$$y_{ca} = \log \frac{T_{ca}}{T_{c0}}, \qquad \mathcal{L} = \sum_c \sqrt{T_{c0}} \sum_a \left[f_a(\boldsymbol{x}_c) - y_{ca} \right]^2 . \tag{92}$$

Here $\boldsymbol{x}_c$ is the input vector and f_a predicts the cost of choice a relative to the native choice. The square-root weight gives longer calculations more influence without making the training loss proportional to their full duration. This weighting is a design choice of this study.

We compare a regression tree, a random forest, and an ensemble of extremely randomized trees [9, 10, 11]. Each ensemble has 48 trees. The candidate depths are 3, 5, and 8, and each leaf requires at least two training cases. Every tree predicts all seven costs. For an ensemble with B trees, the prediction is the arithmetic mean

$$f_a(\boldsymbol{x}) = \frac{1}{B}\sum_{b=1}^{B} f_{ba}(\boldsymbol{x}), \tag{93}$$

where f_{ba} is tree b's prediction for choice a. The production evaluator follows the fitted tree splits directly, and its predictions are checked against the training library before validation.

For any fixed case, $T_{c0} > 0$ is independent of the selected choice. Since the logarithm is strictly increasing,

$$\operatorname*{argmin}_a T_{ca} = \operatorname*{argmin}_a \frac{T_{ca}}{T_{c0}} = \operatorname*{argmin}_a y_{ca}. \tag{94}$$

This identity explains the use of relative log time as the training target; it does not make a fitted prediction exact. Validation selects the ensemble and a gate $g \in \{0, 0.05, 0.15\}$. The predicted best choice is used only when its predicted log cost is at least g below that of the best fixed validation choice. Otherwise the fixed choice is used. Equivalently, the required predicted time ratio is at most e^{-g}. Selection minimizes the sum of validation batch median times, and the model then remains fixed during testing. The fixed comparison uses the same validation-selected setting for every test case.

If the selected calculation cannot reduce the targets into the supplied master basis, the driver tries native choices and then increases each native seed limit by one. The original policy removes repeated action names; different names can still produce equal effective bounds. The later portfolio also removes these equivalent attempts. If the last attempt fails, the request is recorded as unfinished. In timed tests, exact rational-function comparison checks every returned coefficient against the reference. Additional applications withhold the reference coefficients until the selected reduction has finished. Reference construction and training have separate cost records. Initial loading of the caller and model occurs outside the repeated timer, so these measurements do not describe the latency of starting a new application. The comparisons concern repeated reductions with an available basis, rather than the full cost of discovering that basis for a new problem.

X.4 Comparison with a fixed setting for each graph

Let $\mathcal{V}_H$ be the validation cases for graph structure H, and let $\widetilde{T}_{ca}$ be the median of the two validation times for case c and initial choice a. The fixed setting for that graph is

$$a_H = \operatorname*{argmin}_a \sum_{c \in \mathcal{V}_H} \widetilde{T}_{ca}. \tag{95}$$

This definition chooses the least measured validation cost subject to using one setting for the whole graph. It supplies a stronger fixed control than a setting shared by all graphs.

For each graph with at least four training cases, we also fit three regression trees with maximum depths one, two, and three and a minimum of two training cases per leaf. Two additional inputs describe how extra denominator powers are distributed. For case c with target set $\mathcal{T}_c$, define

$$A_c = \max_{\boldsymbol{a} \in \mathcal{T}_c} \max_i a_i, \qquad B_c = \frac{1}{|\mathcal{T}_c|} \sum_{\boldsymbol{a} \in \mathcal{T}_c} \sum_i \mathbf{1}_{a_i > 1}, \qquad \boldsymbol{z}_c = (\boldsymbol{x}_c, A_c, B_c). \tag{96}$$

The indicator $\mathbf{1}_{a_i>1}$ equals one when the propagator power exceeds one and zero otherwise. Thus A_c is the largest individual propagator power and B_c is the mean number of dotted propagators

per target. The patterns $(3,1,\ldots)$ and $(2,2,\ldots)$ both contain two dots, while these inputs record their different distributions. The local trees use Eq. (92) with the 20-component vector $\boldsymbol{z}_c$ and the same labels and weights, restricted to that graph's training cases. The global model continues to use the first 18 components.

Validation compares these local trees with the selected global model and considers gates $0, 0.05, 0.15$, now measured against a_H. It selects the predictor and gate with the least summed batch median cost. The graph-specific trees use no new reduction labels and remain fixed during the comparison.

Write $\pi_H(\boldsymbol{z}_c)$ for the resulting prediction in graph H. The deployed policy uses π_H only when

$$\sum_{c\in\mathcal{V}_H} \widetilde{T}_{c,\pi_H(\boldsymbol{z}_c)} \le 0.98 \sum_{c\in\mathcal{V}_H} \widetilde{T}_{c,a_H}, \tag{97}$$

and the predicted-choice total is strictly smaller in each of the two validation repetitions. Otherwise it uses a_H. The factor 0.98 is a chosen two-percent margin. The local-model fitting and selection procedure was specified before any reserved timings were collected. This rule compares measured validation cost; it is not a probability that a future request will be faster. It preserves the global model's weights and the exact recovery sequence.

Three rotated repetitions compare graph-specific fixed settings, the frozen cost model, and this validated policy on each reference-checked reserved batch. These are separate matched measurements from the native and global-fixed comparison. The public interface also checks the denominator structure and target-feature ranges against the training definitions. Unknown structures or values outside those ranges retain native choices. These checks restrict deployment to the recorded domain without asserting that the model generalizes beyond it.

Validation selects from the measured candidate-reduction costs. The reserved comparisons additionally time the actual feature construction, prediction, and deployment checks. They therefore measure any overhead that the cost-based validation selection omits.

X.5 An exact simplification of sector initialization

Kira's zero-sector test uses the parametric polynomials U and F [18, 5]. A sector is zero when its integrals vanish in dimensional regularization. The present change simplifies the input polynomial of the existing test within the declared sector domain. It preserves the full polynomials used by other Kira operations.

Let $\mathcal{A}$ contain all propagator indices present in at least one declared top sector, and let $R_{\mathcal{A}}$ set the Feynman parameter b_j to zero for every $j \notin \mathcal{A}$. For a polynomial $P(\boldsymbol{b})$, write the weighted Euler expression used in the criterion as

$$C_{\boldsymbol{\kappa}}[P] = \sum_j \kappa_j b_j \frac{\partial P}{\partial b_j} - P. \tag{98}$$

The unknown constants κ_j are the coefficients determined by the sector test. Apply $R_{\mathcal{A}}$ to each term. If $j \notin \mathcal{A}$, the explicit factor b_j makes the term zero. If $j \in \mathcal{A}$, substitution in the other variables commutes with differentiation in b_j. Therefore

$$\begin{aligned} R_{\mathcal{A}} C_{\boldsymbol{\kappa}}[P] &= \sum_{j\in\mathcal{A}} \kappa_j b_j R_{\mathcal{A}} \frac{\partial P}{\partial b_j} - R_{\mathcal{A}} P \\ &= \sum_{j\in\mathcal{A}} \kappa_j b_j \frac{\partial (R_{\mathcal{A}} P)}{\partial b_j} - R_{\mathcal{A}} P. \end{aligned} \tag{99}$$

For every admitted sector $S \subseteq \mathcal{A}$, its further substitution satisfies $R_S R_{\mathcal{A}} = R_S$. Constructing the criterion from $R_{\mathcal{A}} U$ and $R_{\mathcal{A}} F$ therefore gives the same polynomial constraints as constructing

the full criterion and then restricting it to S. The implementation applies this simplification only together with declared-sector initialization, without cut propagators or symbolic propagator powers. The derivation concerns the zero-sector criterion; it does not remove auxiliary numerator factors from the integrals.

A second exact change enumerates the sector files that exist before triangular reduction and back-substitution. It reads those files in the original sector order with the original loading functions. Missing files contain no equations, so removing their repeated existence checks leaves the loaded system unchanged. Both preprocessing changes are deterministic. Their timing benefit is measured separately from the policy's choice of equations and seed limits.

X.6 Conditional four-loop reference confirmation

The generic enlarged-reference check does not complete for the original reserved four-loop bubble box. A rule registered before model fitting therefore adds profile $h = 305$, with reference seed limits $(r, s, d_{\mathrm{dots}}) = (12, 1, 2)$ and check limits $(12, 2, 2)$. Here d_{dots} is the dot limit, distinct from the spacetime dimension. The development study had already verified this pair of nested regions. The new profile uses the same frozen model and the same three timing repetitions for native, fixed, and learned choices. It supplies a separate bounded confirmation; the unfinished original profile is retained in the failure count.

Y Complete-cost portfolio: definitions and derivation

Y.1 Targets and seed choices

For a batch $\mathcal{T}$ of index vectors $\mathbf{a}$, define

$$r_0 = \max_{\mathbf{a}\in\mathcal{T}} \sum_i \max(a_i, 0), \qquad s_0 = \max_{\mathbf{a}\in\mathcal{T}} \sum_i \max(-a_i, 0), \qquad d_0 = \max_{\mathbf{a}\in\mathcal{T}} \sum_i \max(a_i - 1, 0). \quad (100)$$

These definitions count positive powers, numerator rank, and dots separately. Selective expansion uses $(r_0, s_0 + 1, d_0)$, with the positive-index limit floored at the largest declared top-sector size. The full expansion uses $(r_0 + 1, s_0 + 1, d_0 + 1)$. The five portfolio choices combine the earlier validated policy or selective expansion with one or two workers, and retain the previous workflow as a fifth choice. A failed new-policy attempt proceeds to native seeds and then full expansion. Skipping identical effective configurations removes a repeated calculation with the same equations, ordering, basis, and filter; it introduces no new algebraic relation.

The training and validation cases are the 47 and 26 cases defined in Appendix X. The reserved rational cases use the same family definitions at $h = 306$ and the target constructions with $j = 0, 1, 2$. The three symbolic graphs instead use the corner target with $a_4 = 3$, so the fourth physical denominator has power three. For the four-loop bubble box, the reference uses $(r, s, d_{\mathrm{dots}}) = (12, 1, 2)$ and its independent check uses $(12, 2, 2)$. All other reserved references use full expansion and then raise its numerator-rank bound by one more unit. These limits were fixed before any new reserved calculation. A common supplied basis permits exact coefficient comparisons; these checks do not prove its minimality.

Y.2 Learning relative complete cost

Let $T_{ba} > 0$ denote the measured complete time for training batch b and portfolio choice a. Let a_0 denote the earlier cost policy with one worker. We define

$$y_{ba} = \log \frac{T_{ba}}{T_{ba_0}}. \quad (101)$$

For the same batch, subtracting two labels gives $y_{ba} - y_{ba'} = \log(T_{ba}/T_{ba'})$. Since the logarithm is increasing, the choice with the smallest exact label also has the smallest measured time. A regression model f estimates the five labels from the twenty features x_b given in Appendix X. Its fitted objective is

$$\mathcal{L}(f) = \sum_{b \in \mathcal{B}_{\text{train}}} \sqrt{T_{ba_0}} \sum_{a} \big[f_a(x_b) - y_{ba}\big]^2. \tag{102}$$

The positive weight increases the influence of costly training batches without changing their preferred action. It is a chosen training objective, not a proof that the fitted model minimizes future wall time. Each label includes native recovery, so an unsuccessful first attempt carries its measured cost.

We fit nine global models: a regression tree, a 48-tree random forest, and a 48-tree extremely randomized ensemble at depths 3, 5, and 8. Each leaf has at least two training cases. Graphs with at least four training cases also receive three local regression trees at depths 1, 2, and 3. Tree methods follow the cited constructions [9, 10, 11]. The saved evaluator reproduces the fitted predictions before validation begins.

Y.3 Validation and the deployment guard

For graph g, let c_g be the fixed portfolio choice with the smallest sum of validation batch median times. If $a_\star = \arg\min_a f_a(x)$, a predictor with gate γ proposes $a_\star$ only when

$$f_{a_\star}(x) < f_{c_g}(x) - \gamma, \qquad \gamma \in \{0, 0.05, 0.15\}. \tag{103}$$

Otherwise it uses c_g. Under exact labels, exponentiation gives $T_{ba_\star}/T_{bc_g} < e^{-\gamma}$; with fitted labels this is a predicted saving, not a guarantee. Validation selects the predictor and gate for each graph. Deployment enables this outer prediction only if its summed validation median time is at most 0.98 times the fixed-control time and it wins both validation repetitions. The alternative is c_g. Graph structure and feature-range checks send unsupported inputs to native settings. A fixed portfolio choice can contain the earlier policy; the outer guard evaluates only the added decision.

Y.4 Correctness and timing

The input to each reduction is the family, target batch, and prescribed master basis. Kira generates the exact identities and returns symbolic coefficients in that basis. The test then checks each coefficient by exact rational-function subtraction against the independently checked reference. The caller does not provide those reference coefficients to the deployed predictor or to the application. Thus the learned choice affects which calculation is attempted, while the algebraic identities determine the returned relation. Failure to reach the supplied basis triggers recovery; exhausting the bounded attempts returns failure.

For method m, the reported aggregate is $\sum_b \text{median}_{r=1,2,3}\, T_{bmr}$. The speedup relative to control c is the ratio of its aggregate to the selected aggregate. This follows by dividing the times required for the same collection of batches. We also report totals over all repetitions and process CPU times. These summaries describe the measured collection; they are not statistical confidence intervals or guarantees for unseen integral families. All methods use the same exact executable and two-core allocation. The earlier preparation study uses a separate one-core allocation, so its speedup factors are not multiplied by these portfolio ratios.

Z Exact restrictions and selected-equation reuse

Z.1 Restricting before constructing the polynomials

Let b_i be the Schwinger parameter associated with inverse propagator D_i. For algebraic preparation, suppress its infinitesimal prescription and write the quadratic form with the same Minkowski sign convention as the integral definition:

$$-\sum_i b_i D_i = -\boldsymbol{\ell}^{\mathsf{T}} H(b)\boldsymbol{\ell} + 2w(b)^{\mathsf{T}}\boldsymbol{\ell} + J(b). \tag{104}$$

Here H is a symmetric $L \times L$ matrix of scalar coefficients, w is a vector whose entries are external-momentum combinations, and J contains no loop momentum. Lorentz contractions are implicit in the quadratic form. Completing the square gives

$$-(\boldsymbol{\ell} - H^{-1}w)^{\mathsf{T}} H(\boldsymbol{\ell} - H^{-1}w) + J + w^{\mathsf{T}}H^{-1}w. \tag{105}$$

The standard parametric construction [23], with an inessential normalization scale suppressed, therefore uses the polynomials

$$U = \det H, \qquad F = J \det H + w^{\mathsf{T}} \operatorname{adj}(H) w, \tag{106}$$

where $\operatorname{adj}(H)$ is the adjugate matrix. The inverse serves only in the derivation; Eq. (106) remains a polynomial identity when a substituted matrix is singular.

Let B contain the propagator positions that occur in at least one declared top sector. Define R_B by setting $b_i = 0$ for $i \notin B$. Substitution preserves addition and multiplication. Expanding the determinant over permutations π gives

$$R_B(\det H) = R_B\left(\sum_\pi \operatorname{sgn}(\pi) \prod_{j=1}^{L} H_{j,\pi(j)}\right) \tag{107}$$

$$= \sum_\pi \operatorname{sgn}(\pi) \prod_{j=1}^{L} R_B(H_{j,\pi(j)}) = \det(R_B H). \tag{108}$$

Each entry of the adjugate is a signed minor, so the same argument gives $R_B \operatorname{adj}(H) = \operatorname{adj}(R_B H)$. Applying these equalities to Eq. (106) shows that constructing U, F after the substitution produces exactly $R_B U, R_B F$.

Every admitted sector S is a subset of B. Setting the remaining absent parameters to zero then gives

$$R_S R_B P = R_S P \tag{109}$$

for any polynomial P. The sector polynomials used in zero-sector and symmetry checks are consequently identical to those obtained from the full construction. The Euler criterion also agrees because substitution commutes with differentiation in every retained variable, as derived in Appendix X. This proof concerns sector polynomials. It does not set an auxiliary inverse propagator to zero in an integral numerator. The implementation retains those propagators in all IBP and numerator operations. It enables the restriction only for declared-sector calculations without cuts or symbolic propagator powers.

Z.2 Reusing an unchanged set of exact equations

For one generated system, let $Q = (q_1, \ldots, q_k)$ be its ordered list of selected source-equation numbers and let $E_q = \sum_i c_{qi} I_i$ be the corresponding unmodified symbolic row. The first retrieval supplies $E_{q_1}, \ldots, E_{q_k}$ to the basis check. If the subsequent selection returns $Q' = Q$, then

$$(E_{q'_1}, \ldots, E_{q'_k}) = (E_{q_1}, \ldots, E_{q_k}). \tag{110}$$

Passing the saved rows to the final reduction therefore supplies the same exact identities. Copies of the original rows preserve them when the basis check sorts or alters its working rows. We require equality of the full ordered source list, keep the same system and coefficient treatment, and use ordinary regeneration when that condition fails. Storage allows at most 4,096 rows and 4 MiB of accounted row payload, plus bounded container overhead. A development check forces a one-byte allowance and confirms that ordinary regeneration returns the same coefficients. This is within-calculation reuse, with no transfer of coefficients between target requests.

Z.3 Reserved inputs and comparison definitions

The rational families and target construction follow Appendix X with $h = 307$ and $j = 0, 1, 2$. The planar double box and connected bubble boxes use $s = -4693$ and $t = -5319$, and the two-point families use $p^2 = -4693$. The nonplanar and virtuality boxes use $s = -5639$ and $t = -3197$, with virtuality $v = -3833$ for the latter. The pentagon invariants $(s_{12}, s_{23}, s_{34}, s_{45}, s_{51})$ are $(-6151, -6153, -6157, -6159, -6163)$. The three-loop vacuum uses the squared masses stated there with the last value $53 + 2h = 667$. Graph-specific momentum conventions remain those of Appendix N; the connected bubble boxes retain Appendix V's convention. Each dots, mixed, or pinched batch contains up to three distinct indexed targets. The symbolic nonplanar and pentagon targets have power three on the fifth physical denominator. The symbolic virtuality box has power two on its first and fourth denominators. Other physical powers are one and other auxiliary powers are zero.

References use $(r_0 + 1, s_0 + 1, d_0 + 1)$, followed by an independent check with numerator-rank limit increased by one more unit. The four-loop bubble reference instead uses $(12, 1, 2)$ and $(12, 2, 2)$, fixed before testing. The first calculation uses the new exact preparation, and the second uses the previous executable. The model files and decision rules remain frozen throughout the comparison. The definitions of complete-time aggregation and relative speedup are those in Appendix Y. All repetitions and individual batch results remain available; no confidence interval is inferred from three runs.

For the common-dispatch sensitivity calculation, let D_{bcr} be the measured input-check and outer-proposal interval for batch b, control c, and repetition r. We subtract that interval before taking the median:

$$T_c^{\text{optimistic}} = \sum_b \text{median}_{r=1,2,3}(T_{bcr} - D_{bcr}). \tag{111}$$

The unused proposal occupies only part of D_{bcr}. Holding the remaining measured work fixed, removing all of D_{bcr} therefore overstates the saving available from removing that proposal alone. This is a sensitivity calculation on the recorded timings, not a prediction of how a different executable would interact with the operating system.

AA Definitions and timing for the coverage grid

The new grid uses the integral convention in Eq. (1); every denominator includes the same implicit Feynman prescription. Let $Q = \sum_{i=1}^{L} \ell_i$. For $L \geq 2$, the vacuum families have physical denominators

$$D_i = \ell_i^2 - m_i^2 \quad (1 \leq i \leq L), \qquad D_{L+1} = Q^2 - m_{L+1}^2. \tag{112}$$

The one-mass case sets $m_1^2 = \cdots = m_L^2 = 0$ and $m_{L+1}^2 = 835$. The fully massive case sets $m_i^2 = 837 + 6(i - 1)$ for each physical position. At one loop we instead use the single massive tadpole denominator $D_1 = \ell_1^2 - 835$, since the two putative vacuum denominators would have dependent scalar-product coefficients. These numbers use a common squared-momentum unit.

The two-point families replace the final vacuum denominator by

$$D_{L+1} = (p - Q)^2 - m_{L+1}^2, \qquad p^2 = -847. \tag{113}$$

The massless case sets every mass to zero; the massive case uses the same distinct squared masses as the fully massive vacuum. The four-point ladders and boxes with bubble subgraphs use Eqs. (85) and (86), with $s = -5917$ and $t = -6747$. The main grid uses profile $h = 409$. It defines these families through twelve loops before applying the current support limit. The two coincident one-loop definitions are included only once.

For E independent external momenta, the scalar products consist of L loop squares, $L(L-1)/2$ products of distinct loop momenta, and LE loop–external products. Their total is therefore

$$N = L + \frac{L(L-1)}{2} + LE = \frac{L(L+1)}{2} + LE. \tag{114}$$

This is the complete representation required for native IBP generation [5]. We append candidate squared momentum differences only when their exact coefficient vector increases the rank of the physical denominator list. The resulting $N \times N$ coefficient matrix has rank N for every declared family. This construction defines numerator coordinates even for cases that cannot enter the current application. In this grid the 31-index limit permits vacuum definitions through seven loops, two-point definitions through six loops, and four-point definitions through five loops. Permitted input size does not imply completion within the common limits.

Let n be the number of physical denominators and let $\boldsymbol{a}^{(0)}$ have entries one on these denominators and zero on the auxiliary ones. The target batch is

$$\mathcal{T} = \{\boldsymbol{a}^{(0)},\ \boldsymbol{a}^{(0)} + \boldsymbol{e}_1\} \cup \{\boldsymbol{a}^{(0)} - \boldsymbol{e}_{n+1} \text{ if } N > n\}, \tag{115}$$

where $\boldsymbol{e}_i$ has unit entry at position i. The full comparison and first reference use positive-index bound $r = n + 1$, numerator bound $s = 1$, and dot bound one. The independent reference check raises the numerator bound to two. The full comparisons receive no preferred master list. The application comparisons receive the checked basis and retain their frozen learned choices, domain guards, and expansion recovery.

The symbolic additions are the one-loop tadpole, the one-loop massless two-point family, massive two-point families at one and two loops, and four-point ladders at one and two loops. Their nonzero squared masses and external invariants remain independent symbols. The nonplanar box, pentagon, and virtuality-box definitions in Appendix N add rational and symbolic requests. Their rational profile is $h = 411$ in the previously defined parameter formulas. Each of these three structures uses the corner target and one extra power on its last physical denominator. These finite cases do not span arbitrary symbolic kinematics.

The additional massive two-point batches use rational profile $h = 419$, so $p^2 = -867$ and $m_i^2 = 857 + 6(i-1)$. Their symbolic variants retain independent masses and p^2. The target set includes $\boldsymbol{a}^{(0)}$, every $\boldsymbol{a}^{(0)} + \boldsymbol{e}_i$, every $\boldsymbol{a}^{(0)} + 2\boldsymbol{e}_i$, and every $\boldsymbol{a}^{(0)} + \boldsymbol{e}_i + \boldsymbol{e}_j$ with $1 \le i < j \le n$. It contains $1 + 2n + n(n-1)/2$ targets. The last count follows from choosing an unordered pair of distinct physical denominators. The expanded region uses $r = n + 2$, numerator bound one, and dot bound two; the independent check raises the numerator bound to two. These batches supply a separate test of nontrivial reductions after the initial small requests, without replacing the original outcomes.

For the separate restricted vacuum study, the squared-mass ratios are the first R entries of $(823, 829, 853, 857)$, with $R = 2, 3, 4$. We choose six distinct exponent vectors $\boldsymbol{b}$ with $\sum_j b_j = 6$ and at least two positive entries. The vectors are fixed before the comparisons. Each request uses either one loop order or all twelve orders together. The exact recurrence and the Taylor check use the derivations in Appendix L; this study does not infer those identities from a high-loop native reduction. The original Kira control generates and solves the full supplied identity system separately at each loop order. Other identity-based controls and the recurrence can substitute $z = L(d-2)$ after their shared calculation.

For method m, request c, and repetition j, let T_{mcj} denote measured complete time. The

Table 35: The six target exponent vectors for each restricted vacuum profile.

R	Target vectors $\boldsymbol{b}$
2	$(5,1),\ (1,5),\ (4,2)$
	$(2,4),\ (3,3)$
3	$(2,2,2),\ (3,2,1),\ (0,1,5)$
	$(1,1,4),\ (0,2,4),\ (1,5,0)$
4	$(1,2,3,0),\ (0,2,2,2),\ (1,0,2,3)$
	$(3,0,1,2),\ (0,2,3,1),\ (4,1,1,0)$

matched total and reported ratio are

$$S_m(\mathcal{C}) = \sum_{c\in\mathcal{C}} \mathrm{median}_{j=1,2,3}\, T_{mcj}, \qquad R_{A/B} = \frac{S_A(\mathcal{C})}{S_B(\mathcal{C})}. \tag{116}$$

The set $\mathcal{C}$ contains only requests with three exact completions for both compared methods. It is specified separately for each pair; no failed case receives an imputed completed time. The fractional time saving follows directly by subtracting the revised total from the control total and dividing by the control total:

$$\frac{S_B - S_A}{S_B} = 1 - \frac{S_A}{S_B} = 1 - R_{A/B}. \tag{117}$$

The coverage table retains every excluded case. This separates a conditional timing gain from the probability that a request completes. Three timing repetitions supply the stated medians, not a confidence interval or a proof of global optimality.

The supplied-identity Kira workers use a 90-second solver limit, a 1,800 MiB address-space limit, a 1,500 MiB process-tree memory limit, and a 64 MiB temporary-file limit. The recurrence works within the same scheduler allocation and checks the expanded coefficients independently. Native and vacuum calculations run sequentially, so their temporary storage does not overlap.

AB Learning seed bounds: inputs, objective, and timing definitions

AB.1 Integral definitions and target batches

The new study uses the family definition of Eq. (1). The sunrise, planar double box, three-loop vacuum, and massive three-loop two-point ladder retain the ordered denominator momenta in Appendix N, with the masses and kinematic assignments in Eq. (90). The nonplanar box, pentagon, and external-virtuality box use that equation's corresponding assignments and the denominator definitions in the same appendix. The three- and four-loop boxes with bubbles use Eq. (86). The four-loop massless two-point family uses $D_i = \ell_i^2$ for $i = 1,\dots,4$ and $D_5 = (p-\ell_1-\ell_2-\ell_3-\ell_4)^2$, with $p^2 = -2h-29$. Auxiliary squared momentum differences complete its scalar-product coordinates as in Appendix AA. The training profiles are $h = 511, 512$, validation uses $h = 513$, and the reserved test uses $h = 515, 516$. Development uses $h = 501$ and supplies no reserved labels.

Write n for the number of physical denominators, N for the total number of family entries, and $\boldsymbol{a}^{(0)} = (1,\dots,1,0,\dots,0)$ with n leading ones. For $j = 0,1,2$, define $i_j = 1 + [(h+j) \bmod n]$ and $i_j^+ = 1 + (i_j \bmod n)$. When $N > n$, define $k_j = n + 1 + [(h+j) \bmod (N-n)]$. The first seven structures use four target patterns:

$$\begin{aligned} \boldsymbol{a}_j^{\mathrm{dots}} &= \boldsymbol{a}^{(0)} + 2\boldsymbol{e}_{i_j}, & \boldsymbol{a}_j^{\mathrm{pinch}} &= \boldsymbol{a}^{(0)} - \boldsymbol{e}_{i_j} + \boldsymbol{e}_{i_j^+}, \\ \boldsymbol{a}_j^{\mathrm{mixed}} &= \boldsymbol{a}^{(0)} + \boldsymbol{e}_{i_j} - \boldsymbol{e}_{k_j}, & \boldsymbol{a}_j^{\mathrm{rank2}} &= \boldsymbol{a}^{(0)} - 2\boldsymbol{e}_{k_j}. \end{aligned} \tag{118}$$

Here $\boldsymbol{e}_i$ is the unit index vector. For families without auxiliary entries, the last two patterns become $\boldsymbol{a}^{(0)} + \boldsymbol{e}_{i_j} + \boldsymbol{e}_{i_j^+}$ and $\boldsymbol{a}_j^{\text{dots}}$. The remaining three structures use the corner, $\boldsymbol{a}^{(0)} + \boldsymbol{e}_{i_j}$, and $\boldsymbol{a}^{(0)} - \boldsymbol{e}_{k_j}$. Repeated vectors within a batch appear once. These equations define the requested integrals directly, with no assumed optimal reduction path.

AB.2 The ten candidate configurations

Let (r_0, s_0, d_0) be the target maxima in Eq. (89), and let t_{top} be the largest number of positive indices in a declared top sector. Table 36 specifies the ten candidates as changes to these maxima. Negative bounds become zero, and the positive-index bound is at least t_{top}. The word "native" in the table denotes the unshifted target-bound configuration. Kira's lower sector restriction has

Table 36: Candidate seed settings. A dash means that the lower sector truncation is disabled.

Choice	Δr	Δs	Δd_0	l
Native bounds	0	0	0	—
Numerator expansion	0	1	0	—
Tight bounds	-1	-1	-1	—
Reduced dots	-1	0	-1	—
Reduced numerator rank	0	-1	0	—
Full expansion	1	1	1	—
Truncated native	0	0	0	$t_{\text{top}} - s_0$
Truncated numerator expansion	0	1	0	$t_{\text{top}} - s_0$
Truncated reduced dots	-1	0	-1	$t_{\text{top}} - s_0$
Relaxed truncation	0	0	0	$t_{\text{top}} - s_0 - 1$

the form [5]

$$s \leq \max\{1, t - l + 1, s_{\text{max,sector}}\}, \tag{119}$$

where t is the number of positive indices in that sector, l is the chosen truncation parameter, and $s_{\text{max,sector}}$ is an explicitly assigned sector limit. This bound supplements the candidate seed limits. Every unsuccessful candidate is followed by native bounds and full expansion, with repeated effective bound-and-truncation pairs removed. All attempts use integral ordering 1 and one worker.

AB.3 Features and fitted objective

The first eighteen features are those defined in Appendix X. The remaining nine are the minimum dots and numerator rank, mean number of positive indices, largest absolute index, mean dots on massive entries, mean numerator-weighted loop incidence, mean dot-weighted loop incidence, maximum sum of positive indices, and maximum number of absent physical denominators. All means run over the target batch. If $B_{i\ell} = 1$ when loop momentum ℓ appears in denominator momentum q_i and zero otherwise, the two weighted incidence quantities are

$$\frac{1}{|\mathcal{T}|} \sum_{\boldsymbol{a} \in \mathcal{T}} \sum_i \max(-a_i, 0) \sum_\ell B_{i\ell}, \qquad \frac{1}{|\mathcal{T}|} \sum_{\boldsymbol{a} \in \mathcal{T}} \sum_i \max(a_i - 1, 0) \sum_\ell B_{i\ell}. \tag{120}$$

These definitions describe index structure; they do not encode a target's reference reduction or measured runtime.

For a training case c and candidate a, let $T_{ca} > 0$ be its measured time with recovery, and let $a = 0$ denote native bounds. We fit $y_{ca} = \log(T_{ca}/T_{c0})$. Subtracting two such labels gives

$$y_{ca} - y_{cb} = \log T_{ca} - \log T_{cb} = \log(T_{ca}/T_{cb}). \tag{121}$$

The logarithm is increasing, so minimizing the exact label selects the least measured time for that case. This statement concerns the labels; prediction error can change the selected configuration. The tree models fit weighted squared errors with $w_c = \sqrt{T_{c0} / \operatorname{median}_{c'} T_{c'0}}$. The twelve single trees have maximum depths $2, 3, 5, 8$ and minimum leaf sizes $1, 2, 4$. Six ensembles combine random forests or extremely randomized trees with depths $3, 5, 8$, 48 trees, and minimum leaf size two [9, 10, 11].

The twelve neural candidates use hidden widths (16), (32), or $(32, 16)$, a rectified-linear or hyperbolic-tangent activation, and regularization parameter 0.01 or 0.1. They standardize features using training means and standard deviations, fit unweighted squared errors with a squared-weight penalty, and use L-BFGS with a 1,500-iteration limit [12]. Their hidden states satisfy $\boldsymbol{h}^{(j+1)} = \sigma(W_j^{\mathsf{T}} \boldsymbol{h}^{(j)} + \boldsymbol{b}_j)$, while the output layer is linear. This equation defines the implemented network. Validation compares actual candidate costs, so differences between the fitting objectives do not determine the final choice by themselves. Every stochastic fit uses random seed 41.

Let b_H be the fixed candidate with the smallest summed training-plus-validation time for graph H, and let $a_\star = \arg\min_a f_a(\boldsymbol{x}_c)$ be the predicted best candidate. A model with gate g uses $a_\star$ when

$$f_{a_\star}(\boldsymbol{x}_c) < f_{b_H}(\boldsymbol{x}_c) - g, \qquad g \in \{0, 0.03, 0.10, 0.20\}, \tag{122}$$

and otherwise uses b_H. Under exact labels, this inequality becomes $T_{ca_\star} / T_{cb_H} < e^{-g}$ by exponentiation. With fitted labels it expresses a predicted saving. Validation selects the model and gate with the lowest sum of recorded configuration times. No reserved result enters that choice. The saved evaluator is checked against the training library for every validation prediction before the model is frozen.

AB.4 Measured ratios, uncertainty, and training cost

Let $\widetilde{T}_{cm}$ be the median of five complete times for case c and method m. For a matched set $\mathcal{C}$, define

$$Q_m = \sum_{c \in \mathcal{C}} \widetilde{T}_{cm}, \qquad R_b = Q_{\mathrm{ML}} / Q_b, \qquad S_b = 1 / R_b. \tag{123}$$

Summing accounts for the time required by the same batch collection; division gives its relative time, and inversion gives its speedup factor. The percentage time saving is $100(1 - R_b)$. Matched cases must complete all methods and repetitions with a nontrivial reduction. We also retain the total elapsed time and completion counts for every timed request, including failures and identity-only outputs.

For the graph interval, we sample the observed graph groups with replacement and recompute R_b, preserving each selected graph's complete batch block. For the repetition interval, each batch samples five paired repetition positions with replacement and recomputes its medians and then R_b. Both calculations use 5,000 draws and report the 2.5th and 97.5th percentiles. These are descriptive resampling intervals for the specified collection. The graph count is small, and repeated index patterns remain correlated within a graph.

Let T_{off} be the recorded development, training, validation, and associated reference cost, including fitting. If the future mean saving equals the estimate from the matched workload $\Delta_b = (Q_b - Q_{\mathrm{ML}}) / |\mathcal{C}| > 0$, then N requests recover this cost when $N \Delta_b \geq T_{\mathrm{off}}$. Solving this inequality gives

$$N_{\mathrm{break}} = \lceil T_{\mathrm{off}} / \Delta_b \rceil . \tag{124}$$

This calculation excludes reserved-test expense and assumes that an appropriate master basis is already available. It states the cost of collecting the model's evidence as well as its application saving. It does not prove that future workloads will have the same mean saving.

AC Equation refinement: fitting and evaluation

AC.1 Inputs available before coefficient construction

Let $\boldsymbol{\sigma}$ be a seed with N indices, and let $\mathcal{T}$ be the target batch. Define the number of positive indices, the extra denominator powers, and the numerator rank by

$$n_+(\boldsymbol{\sigma}) = \sum_i \mathbf{1}[\sigma_i > 0], \qquad \delta(\boldsymbol{\sigma}) = \sum_i \max(\sigma_i - 1, 0), \qquad \rho(\boldsymbol{\sigma}) = \sum_i \max(-\sigma_i, 0). \tag{125}$$

The target maxima are $\delta_{\max} = \max_{\boldsymbol{a}\in\mathcal{T}} \delta(\boldsymbol{a})$ and $\rho_{\max} = \max_{\boldsymbol{a}\in\mathcal{T}} \rho(\boldsymbol{a})$. For the seed's sector S, let b_S be one if the batch contains a target in that sector and zero otherwise. Define δ_S, ρ_S as the corresponding maxima over targets in S, or zero when that sector has no target. The minimum target distance is

$$\Delta(\boldsymbol{\sigma}, \mathcal{T}) = \min_{\boldsymbol{a}\in\mathcal{T}} \sum_i |\sigma_i - a_i|. \tag{126}$$

These quantities depend only on indices. Table 37 specifies the complete input vector. The power differences are signed; a negative value means that the seed has fewer powers than the stated target maximum. Here L is the number of loops, E is the number of independent external momenta, and o is the operator index in Kira's candidate list.

Table 37: The seventeen inputs to the new equation classifiers. A whole-seed choice uses the same inputs with the operator index removed.

Group	Quantities	Number
Family and request	$L, E, N, \log(1 + \lvert\mathcal{T}\rvert)$	4
Seed powers	$n_+(\boldsymbol{\sigma}), \delta(\boldsymbol{\sigma}), \rho(\boldsymbol{\sigma})$	3
Target differences	$\delta(\boldsymbol{\sigma}) - \delta_{\max}$ and $\rho(\boldsymbol{\sigma}) - \rho_{\max}$	2
Sector information	b_S, $\delta(\boldsymbol{\sigma}) - \delta_S$, and $\rho(\boldsymbol{\sigma}) - \rho_S$	3
Target proximity	$\Delta(\boldsymbol{\sigma}, \mathcal{T})$ and $\mathbf{1}[\boldsymbol{\sigma} \in \mathcal{T}]$	2
Operator	Index o for an IBP or Lorentz-invariance identity	1
Overall limits	$\delta_{\max}, \rho_{\max}$	2

AC.2 Sampling and tree scores

The physical families, target patterns, and tuned seed configurations are those in Appendix AB. Training uses $h = 511, 512$, validation uses $h = 513$, and the new reserved test uses $h = 525, 526$. The earlier reserved profiles $515, 516$ supply no training or validation labels for this filter. Within a batch c, let N_{cy} be the number of native equation records with label $y \in \{0, 1\}$ and let $N_c = N_{c0} + N_{c1}$. We sample $n_{cy} = \min(2000, N_{cy})$ records without replacement from each nonempty class, using a deterministic batch-specific random seed. For fitting, every sampled record in that class receives weight

$$w_{cy} = \frac{N_{cy}}{n_{cy} N_c}. \tag{127}$$

For fitting, its class has total sampled weight $n_{cy} w_{cy} = N_{cy}/N_c$, so summing over classes gives unit weight per batch. This calculation explains both the correction for class sampling and the equal batch contribution. Validation recall and retained fractions instead use N_{cy}/n_{cy}, which reconstructs native record counts across the validation batches.

For a tree leaf ℓ, let $W_{\ell 1}$ and $W_{\ell 0}$ be the sums of these weights for positive and negative training records. Its score is

$$p_\ell = \frac{W_{\ell 1}}{W_{\ell 0} + W_{\ell 1}}. \tag{128}$$

This is the weighted positive fraction in the leaf. It does not assert calibrated probabilities for new equations. The tree chooses splits through weighted Gini impurity [9], using the classification-tree implementation of Ref. [12]. At a node, let W_0 and W_1 be the negative and positive total weights, $W = W_0 + W_1$, and $p = W_1/W$. The weighted impurity is

$$G = W[1 - p^2 - (1-p)^2] = 2Wp(1-p) = \frac{2W_0W_1}{W_0 + W_1}. \tag{129}$$

The equalities follow by expanding the two squares and substituting the weighted class fractions. For each allowed split, subtract the two child impurities from the parent impurity. The tree selects the largest decrease, subject to its depth, leaf-count, and minimum-population limits. The eight depth and leaf-size choices appear in the main text. We compare shared fits with separate fits for each graph before the complete-cost validation. All fits use random seed 611.

AC.3 Validation and frozen deployment

For each graph, candidate thresholds include 10^{-12}, 0.01, 0.05, 0.1, 0.25, and 0.5, together with the 0, 0.1, 0.5, 1, and 2 percentiles of positive validation scores. For each required weighted recall of 0.9999, 0.995, and 0.98, we choose the tree and threshold with the smallest retained fraction among candidates that meet that recall. Repeated choices appear once. These thresholds concern reference labels; they do not guarantee a complete reduction.

We run each shortlisted filter and the all-ML-disabled control three times for every validation batch. For each graph, we sum the per-batch median times and choose the cheapest candidate, retaining it only if it beats the control by at least 5%. This threshold for deployment is fixed before the reserved experiment. For the shortlisted source trees, we also calculate the maximum score over operators and select seed thresholds using the same recall levels and retained-fraction criterion. The seed threshold grid adds 0.75, 0.9, and 1 to the equation grid. We compare up to three seed variants with the equation filters in the same complete-cost validation. The selected filters are The model tests 17 features before symbolic coefficient construction. It retains

Table 38: Filters fixed by complete-time validation. The columns give the filter type, compiled node count, source-tree depth limit, minimum leaf population, and original score threshold.

Graph	Filter	Nodes	Depth limit	Leaf size	Threshold
Planar double box	Equation	1161	18	16	1e-12
Massive vacuum	Equation	369	14	4	1e-12
Massive three-loop ladder	Seed	60	14	16	0.02232
Nonplanar box	Seed	57	14	4	0.05

a candidate when $p_\ell \geq \tau$, where τ is the graph's threshold. If that attempt fails, the next attempt uses the tuned seed setting without the model. Native and fully expanded bounds follow if necessary, with identical configurations removed. Every attempt must express all mandatory targets in the prescribed basis. Independent exact coefficient comparisons then check the exported answer. Every failed attempt remains in the timing record.

AC.4 A learned choice of whole seeds

For a seed $\boldsymbol{\sigma}$, target batch $\mathcal{T}$, and candidate operator o, let $p_\theta(\boldsymbol{\sigma}, o, \mathcal{T})$ be the equation score. The equation filter retains the candidate when this score is at least τ. Its whole-seed variant instead uses

$$K_\theta(\boldsymbol{\sigma}, \mathcal{T}) = \mathbf{1}\left[\max_{o \in \mathcal{O}} p_\theta(\boldsymbol{\sigma}, o, \mathcal{T}) \geq \tau\right]. \tag{130}$$

Here $\mathcal{O}$ is the finite list of candidate operators, and $\mathbf{1}$ is one when its condition holds and zero otherwise. A maximum reaches τ exactly when at least one member reaches it. Therefore

Eq. (130) is the logical union of the original equation decisions. If $K_\theta = 1$, all operators from that seed remain available. The resulting set contains every equation that the source filter would retain, although it can contain more. This inclusion does not prove that the reduction will complete.

For L loop momenta and E independent external momenta, Kira's candidate list contains $L(L+E)$ IBP operators and $E(E-1)/2$ external Lorentz-invariance operators [4]. The first count follows by choosing a loop derivative and a contraction vector from the loop and external momenta. The second counts unordered pairs of external vectors for antisymmetric contractions. These are candidate counts before redundancies are removed. The seed filter takes the maximum over that full list. Its decision no longer depends on which operator is being considered, so it uses sixteen of the seventeen features. We evaluate the same logical decision directly during generation and check its equivalence to the original operator scores. Kira still enumerates the configured seed region, applies its native symmetries, and performs exact algebra.

AC.5 Ratios and uncertainty

For each complete matched batch, take the median of five times for each method, then sum the medians as in Eq. (123). This gives the learned-to-control ratio and the percentage saving $100(1-R)$. The active subset consists of batches for which the frozen domain and graph policy enable a filter. It uses the same ratio definition. We resample paired repetition positions within each batch 5,000 times and report the 2.5th and 97.5th percentiles. Failed reference checks, incomplete method comparisons, and identity-only targets remain outside the nontrivial paired ratios. Their counts remain in the main results.

AD Target partitions, cost learning, and exact checks

AD.1 Why target grouping changes the seed region

Let $\mathcal{T}$ be a finite batch of distinct index vectors from one family. For a vector $\boldsymbol{a}$, define its extra denominator powers and numerator rank as $\delta(\boldsymbol{a}) = \sum_i \max(a_i - 1, 0)$ and $\rho(\boldsymbol{a}) = \sum_i \max(-a_i, 0)$, as in Appendix AC. A partition $\Pi = \{\mathcal{T}_1, \ldots, \mathcal{T}_K\}$ consists of disjoint nonempty subsets whose union is $\mathcal{T}$. The number K is the number of separate reductions. For each part, the corresponding maxima obey

$$\max_{\boldsymbol{a}\in\mathcal{T}_k} \delta(\boldsymbol{a}) \le \max_{\boldsymbol{a}\in\mathcal{T}} \delta(\boldsymbol{a}), \qquad \max_{\boldsymbol{a}\in\mathcal{T}_k} \rho(\boldsymbol{a}) \le \max_{\boldsymbol{a}\in\mathcal{T}} \rho(\boldsymbol{a}). \tag{131}$$

These inequalities follow because each maximum on the left runs over a subset of the indices on the right. They explain how a partition can lower one or both seed limits. They do not imply a time saving: separate parts can generate the same intermediate equations and repeat the preparation. For example, one part can contain targets with large δ and zero ρ, while another contains targets with zero δ and large ρ. Their union requires both large bounds simultaneously. Kira's lower-sector truncation remains active in every comparison where the fixed seed policy requests it [5].

The five candidate partitions keep the batch together, group equal values of ρ, group equal values of δ, group equal sets of positive indices, or divide the given target sequence into consecutive groups of at most three. The first three group definitions depend on integral powers, and the fourth uses the set of positive indices that defines a sector. The last definition also depends on the order supplied by the caller. Every target occurs in exactly one part. The learned method selects one partition before equation generation and processes its parts sequentially with one Kira worker. An additional candidate keeps the batch together and disables the equation filter. Thus the cost predictor compares six calculations. This candidate allows a target-dependent decision to avoid a filter whose retry cost exceeds its saving. When a batch has at most three targets

and all targets have equal rank, equal extra denominator powers, and the same sector, all five partitions contain just that batch. If the earlier policy also disables its equation filter on this batch, the application skips loading the cost model and uses the undivided unfiltered request. This follows directly from the partition definitions and preserves the available reduction choice.

If a completed part gives $I_{\boldsymbol{a}} = \sum_m r_{\boldsymbol{a}m} M_m$ for every $\boldsymbol{a} \in \mathcal{T}_k$, then the union of these relations supplies the reduction of every target in $\mathcal{T}$. Indeed, disjointness assigns each target to one part, and the union property ensures that no target is missing. All parts use the same prescribed master basis. For a target expression $T = \sum_{\boldsymbol{a}\in\mathcal{T}} c_{\boldsymbol{a}} I_{\boldsymbol{a}}$, substitution and interchange of the finite sums gives

$$T = \sum_{k=1}^{K} \sum_{\boldsymbol{a}\in\mathcal{T}_k} c_{\boldsymbol{a}} \sum_m r_{\boldsymbol{a}m} M_m = \sum_m \left(\sum_{k=1}^{K} \sum_{\boldsymbol{a}\in\mathcal{T}_k} c_{\boldsymbol{a}} r_{\boldsymbol{a}m} \right) M_m. \tag{132}$$

The model changes the division of the calculation. Equation (132) shows how the exact part reductions provide the same requested expression.

AD.2 Physical requests and reference construction

The denominator momenta, masses, and rational kinematic profiles are those specified in Appendix AB. The ten structures are the sunrise, planar double box, massive three-loop vacuum, massive three-loop ladder, nonplanar box, pentagon, external-virtuality box, three- and four-loop boxes with bubbles, and four-loop massless two-point family. The dimension d remains symbolic. For the first seven structures, let D, R, X, and P denote the three-target sets with extra denominator powers, rank two, mixed powers, and a pinched propagator in Eq. (118). The requested batches are D, R, X, $D \cup R$, $P \cup R$, and $D \cup R \cup X \cup P$. The two bubble-box structures and the massless two-point family use the corresponding one-dot, one-numerator, and corner sets from that appendix, with the corner set also taking the place of P. Repeated index vectors and identical complete batches at a profile appear once. Development uses $h = 701$, training uses $h = 711$, validation uses $h = 712$, and the reserved comparison uses $h = 721, 722$.

Each component first receives a native reduction with expanded seed limits. We collect the resulting master indices into a common basis, reduce the components in that basis, and check each coefficient again after increasing the numerator limit by one. The checked component relations form the reference for each union. A failure in discovery, common-basis reduction, or the enlarged check excludes that batch from the timed comparison and remains in the record. Agreement with this finite enlargement does not prove a minimal basis. Reference construction remains outside the application timer; the timed request receives the basis and generates its own IBP equations.

AD.3 Predictor inputs and objective

The predictor uses the 27 family and target quantities specified in Appendix AB, together with five descriptions of the distribution of powers within the batch:

$$|\{\delta(\boldsymbol{a}) : \boldsymbol{a} \in \mathcal{T}\}|, \quad |\{\rho(\boldsymbol{a}) : \boldsymbol{a} \in \mathcal{T}\}|, \quad \delta_{\max} - \delta_{\min}, \quad \rho_{\max} - \rho_{\min}, \quad \frac{1}{|\mathcal{T}|} \sum_{\boldsymbol{a}\in\mathcal{T}} \delta(\boldsymbol{a})\rho(\boldsymbol{a}). \tag{133}$$

Here $|\cdot|$ denotes the number of members, and the maxima and minima run over the batch. The last quantity distinguishes targets that carry both kinds of power from a batch in which different targets carry them separately. No reference coefficient, measured test time, or kinematic profile number enters the features.

Candidate definitions can produce identical ordered parts on a particular batch. Such candidates share the same measured training or validation cost, and the record identifies the common measurement. The reserved comparison still executes every complete application method separately, including its own selection and loading costs.

For training batch b and partition a, let $\bar{T}_{ba}$ be the median of two complete timings when both reductions succeed. A failed candidate receives a training cost equal to the larger of 300 seconds and the sum of its measured attempt times. This chosen penalty makes a bounded failure costly to the predictor; it is not an observed completion time. With $a = 0$ denoting the undivided batch, the labels and normalized weights are

$$y_{ba} = \log \frac{\bar{T}_{ba}}{\bar{T}_{b0}}, \qquad u_b = \sqrt{\min_a \bar{T}_{ba}}, \qquad w_b = \frac{u_b}{B^{-1} \sum_{b'=1}^{B} u_{b'}}. \tag{134}$$

The positive integer B is the number of usable training batches. The normalization gives $B^{-1} \sum_b w_b = 1$ by direct substitution. The fitted multi-output regressors use weighted squared error,

$$\mathcal{L}(\theta) = \sum_{b=1}^{B} w_b \sum_a \left[f_{\theta,a}(x_b) - y_{ba} \right]^2. \tag{135}$$

Here x_b is the feature vector, θ denotes the fitted regressor parameters, and $f_{\theta,a}$ is the predicted log-cost ratio for partition a. Since $y_{ba} - y_{ba'} = \log(\bar{T}_{ba}/\bar{T}_{ba'})$ and the logarithm is increasing, the smallest exact label corresponds to the smallest assigned cost. Fitting Eq. (135) approximates these labels; it supplies no guarantee that a prediction is optimal on a new batch.

We fit eighteen candidates: regression trees, random forests with 32 trees, and extremely randomized forests with 32 trees, each with maximum depth 2, 4, or 6 and minimum leaf population 1 or 3 [9, 10, 11]. The implementation uses Ref. [12]. Three complete validation repetitions select a fixed partition c_g for each graph g. A model proposes $a_\star = \arg\min_a f_{\theta,a}(x)$ when

$$f_{\theta,a_\star}(x) < f_{\theta,c_g}(x) - \gamma, \qquad \gamma \in \{0, 0.02, 0.05, 0.1\}; \tag{136}$$

otherwise it uses c_g. Validation chooses the model and margin together. Deployment requires at least a three-percent saving in summed validation cost relative to the graph-specific fixed partitions. A separate validation run with every ML model disabled selects the fixed control for that condition. Ordered physical structures and target-count and power ranges constrain deployment; an unmatched input retains an undivided batch.

To measure the remaining opportunity within this candidate pool, we also sum the smallest recorded cost for each validation batch:

$$T_{\text{min,recorded}} = \sum_b \min_a \bar{T}_{ba} \le \sum_b \bar{T}_{b,c_{g(b)}} = T_{\text{fixed}}. \tag{137}$$

Here $g(b)$ is the graph of batch b, and the sums run over validation batches. Each minimum is no greater than the chosen fixed candidate's cost, which proves the inequality after summation. This retrospective quantity uses measured costs that a new request does not supply. It is neither an achieved prediction time nor a lower bound on all possible reduction methods.

AD.4 Timed scope and failures

The application loads the policy for each request. Each part uses the fixed seed configuration for its physical graph. The previous equation filter is available only inside its original checked domain, with its weights and threshold unchanged. An insufficient filtered system retries without the filter, then uses unshifted and expanded seed limits where distinct. A failed partition can retry the undivided unfiltered calculation if time remains. The common request budget is 150 seconds, and each native attempt has at most 60 seconds. A request that exhausts this budget remains a failure.

The reserved test rotates five methods through five repetition positions: the previous undivided workflow, that workflow with every ML model disabled, the new graph-specific fixed partitions

with filters available, independently tuned fixed partitions with every ML model disabled, and the learned partition choice with filters available. Every method uses the same exact implementation, worker count, basis, and coefficient checks. Initial Python imports precede these repeated timers. The separate application check starts a new Python process and includes imports, output, and the exact comparison.

For deployment, we retain the frozen domain limits, fixed choices, and prediction margin. We retain the tree arrays only when validation enables the predictor. When the predictor is disabled, its choice rule returns before consulting those arrays, so removing them leaves each choice unchanged. We check equality of the returned choices on every training, validation, and reserved input. This check uses no reserved timing or coefficient outcome. The repeated test retains its original frozen implementation; the fresh-process application check uses the compact deployment form. All ML-disabled controls keep their existing behavior.

We sum per-batch median times over requests for which all methods and all repetitions succeed and whose output is not entirely the identity. Failed references, incomplete timed batches, and identity-only batches remain separate. The ratio of two such sums compares the same completed workload. Paired resampling of the five repetition positions within each batch gives 5,000 ratios; the 2.5th and 97.5th percentiles describe timing variation for this specified collection. They do not establish transfer to untested topologies or loop orders. Some index sets recur across rational profiles, and the report records this overlap.

Independent graph groups run on two workers during data collection and four workers during the reserved comparison. This shortens the experiment without parallelizing an individual timed reduction. Every reserved batch keeps all methods and repetitions on the same worker, with the same rotated order described above. Complete training measurements made before this scheduling change remain in the training record.

If a cluster allocation ends during a comparison, we retain each complete batch and repeat every method and repetition for an unfinished batch on one replacement worker. The saved partial comparison remains in a separate archive and does not enter the timing ratios. We assign the remaining whole batches from validation costs and keep the frozen models, global method rotation, exact checks, and timing definitions unchanged. The audit checks the worker identity for every completed comparison batch.

AE An exact reduction for massless banana integrals

AE.1 Definition and convolution

Consider the massless two-point family

$$B_L(\boldsymbol{a}; p^2, d) = \int \prod_{j=1}^{L} \frac{d^d \ell_j}{i\pi^{d/2}} \frac{1}{\prod_{j=1}^{L} (\ell_j^2 + i0)^{a_j} [(p - \sum_{j=1}^{L} \ell_j)^2 + i0]^{a_{L+1}}}. \tag{138}$$

Here $L \geq 1$, all a_i are positive integers, and $p^2 \neq 0$. The family has two vertices joined by $L+1$ massless lines and is often called a banana or watermelon diagram. The implemented specialization covers the displayed scalar integrals. Auxiliary denominator entries may be present in the Kira definition, but their target indices must be zero. The proof does not cover arbitrary numerator insertions or other propagator structures.

The one-loop convolution underlying this calculation is standard [13]. For completeness, write its Euclidean form as

$$E_1(a, b; p_E^2) = \int \frac{d^d k_E}{\pi^{d/2}} (k_E^2)^{-a} [(p_E - k_E)^2]^{-b}. \tag{139}$$

Here k_E and p_E are Euclidean loop and external momenta, and a, b are the two denominator powers. The Euler Gamma function is $\Gamma(z) = \int_0^\infty \eta^{z-1} e^{-\eta} d\eta$ for $\operatorname{Re} z > 0$. Changing the

integration variable to $\eta = tD$ gives the Schwinger representation $D^{-a} = \Gamma(a)^{-1} \int_0^\infty t^{a-1} e^{-tD} dt$ for positive D in its convergence domain. Applying this representation to both denominators and completing the square gives

$$tk_E^2 + u(p_E - k_E)^2 = (t+u)\left(k_E - \frac{u}{t+u} p_E\right)^2 + \frac{tu}{t+u} p_E^2. \tag{140}$$

For $\alpha > 0$, squaring $\int_{\mathbb{R}} e^{-\alpha x^2} dx$ gives a two-dimensional Gaussian integral. Polar coordinates evaluate it as $2\pi \int_0^\infty r e^{-\alpha r^2} dr = \pi/\alpha$. The positive square root and the product over momentum components give the normalized Gaussian momentum integral $(t+u)^{-d/2}$, continued in d from integer dimension. Thus the Schwinger parameters $t, u > 0$ give

$$E_1 = \frac{1}{\Gamma(a)\Gamma(b)} \int_0^\infty dt\, du\, t^{a-1} u^{b-1} (t+u)^{-d/2} \exp\left[-\frac{tu}{t+u} p_E^2\right]. \tag{141}$$

Set $t = \lambda x$, $u = \lambda(1-x)$, with Jacobian λ, $\lambda > 0$, and $0 < x < 1$. The λ integral gives $\Gamma(a+b-d/2)[x(1-x)p_E^2]^{d/2-a-b}$. The remaining integral is $\int_0^1 x^{d/2-b-1}(1-x)^{d/2-a-1} dx$. To evaluate it, apply the same sum-and-fraction change of variables to the two integrals defining $\Gamma(z)\Gamma(w)$. The sum integral gives $\Gamma(z+w)$, which yields

$$\int_0^1 x^{z-1}(1-x)^{w-1} dx = \frac{\Gamma(z)\Gamma(w)}{\Gamma(z+w)} \quad (\operatorname{Re} z, \operatorname{Re} w > 0). \tag{142}$$

Taking $z = d/2 - b$ and $w = d/2 - a$ then gives

$$E_1 = (p_E^2)^{d/2-a-b} G(a,b), \qquad G(a,b) = \frac{\Gamma(a+b-d/2)\Gamma(d/2-a)\Gamma(d/2-b)}{\Gamma(a)\Gamma(b)\Gamma(d-a-b)}. \tag{143}$$

The parameter integrations first hold in a common convergence region. Analytic continuation defines the dimensionally regulated expression outside that region, as in the usual one-loop formula [13].

Let E_L be the Euclidean version of Eq. (138), with measure $\prod_j d^d \ell_{j,E}/\pi^{d/2}$, and write $n = L+1$ and $A = \sum_{i=1}^n a_i$. Integrating the last loop combines the last two indices into $b = a_L + a_{L+1} - d/2$. Applying Eq. (143) and then the $(L-1)$-loop formula cancels the factors $\Gamma(b)$ and $\Gamma(d - a_L - a_{L+1})$ between the two stages. Induction from $L = 1$ therefore gives

$$E_L(\boldsymbol{a}; p_E^2, d) = (p_E^2)^{Ld/2-A} \frac{\Gamma(A - Ld/2)}{\Gamma(nd/2 - A)} \prod_{i=1}^n \frac{\Gamma(d/2 - a_i)}{\Gamma(a_i)}. \tag{144}$$

Wick rotation to the denominator convention in Eq. (138) gives one factor of i per loop measure, cancelling its normalization, and changes the sign of each quadratic denominator. It therefore replaces the power factor by $(-1)^A(-p^2 - i0)^{Ld/2-A}$. The remaining Gamma factors stay unchanged. This sign convention matters when reducing an integral with an additional denominator power.

AE.2 Rational coefficients in a single master

Choose $M_L = B_L(1, \ldots, 1; p^2, d)$ and define $b_i = a_i - 1 \geq 0$ and $K = \sum_i b_i = A - n$. The Gamma function recurrence gives

$$\frac{\Gamma(z+K)}{\Gamma(z)} = (z)_K, \qquad \frac{\Gamma(z)}{\Gamma(z-K)} = (-1)^K (1-z)_K, \tag{145}$$

where $(z)_K = \prod_{j=0}^{K-1}(z+j)$ and $(z)_0 = 1$. Integration by parts in the defining Gamma integral gives $\Gamma(z+1) = z\Gamma(z)$ when the boundary terms vanish; analytic continuation extends the relation.

Applying this recurrence repeatedly gives the first identity. Reversing the finite product and factoring out -1 from each factor gives the second. In the ratio of Eq. (144) to its corner value, the dimension-independent index factors give $\Gamma(1)/\Gamma(a_i) = 1/b_i!$. The shifted factors satisfy

$$\prod_i \frac{\Gamma(d/2-1-b_i)}{\Gamma(d/2-1)} = \frac{(-1)^K}{\prod_i (2-d/2)_{b_i}}. \tag{146}$$

This sign cancels the sign from the inverse Gamma ratio. The remaining Minkowski power factor is $(-1)^K(-p^2-i0)^{-K} = (p^2+i0)^{-K}$ because K is an integer. Thus the required reduction is

$$\boxed{B_L(\boldsymbol{a};p^2,d) = \frac{(n-Ld/2)_K\,(n+1-nd/2)_K}{(p^2)^K \prod_{i=1}^n (2-d/2)_{b_i}\, b_i!}\, M_L.} \tag{147}$$

We understand the rational coefficient by meromorphic continuation in d and generic nonzero p^2. The corner value is generically nonzero, and Eq. (147) expresses every scalar target in this class through it. This establishes a one-element basis for the stated scalar class without a new master-discovery calculation.

The implementation forms finite products of linear factors in d and cancels common factors with exact rational arithmetic before formatting the result. It uses no trained model to predict a coefficient. The domain check verifies the physical denominator momenta, zero masses, the single external squared momentum, positive physical indices, and zero auxiliary indices. If the caller supplies a basis, that basis must contain M_L. The implementation caps the loop count at 64 and the total extra powers at 256 to bound work; the mathematical identity holds for every positive integer loop count under the stated assumptions.

AE.3 Independent comparisons

The native comparison uses three targets at each of one through six loops, with rational squared momentum $p^2 = -1651$, and repeats the calculation three times. A second comparison keeps p^2 symbolic at one through four loops. The requested indices raise one physical power by one or two. Kira receives the corner basis, generates native IBP equations with all ML disabled, and returns rational coefficients. Exact subtraction checks these coefficients against Eq. (147). Both reported timings include their coefficient check; the direct formula also includes its domain check. Shared Python imports precede these timings.

Eight initial higher-loop requests fail, including corner-basis rejections and an allocation failure in the six-loop preparation. We retain them and make a separate fixed-seed comparison. Let r be the largest sum of positive target indices, s the largest numerator rank, and q the largest number of extra denominator powers. Development at $p^2 = -1649$ compares seed bounds $(r-1,s,q-1)$, (r,s,q), $(r,s+1,q)$, and $(r+1,s+1,q+1)$; the positive-power limit never falls below the number of physical denominators. Symbolic development changes the raised propagator positions at the same index profile. For each loop count and kinematic form, the fastest successful complete development calculation fixes the seed choice. A new comparison at $p^2 = -1655$, or symbolic p^2 with the new index profile, uses three repetitions of that fixed choice. Timed failures remain separate from matched speedup ratios.

Further checks use one through twelve loops and loop counts 16, 24, and 32, with eight distributions of one through eight extra powers at each loop count. The direct rational result is compared with the uncancelled Gamma ratio from Eq. (144) at $d = 257/101$ and $d = 401/97$. The evaluation uses 70-digit arithmetic and a relative error threshold of 10^{-55} with denominator $\max(1,|r|)$, where r is the Gamma-ratio value. These numerical checks supplement the derivation. Native timing ratios are reported only where the native calculation completes; no native speedup is assigned to the higher-loop cases checked by the formula alone.

The combined-interface application check uses a fresh process for each request. Four scalar banana requests at one, four, eight, and twelve loops use an independent symbolic implementation of Eq. (147) as their reference. It constructs rising factorials directly and does not call the live factor-cancellation routine. The native application requests use the independently checked IBP references of the target-batch study. All reference construction precedes application timing. Both comparison strategies use the same analytic route; their difference on native requests comes from the frozen learned or fixed reduction choices. Exact rational checks at $d = 257/101$ also test the implemented power limit: one- and 64-loop targets have 256 extra powers on one denominator. A direct product calculation checks the factored result without expanding its polynomials. Further checks verify the corner identity and reject requests beyond the implemented loop or power limits.

AF Cost-trained seed regions

AF.1 The selected integral lattice

We specify a seed by its integer index vector $\boldsymbol{\sigma}$. Let P be its positive-index set, let n_{top} be the largest number of propagators in a declared top sector, and let $h = n_{\mathrm{top}} - |P|$ be the number of absent propagators relative to that count. The number of extra denominator powers and the numerator rank are

$$D(\boldsymbol{\sigma}) = \sum_i \max(0, \sigma_i - 1), \qquad R(\boldsymbol{\sigma}) = \sum_i \max(0, -\sigma_i). \tag{148}$$

For a target $\boldsymbol{a}$, write $v_i = \max(0, -a_i)$, while $u_i = \max(0, -\sigma_i)$ denotes a seed numerator power. We measure how far the seed lies outside the target's numerator-power box by

$$q(\boldsymbol{\sigma}, \mathcal{T}) = \min_{\boldsymbol{a} \in \mathcal{T}} \sum_{i \notin P} \max(0, u_i - v_i). \tag{149}$$

Here $\mathcal{T}$ is the requested target set. To obtain this expression, project each coordinate u_i onto the interval $[0, v_i]$. Since $u_i \geq 0$, the shortest coordinate distance is $\max(0, u_i - v_i)$. Adding these distances gives the distance in the sum-of-absolute-values norm, and minimizing over the targets gives the distance to their union of boxes. With several numerator directions this union can contain a box with a nonzero interior; the construction does not impose the zigzag paths of Ref. [8].

Let R_T and D_T be the maximum target rank and extra denominator power. A parameter vector $\boldsymbol{\theta} = (\theta_0, \ldots, \theta_6)$ defines the additional inequalities

$$q(\boldsymbol{\sigma}, \mathcal{T}) \leq \max(0, \lfloor \theta_0 + \theta_1 h + \theta_2 D(\boldsymbol{\sigma}) + \theta_6 R(\boldsymbol{\sigma}) \rfloor), \tag{150}$$

$$R(\boldsymbol{\sigma}) \leq \max(0, \lfloor R_T + \theta_3 - \theta_4 h \rfloor), \tag{151}$$

$$D(\boldsymbol{\sigma}) \leq \max(0, \lfloor D_T + \theta_5 \rfloor). \tag{152}$$

The outer Kira bounds and its fixed subsector truncation also apply. The generator adds admissible target and master seeds and retains sector corners. These additions preserve useful starting identities but do not prove that the selected system is sufficient. The comparisons keep all positive target powers equal to one, so $D_T = 0$. The dot-dependent extension therefore remains untested; five boundary parameters affect the numerator-only search, while $\theta_5 = 1$ remains fixed.

The geometric restriction can reduce the growth with numerator rank. For one numerator direction of degree R_T, suppose a surrounding width w stays fixed and there are $n_\perp$ other numerator directions. Distributing at most w powers over these transverse directions gives $\binom{n_\perp + w}{w}$ assignments. This count follows by adding a nonnegative slack variable to make the sum equal to w and counting the resulting compositions. The parallel coordinate has at most $R_T + w + 1$

values, so

$$N_{\text{seed}} \le (R_T + w + 1)\binom{n_\perp + w}{w} \tag{153}$$

for this fixed-sector, fixed-denominator-power example. Thus the bound grows linearly in R_T when w and $n_\perp$ remain fixed. A changing learned width or several simultaneously powered numerator directions need not have this scaling. The timing results use measured reductions and do not infer a general speedup from this counting argument.

AF.2 Training on completed reductions

For each graph, the first search starts from complete Kira reductions at three training targets. It evaluates twenty-four boundary proposals through three populations. The four proposals with the lowest mean log-time ratio determine the mean and component standard deviations for the next Gaussian population, with a minimum standard deviation of 0.2. We round parameters to quarter units and clip them to the chosen ranges: $\theta_0 \in [0, 3]$, $\theta_1, \theta_2 \in [-1, 1]$, $\theta_3 \in [-1, 2]$, $\theta_4 \in [0, 2]$, and $\theta_6 \in [-1, 1/2]$, with $\theta_5 = 1$. These choices define a finite search and do not establish a globally optimal seed set. Evolutionary optimization of IBP seed decisions is also used in Ref. [8]; the present search directly measures complete Kira time with symbolic dimension.

For training request b, let $T_b(\boldsymbol{\theta})$ include seed prediction, every reduction attempt, recovery, and the exact reference comparison. Let $T_b^{(0)}$ be the corresponding time with the tuned native seed settings. The objective is

$$J(\boldsymbol{\theta}) = \frac{1}{B}\sum_{b=1}^{B} \log \frac{T_b(\boldsymbol{\theta})}{T_b^{(0)}}. \tag{154}$$

Exponentiating the mean gives the geometric mean of the complete-time ratios, which explains the equal weight assigned to each request. A failed bounded reduction receives a cost of 300 seconds when forming the objective. This is a training penalty, not a successful measured reduction time. Successful recovery contributes its actual elapsed time.

The first planar-box fit selects

$$\boldsymbol{\theta}^* = (7/4,\ 1/4,\ 1/4,\ 3/4,\ 0,\ 1,\ -1/4). \tag{155}$$

The fixed control uses $(1, 0, 0, 1, 0, 1, 0)$, and the fitted vector allows the seed-distance bound to change with rank through its last component. Its second component changes the allowed width after a physical propagator is removed. Both choices retain the same exact Kira algebra and recovery procedure.

The second stage adds twelve training requests per graph, covering ranks two, three, and four with one or two powered auxiliary entries at two new rational profiles. It compares the preceding fitted boundaries, independently varied rank allowances, two fixed surrounding widths, and unfiltered Kira. Six regression models comprise trees, random forests, and extremely randomized forests with depths two or four; forests contain 32 trees and every leaf has at least two training examples [9, 10, 11]. Their targets are the measured log-time ratios for each boundary. Inputs include the physical family and index summaries of Appendix AB, the individual numerator and extra denominator powers, and the number of powered numerator directions. Reference coefficients do not enter these inputs.

Ten separate validation requests per graph cover ranks two through four. Repeated complete reductions select a fixed sparse boundary for each graph and compare it with the cost predictors. If $\widehat{y}_j$ is a predicted log cost and f is the fixed choice, the model selects candidate j only when

$$\widehat{y}_j < \widehat{y}_f - \eta, \qquad \eta \in \{0, 0.03, 0.1\}. \tag{156}$$

Exponentiating the difference gives the equivalent predicted ratio condition $\exp(\widehat{y}_j - \widehat{y}_f) < \exp(-\eta)$. Validation selects the margin and enables the cost predictor only when its total recorded

time is at least three percent lower than the separately selected fixed sparse choice. Otherwise the deployment retains that fixed choice. The validation ranking uses recorded candidate costs; the independent application comparisons also include model loading and forecasting. This gate is a selection rule, rather than a proof of a future saving.

AF.3 Exactness, controls, and physical coverage

Removing seeds removes rows from an exact IBP system. Every remaining row still follows from an integration-by-parts identity, and exact row combinations preserve those identities. The final coefficients therefore remain valid if the selected system reaches the prescribed basis. Kira checks basis membership, and the experiments compare every returned coefficient with a separately computed unfiltered reduction. An insufficient sparse system first repeats the same outer region without the sparse restriction; native and enlarged bounds follow when required. All repeated work enters the complete-time measurement.

The first independent test uses eight planar-box requests at new rational profiles. The second uses ten requests each for the planar and nonplanar boxes. Both retain symbolic d and a supplied master basis. The physical denominators follow Appendix N; each target has unit positive physical indices and total auxiliary numerator rank two, three, or four. A single-direction target puts the full rank on one auxiliary entry. A distributed target splits it into $\lceil R_T/2 \rceil$ and $\lfloor R_T/2 \rfloor$ powers on two entries. The chosen auxiliary entry changes between the two profiles. Some index patterns recur across training and testing, so these comparisons test new kinematics within known structures and target patterns. Master discovery precedes the repeated timer.

Let κ be an integer that labels a kinematic profile. In a common squared-momentum unit, the profiles are defined by

$$\begin{aligned}(s,t)_{\text{planar}} &= (-1009-12\kappa, -1021-14\kappa),\\ (s,t)_{\text{nonplanar}} &= (-113-18\kappa, -127-10\kappa).\end{aligned} \tag{157}$$

These are chosen kinematic inputs, with the graph-specific scalar-product conventions in Appendix N. The first search uses $\kappa = 811, 812$ for training, $\kappa = 813, 814$ for validation, and $\kappa = 821, 822$ for testing. The larger cost study uses $\kappa = 831, 832$, $\kappa = 841, 842$, and $\kappa = 851, 852$ for those three splits. A single-direction target places its numerator on auxiliary index $8 + (\kappa \bmod 2)$. A distributed target places the ceiling power on $8 + ((\kappa + 1) \bmod 2)$ and the floor power on the other auxiliary index. Together with the seven unit physical powers, these rules specify the tested integrals. The second reserved set contains single-direction ranks two, three, and four, and distributed ranks three and four at each profile.

Seven repetitions rotate method order on the same worker for every request, using one Kira thread. Different requests can use different cluster workers. The native control disables all learned decisions and keeps the tuned outer bounds. The fixed sparse control retains one validation-selected boundary per graph, including boundaries found during training, and performs no cost prediction. It therefore isolates the additional value of request-dependent prediction. The previous application uses the preceding published batch and equation policies. Failed references and incomplete calculations remain in the record and outside matched successful timing ratios. Paired repetition resampling describes timing variation on this fixed workload; it does not establish transfer to arbitrary graphs, masses, symbolic invariants, or higher loop orders.

AG Joint configuration choice and exact scale transformation

AG.1 Inputs, candidate reductions, and training objective

The integral definitions are the planar and nonplanar massless two-loop boxes in Appendix AF. We keep their propagator order and supplied master indices. Every target has seven unit positive

physical powers and two auxiliary indices whose nonpositive powers sum to numerator rank $R \in \{2, 3, 4\}$. For a single powered auxiliary entry, its position is $8 + (\kappa \bmod 2)$. For two powered entries, that position has power $-\lfloor R/2 \rfloor$, and the other has power $-\lceil R/2 \rceil$. Here κ labels the rational kinematic profile. The planar profiles have $(s, t) = (-1009 - 12\kappa, -1021 - 14\kappa)$, and the nonplanar profiles have $(s, t) = (-113 - 18\kappa, -127 - 10\kappa)$.

Development uses profile 901 with the single-entry rank-three target and both rank-four patterns. For each graph, training uses profiles 911 and 912, validation uses 921 and 922, and testing uses 931 and 932. Each of these profiles contributes both numerator patterns at all three ranks. Thus each graph has twelve training, twelve validation, and twelve test requests. These splits separate kinematic values while retaining known graphs and some recurring index patterns.

The sparse boundary is defined in Appendix AF; its seven parameters retain the same order. Development compares the native region, the preceding application, eight integral orderings with a fixed sparse boundary, four alternative boundaries, and two internal propagator-permutation settings. The planar fixed boundary is $(1, 0, 0, -1, 0, 1, 0)$, while the nonplanar fixed boundary is $(1, 0, 0, 0, 1, 1, 0)$. The four alternatives are $(0, 0, 0, 0, 0, 1, 0)$, $(1, 0, 0, -2, 0, 1, 0)$, $(2, 0, 0, 0, 0, 1, 0)$, and $(1, 0, 0, 1, 0, 1, -1/4)$. The alternative boundaries use integral ordering 1. The two permutation settings also use ordering 1 and the fixed boundary. These are available Kira configuration choices [7, 5].

For each graph, the candidate set retains the four fastest aggregate development configurations and the fastest configuration for each individual development target. Duplicate configurations are removed, and the native region is included. The preceding application is measured separately as a baseline. Each candidate uses exactly the same retry sequence: an unfiltered attempt with native integral ordering at the tuned bounds, followed by native and expanded bounds when required. Duplicate attempts are skipped.

Let T_{ij} be the complete measured time for training request i and candidate j, including any failed attempt and recovery. We use the dimensionless response

$$y_{ij} = \log\left(\frac{T_{ij}}{T_{i0}}\right), \qquad \widehat{\boldsymbol{y}}_i = f_{\boldsymbol{w}}(\boldsymbol{z}_i), \tag{158}$$

where candidate 0 is the native control, $\boldsymbol{z}_i$ contains the 68 target and family features defined in Appendix AF, and $\boldsymbol{w}$ denotes the fitted tree parameters. We fit each regression tree by reducing squared prediction error, as in the established tree and ensemble methods [9, 10, 11]. Dividing by T_{i0} puts different request costs on a common relative scale. Because the logarithm is strictly increasing, minimizing the exact y_{ij} for a fixed request selects the same candidate as minimizing T_{ij}.

The fitted alternatives are a single regression tree, a random forest of 48 trees, and an ensemble of 48 extremely randomized trees. Each uses maximum depth 2, 3, or 4 and a minimum leaf size of one training request. These choices give nine fitted models per graph. Validation tests each model with decision margin $\delta \in \{0, 0.03, 0.08\}$. If j_* minimizes the predicted response and j_f is the fastest fixed validation configuration, the choice is

$$j_{\mathrm{use}} = \begin{cases} j_*, & \widehat{y}_{ij_*} < \widehat{y}_{ij_f} - \delta, \\ j_f, & \text{otherwise.} \end{cases} \tag{159}$$

This condition follows by comparing the two predicted log costs; exponentiation gives a predicted cost ratio below $e^{-\delta}$. A second validation condition requires at least 2% less summed time than the fixed choice before prediction is enabled. Deployment also requires at least 2% less time than the preceding application. Otherwise that application remains in use. Validation ranks the choices from recorded candidate-reduction times, while the frozen tests also measure actual model loading and prediction. These margins control deployment; they are not confidence intervals or proofs of a gain on an untested request.

AG.2 Derivation of exact reuse under a common scale change

Consider a massless family whose inverse propagators are homogeneous quadratic functions of loop and external momenta. Define $\nu(\boldsymbol{a}) = \sum_i a_i$, including negative numerator indices. Scale every external scalar product by the same positive number λ, and change integration variables to $\ell_h = \sqrt{\lambda}\, k_h$. Each loop measure supplies $\lambda^{d/2}$ and each inverse propagator supplies λ, which gives

$$\begin{aligned} I_{\boldsymbol{a}}(\lambda\boldsymbol{x}, d) &= \int \prod_{h=1}^{L} \frac{\lambda^{d/2} d^d k_h}{i\pi^{d/2}} \prod_i [\lambda D_i(\boldsymbol{k}, \boldsymbol{p})]^{-a_i} \\ &= \lambda^{Ld/2-\nu(\boldsymbol{a})} I_{\boldsymbol{a}}(\boldsymbol{x}, d). \end{aligned} \tag{160}$$

The positive scale preserves the signs of the invariants and the Feynman prescription. The identity extends by analytic continuation in d wherever the original dimensionally regulated reduction applies.

Suppose an exact reduction is already available at $\boldsymbol{x}$:

$$I_{\boldsymbol{a}}(\boldsymbol{x}, d) = \sum_m r_{\boldsymbol{a}m}(\boldsymbol{x}, d) I_{\boldsymbol{b}_m}(\boldsymbol{x}, d), \tag{161}$$

where $\boldsymbol{b}_m$ is the index vector of master m. Substituting Eq. (160) for the target and each master into Eq. (161) yields

$$\begin{aligned} I_{\boldsymbol{a}}(\lambda\boldsymbol{x}, d) &= \sum_m \lambda^{Ld/2-\nu(\boldsymbol{a})} r_{\boldsymbol{a}m}(\boldsymbol{x}, d) \lambda^{-Ld/2+\nu(\boldsymbol{b}_m)} I_{\boldsymbol{b}_m}(\lambda\boldsymbol{x}, d) \\ &= \sum_m \lambda^{\nu(\boldsymbol{b}_m)-\nu(\boldsymbol{a})} r_{\boldsymbol{a}m}(\boldsymbol{x}, d) I_{\boldsymbol{b}_m}(\lambda\boldsymbol{x}, d). \end{aligned} \tag{162}$$

Thus the dimension-dependent powers cancel between target and master, leaving the integer exponent in Eq. (4). Rational positive λ gives an exact rational multiplier, so the transformation needs neither a new IBP system nor an approximation to d.

The implementation applies this identity only to the two verified massless graph definitions. It compares every external scalar product after division by the absolute value of the first nonzero one, and also requires the same propagator definitions, target indices, and master basis. This checks all dimensionless ratios. Symbolic scalar products, nonzero masses, and changed invariant ratios bypass reuse. Each stored result comes from a completed exact reduction in the current application session. The measured sequences use scale factors 1, 3/2, and 2, with the initial reduction included in the total time. Independent native calculations at all three scales check the transformed coefficients.

References

[1] Stefan Weinzierl. *Feynman Integrals: A Comprehensive Treatment for Students and Researchers.* UNITEXT for Physics. Springer, 2022. arXiv:2201.03593; publisher DOI.

[2] K. G. Chetyrkin and F. V. Tkachov. Integration by parts: The algorithm to calculate β-functions in 4 loops. *Nuclear Physics B*, 192:159–204, 1981. publisher DOI.

[3] Stefano Laporta. High-precision calculation of multiloop Feynman integrals by difference equations. *International Journal of Modern Physics A*, 15:5087–5159, 2000. arXiv:hep-ph/0102033.

[4] Philipp Maierhöfer, Johann Usovitsch, and Peter Uwer. Kira—a Feynman integral reduction program. *Computer Physics Communications*, 230:99–112, 2018. arXiv:1705.05610.

[5] Fabian Lange, Johann Usovitsch, and Zihao Wu. Kira 3: Integral reduction with efficient seeding and optimized equation selection. *Computer Physics Communications*, 322:109999, 2026. arXiv:2505.20197.

[6] Zhuo-Yang Song, Tong-Zhi Yang, Qing-Hong Cao, Ming-xing Luo, and Hua Xing Zhu. Explainable AI-assisted optimization for Feynman integral reduction, 2025. arXiv:2502.09544.

[7] Jonas Klappert, Fabian Lange, Philipp Maierhöfer, and Johann Usovitsch. Integral reduction with Kira 2.0 and finite field methods. *Computer Physics Communications*, 266:108024, 2021. arXiv:2008.06494.

[8] Justin Berman, Francois Charton, Andres Luna, Matthias Wilhelm, and Mao Zeng. Efficient ai-inspired reduction of feynman integrals via tube seeding, 2026. arXiv:2606.10698.

[9] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Chapman & Hall, 1984. Publisher record.

[10] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.

[11] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63:3–42, 2006.

[12] Fabian Pedregosa, Gael Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. Journal article.

[13] A. G. Grozin. Lectures on multiloop calculations. *International Journal of Modern Physics A*, 19:473–520, 2004.

[14] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander Smola. Deep sets. In *Advances in Neural Information Processing Systems*, volume 30, 2017. arXiv:1703.06114.

[15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. arXiv:1706.03762.

[16] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. arXiv:1711.05101.

[17] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015.

[18] Roman N. Lee. LiteRed 1.4: a powerful tool for the reduction of the multiloop integrals. *Journal of Physics: Conference Series*, 523:012059, 2014.

[19] Philipp Maierhöfer and Johann Usovitsch. Kira 1.2 release notes, 2018. arXiv:1812.01491.

[20] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. publisher DOI.

[21] Richard Bellman. On the theory of dynamic programming. *Proceedings of the National Academy of Sciences*, 38(8):716–719, 1952. publisher DOI.

[22] Leo Breiman. Bagging predictors. *Machine Learning*, 24:123–140, 1996. Publisher record.

[23] Christian Bogner and Stefan Weinzierl. Feynman graph polynomials, 2010. arXiv:1002.3458.